DMA 设计方法学与自动化

10 天补考研习手册·NM6004 中文学习版

2026-11-08

目录

1 如何使用这份手册 6
1.1 时间与取舍 . . . 6
1.2 关键问题—先修知识—模块—练习/实验—来源 . . . 6

2 10 天路线与能力证据 9
2.1 Day 1 ·先诊断，再修复布尔基础 . . . 9
2.2 Day 2 ·精确最小化：把步骤做稳 . . . 9
2.3 Day 3 ·启发式、共享逻辑与函数分解 . . . 9
2.4 Day 4 ·ROBDD 与 FSM：图不是装饰 . . . 10
2.5 Day 5 ·事件仿真与 VHDL：掌握时间语义 . . . 10
2.6 Day 6 ·故障覆盖与布尔差分 . . . 10
2.7 Day 7 ·D 算法、故障树和扫描设计 . . . 11
2.8 Day 8 ·第一轮整卷：按证据分配修复时间 . . . 11
2.9 Day 9 ·第二轮模拟：迁移与速度 . . . 11
2.10 Day 10 ·收口：可用的知识与稳定状态 . . . 12

3 01 课程地图与数字设计的三种视角 12
3.1 你要掌握的主线：表示、变换、验证 . . . 12
3.2 Y-chart：域与抽象层级是两根不同坐标 . . . 13
3.3 设计流程：每次转换都要带上约束 . . . 13
3.4 贯穿案例：一位全加器的等价实现 . . . 13
3.5 仿真、形式验证、制造测试：证据不能互相替代 . . . 13
3.6 练习：先作答，再查附录 . . . 14
3.7 本章来源 . . . 14

4 02 布尔空间、cube 与 Shannon 展开 14
4.1 先统一符号：同一个“加号”有两种语义 . . . 15
4.2 cube：少一个 literal，就多一个自由维度 . . . 15
4.3 On/off/DC 与 SOP/POS 的转换 . . . 15
4.4 完整示范：固定输入、重建函数、得到 MUX . . . 15
4.5 依赖、单调、量词与布尔差分 . . . 16
4.6 你该怎样在纸上验证自己 . . . 16
4.7 练习：先作答，再查附录 . . . 16
4.8 本章来源 . . . 17

5 03 精确两级综合：从全部 PI 到最小覆盖 17
5.1 先分清四种“最简” . . . 17
5.2 示范一：把所有合并做完，别漏 PI . . . 17
5.3 示范二：覆盖表与选择变量，不把项变量当输入 . . . 18
5.4 EPI 与支配：删行删列方向要有理由 . . . 18
5.5 Don’t-care 与多输出：扩大候选，不扩大必须完成的任务 . . . 18
5.6 考场步骤、反证与迁移 . . . 19
5.7 练习：先作答，再查附录 . . . 19
5.8 本章来源 . . . 19

6 04 启发式优化与恒真检验 20
6.1 为什么不用穷举解决所有问题 . . . 20
6.2 三种修改的严格条件 . . . 20
6.3 完整示范：从三个点到两个大 cube . . . 20
6.4 关键反例：分别可删，不能一起删 . . . 21

6.5 怎样高效做恒真检验 . . . 21
6.6 方案比较与验收 . . . 21
6.7 练习：先作答，再查附录 . . . 21
6.8 本章来源 . . . 22

7 05 多级逻辑与函数分解 22
7.1 两级最优并不是所有电路结构里的最优 . . . 22
7.2 函数分解：只向后级传递真正需要区分的信息 . . . 23
7.3 完整示范：3位 bound 输入压成 2 位 . . . 23
7.4 编码选择、LUT容量与 BDD 切割的联系 . . . 23
7.5 深入选学：代数除法、kernel与 co-kernel . . . 23
7.6 公平比较与失败反例 . . . 24
7.7 练习：先作答，再查附录 . . . 24
7.8 本章来源 . . . 25

8 06 ROBDD：共享余函数的决策图 25
8.1 图中的一个节点究竟表示什么 . . . 25
8.2 两条约简规则与规范性的前提 . . . 25
8.3 完整示范：MUX的两种变量序 . . . 25
8.4 从图读函数、余因子与反例 . . . 26
8.5 用 BDD 证明恒真，连接包含与分解 . . . 26
8.6 成本、异常与可验证的模型边界 . . . 26
8.7 练习：先作答，再查附录 . . . 27
8.8 本章来源 . . . 27

9 07 FSM：状态建模、最小化与编码 27
9.1 状态是过去信息的摘要，输出时间必须说清 . . . 28
9.2 示范一：从文字规格画出三状态售货机 . . . 28
9.3 等价的定义与分区算法 . . . 28
9.4 示范二：一轮不够，完整细分并找区分序列 . . . 29
9.5 从三状态图到二进制逻辑：别停在画圈 . . . 29
9.6 验证、异常与考试验收 . . . 29
9.7 练习：先作答，再查附录 . . . 30
9.8 本章来源 . . . 30

10 08 逻辑仿真：把变化放回时间轴 30
10.1 先分清你正在模拟哪一层 . . . 31
10.2 示范：事件怎样从一个门传到下一个门 . . . 31
10.3 示范：3 ns 脉冲通过 5 ns 延迟 . . . 31
10.4 静态冒险：从布尔函数回到门网络 . . . 32
10.5 反馈、锁存器与“不前进的时间” . . . 32
10.6 从示范转为独立作答：固定五步 . . . 32
10.7 练习：先作答，再查附录 . . . 32
10.8 本章来源 . . . 33

11 09 VHDL：读懂并发与赋值的真实含义 33
11.1 把文件结构与硬件结构分开 . . . 33
11.2 示范：一拍延迟来自哪里 . . . 34
11.3 组合逻辑：完整敏感、完整赋值 . . . 34
11.4 寄存器、锁存器和延迟语句的边界 . . . 35
11.5 自检比“波形看起来正确”更可靠 . . . 35
11.6 案例：一个 mux 为何三种描述都能成立 . . . 35

11.7 练习：先作答，再查附录 . . . 36
11.8 本章来源 . . . 36

12 10 故障模型与最小测试集 36
12.1 测试不是设计验证的另一个名字 . . . 37
12.2 完整示范：二输入 AND 的故障字典 . . . 37
12.3 检测、等价、支配：先写集合，再看名称 . . . 37
12.4 求最小测试集：与逻辑覆盖相同的数学骨架 . . . 38
12.5 覆盖率的分母决定你到底声称了什么 . . . 38
12.6 案例：功能测试与结构测试怎样配合 . . . 38
12.7 练习：先作答，再查附录 . . . 38
12.8 本章来源 . . . 39

13 11 ATPG：激活故障，再把差异送到输出 39
13.1 布尔差分不是连续微积分 . . . 39
13.2 完整示范：一个内部故障的两种解法 . . . 40
13.3 D 算法的每一张表都要有理由 . . . 40
13.4 重点补充：global implication 与学习准则 . . . 40
13.5 重汇合：一条能走通的路并不等于整个故障能被看见 . . . 41
13.6 PODEM 与可控性：把搜索空间和估算边界说清楚 . . . 41
13.7 案例：代数 ATPG 与结构 ATPG 怎样交叉检查 . . . 41
13.8 练习：先作答，再查附录 . . . 42
13.9 本章来源 . . . 42

14 12 故障仿真：给定测试到底检出了谁 42
14.1 把问题方向与 ATPG 反过来 . . . 43
14.2 完整示范：一个向量检出六个故障 . . . 43
14.3 局部可观察性与故障树 . . . 43
14.4 四种方案：共同目标，不同共享方式 . . . 44
14.5 选学示范：集合传播如何保留重汇合相关性 . . . 44
14.6 怎样比较工具，又怎样验证自己的手算 . . . 44
14.7 练习：先作答，再查附录 . . . 45
14.8 本章来源 . . . 45

15 13 时序测试、Scan 与 BIST 45
15.1 先看一段状态序列，而不是一个输入向量 . . . 46
15.2 Scan 怎样让寄存器成为可访问的边界 . . . 46
15.3 手算测试时间：先声明有没有重叠 . . . 46
15.4 LFSR：先固定比特顺序，再谈多项式 . . . 46
15.5 签名分析与 aliasing：压缩不会凭空创造信息 . . . 47
15.6 BIST 的取舍：图样生成、响应压缩与控制缺一不可 . . . 47
15.7 练习：先作答，再查附录 . . . 47
15.8 本章来源 . . . 48

16 交互实验的模型、手算与边界 48
16.1 把真值表压成最小 SOP . . . 48
16.2 改变顺序，观察 BDD 大小 . . . 49
16.3 短脉冲与事件队列 . . . 49
16.4 激活、传播与最小测试集 . . . 49
16.5 LFSR 的状态与周期 . . . 50
16.6 接受坏解的温度 . . . 50

17 机制、成本与方案比较 **50**
17.1 同一函数：精确 QM 与启发式覆盖 . . . 50
17.2 同一函数：两级 SOP 与因式分解 . . . 51
17.3 同一函数：ROBDD两种变量顺序 . . . 51
17.4 同一测试任务：串行与并行故障仿真 . . . 52
17.5 同一状态机：二进制编码与 one-hot . . . 52

18 历年卷地图与模考 **53**
18.1 先问自己“哪一步不会”，再安排学习 . . . 53
18.2 每个题型都走“示范—补全—独立—迁移” . . . 53
18.3 60 分钟训练卷如何分配时间 . . . 53
18.4 错题记录要留下可验证的修复 . . . 54
18.5 怎么使用这批资料而不被文件数量淹没 . . . 54
18.6 把新知识接到熟悉的计算机知识上 . . . 54
18.7 2019_retake 标签卷（卷面日期 2020-06-29） . . . 54
18.8 2020 正考（2020-12-03） . . . 55
18.9 2021 正考（2021-12-10） . . . 55
18.1021_retake 标签答案卷（卷面日期 2022-05-06） . . . 55
18.11DMA22_retake 标签卷（卷面日期 2023-06-20） . . . 56
18.12原创综合模拟 A：从机制到验证 . . . 56

19 附录 A：练习参考答案与自评量表 **58**
19.1 课程地图与数字设计的三种视角 . . . 58
19.2 布尔空间、cube与 Shannon 展开 . . . 60
19.3 精确两级综合：从全部 PI 到最小覆盖 . . . 61
19.4 启发式优化与恒真检验 . . . 62
19.5 多级逻辑与函数分解 . . . 63
19.6 ROBDD：共享余函数的决策图 . . . 64
19.7 FSM：状态建模、最小化与编码 . . . 66
19.8 逻辑仿真：把变化放回时间轴 . . . 67
19.9 VHDL：读懂并发与赋值的真实含义 . . . 68
19.10故障模型与最小测试集 . . . 69
19.11ATPG：激活故障，再把差异送到输出 . . . 71
19.12故障仿真：给定测试到底检出了谁 . . . 72
19.13时序测试、Scan 与 BIST . . . 74
19.14原创模拟卷参考答案 . . . 75

20 附录 B：术语中英对照 **78**

21 附录 C：来源与阅读边界 **79**
21.1 T25 ·2025 Tutorial Notes —Design Methodology and Automation . . . 79
21.2 L25 ·2025 Lecture Notes —Design Methodology and Automation . . . 80
21.3 F006 ·Day 2_DMA_Heuristic_Opt_2018.pdf . . . 80
21.4 F055 ·DMA22_retake.pdf . . . 80
21.5 F071 ·#DMA - Slides - Testing.pdf . . . 80
21.6 F072 ·#Intro (DMA - Folien 01 - Lecture Introduction 202311).pdf . . . 80
21.7 F073 ·#Logic Simulation (DMA - Folien 10 - Logiksimulation 11-09).pdf . . . 80
21.8 F081 ·#19_retake_solution.pdf . . . 81
21.9 F082 ·#20_solution.pdf . . . 81
21.10F084 ·#21_solution_update.pdf . . . 81
21.11F085 ·#DMA22_retake_solution.pdf . . . 81

21.12F086 ·19_retake_exam.pdf . . . 81
21.13F087 ·20_exam.pdf . . . 81
21.14F088 ·21_exam.pdf . . . 82
21.15F089 ·21_retake_exam.pdf . . . 82
21.16TUM-COURSE ·TUM EDA —Design Methodology and Automation 官方课程页 . . . 82
21.17ABC-DOC ·ABC: A System for Sequential Synthesis and Verification . . . 82
21.18ABC-CODE ·berkeley-abc/abc 作者代码 . . . 82
21.19REFERENCE-SITE ·memory-architecture-lab 参考站与源码 . . . 82
21.20BRYANT86 ·Randal E. Bryant —Graph-Based Algorithms for Boolean Function Manipulation . . . 83
21.21MULTILEVEL90 ·Robert K. Brayton / Gary D. Hachtel / Alberto L. Sangiovanni-Vincentelli —Multi-level Logic Synthesis . . . 83
21.22SIS92 ·Ellen M. Sentovich 等—SIS: A System for Sequential Circuit Synthesis . . . 83
21.23IEEE1076 ·IEEE 1076-2019 —IEEE Standard for VHDL Language Reference Manual . . . 83
21.24GHDL-SIM ·GHDL Documentation —Simulation (runtime) . . . 83
21.25GHDL-STD ·GHDL Documentation —Implementation of VHDL . . . 84
21.26ROTH66 ·J. Paul Roth —Diagnosis of Automata Failures: A Calculus and a Method . . . 84
21.27GOEL80 ·Prabhakar Goel —An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits . . . 84
21.28AGRAWAL-FAULTSIM ·Michael L. Bushnell / Vishwani D. Agrawal —VLSI Testing Course, Lecture 7: Fault Simulation . . . 84
21.29OPENTITAN-LFSR ·OpenTitan —Primitive Component: LFSR . . . 85
21.30XAPP052 ·Peter Alfke —Efficient Shift Registers, LFSR Counters, and Long Pseudo-Random Sequence Generators . . . 85
21.31本地资料审计 . . . 85

1 如何使用这份手册

本手册主题是 Design Methodology and Automation（设计方法学与自动化），不是直接存储器访问。目标是建立可在纸上独立复现的推理链：定义问题、选择表示、执行算法、检查结果、解释代价。先做示范，再补全，再独立作答。网站答案默认折叠；纸质版答案集中于附录，练习时先遮住。

证据顺序：2025-10-31 讲义与习题为课程主线；近年卷只证明历史考查；外部规范、论文、作者代码用于交叉核验。2026-11-08 为本版资料核验日，学校本次补考安排与允许携带资料尚未提供。

1.1 时间与取舍

默认每天 5 小时净学习（第 10 天 4 小时），每 50 分钟安排约 10 分钟休息，休息不计入净学习。50 小时左右不能替代整学期，但足够建立有优先级的修复循环。先用诊断定位缺口，再从示范走向补全和独立作答，最后用新题检验迁移。历史样本支持把综合/优化、事件仿真、故障测试作为主干；这不是押题。每章阅读完成与能力证据分开记录。对确已熟悉的模块，先做独立题达到 80% 评分点且能解释关键反例，再跳过基础讲解。自评分只用于调度复习，不作为学校成绩预测。

- **每天只有 3 小时**：保留 20 分钟复习、70 分钟主课、70 分钟独立题、20 分钟复盘；删去扩展阅读和工具实现，继续保留两次 60 分钟整卷。功能分解、QM、FSM、事件仿真和 ATPG 不可整章跳过。
- **布尔基础诊断低于一半评分点**：D1/D2 合并投入更多基础和 QM；D3 的深层代数 kernel、D7 的 MISR 推导改为选学。不要用背答案替代 cofactor 与真值表。
- **已会某章**：先做未见过的独立题并解释反例；通过后把节省时间交给两次诊断中失分最大的模块。
- **距离考试不足 10 天**：按依赖合并相邻日，每天保留复习和独立作答；优先完成核心题型，再做一份未见答案的整卷，最后一天停止大量新内容。

1.2 关键问题—先修知识—模块—练习/实验—来源

1.2.1 1. 课程地图与数字设计的三种视角

关键问题：同一个电路为什么要有行为、结构、物理三种描述，综合、仿真和测试分别证明什么?

先修：无需额外先修。核心目标：画出 Y-chart 并给每个域举例；把需求逐步转成 RTL、门级结构和物理实现；区分功能正确、时序正确与制造测试；用统一口径比较面积、延迟和资源取舍。

活动：补全 → 独立 → 迁移；章节手算与画图。来源：L25 2025-10-31；Introduction，印刷 pp.3–7 / PDF pp.7–11；F072 2023 Introduction slides，PDF pp.23–28；历史版本，仅用于设计流程。

1.2.2 2. 布尔空间、cube 与 Shannon 展开

关键问题：一个横杠到底代表多少点，固定一个变量以后为什么能重建整个函数?

先修：m01。核心目标：在 minterm、cube、集合、SOP/POS 之间互换；计算维数、覆盖点数、cube 距离与包含关系；求 cofactor、Shannon 展开、Boolean difference；用严格包含关系判断 unate 与变量依赖。

活动：补全 → 独立 → 迁移；章节手算与画图。来源：L25 2025-10-31；I.1–I.2，印刷 pp.9–20 / PDF pp.13–24；T25 Tutorial 1，印刷 pp.3–12 / PDF pp.5–14。

1.2.3 3. 精确两级综合：从全部 PI 到最小覆盖

关键问题：怎样证明一个 SOP 正确而且在指定成本下最优，而不是只看起来更短?

先修：m02。核心目标：完整列出所有素蕴含项并解释为什么不能继续扩张；从覆盖表识别必要项，正确处理行列支配；建立选择变量方程，区分贪心结果与精确最优；对 don’t-care 与多输出共享分别计费并验证等价。

活动：补全 → 独立 → 迁移；把真值表压成最小 SOP。来源：L25 2025-10-31；I.3，印刷 pp.21–33 / PDF pp.25–37；T25 Tutorial 2，印刷 pp.13–24 / PDF pp.15–26；Tutorial 3 P1、P4，PDF pp.27、29–33、38；F086 文件名 19_retake；卷内日期 2020-06-29；Problem 1c，PDF p.3；F088 2021 exam；Problem 1b，PDF p.3。

1.2.4 4. 启发式优化与恒真检验

关键问题：为什么“单独能删”不代表“可以一起删”，为什么先变复杂有时能通向更好的解？

先修：m02、m03。核心目标：推导 expand、reduce、remove 的合法性条件；把 cube 包含转成 cofactor 恒真问题；解释配置图的可行解、邻域、局部与全局最小；对每步修改给出证明或反例而非只写是/否。

活动：补全 → 独立 → 迁移；接受坏解的温度。来源：L25 2025-10-31；I.4，印刷 pp.34–36 / PDF pp.38–40；多输出操作 PDF p.35；T25 Tutorial 3 P2、P3，PDF pp.28–29、34–37；F087 2020 exam，Problem 1g，PDF p.7；F088 2021 exam，Problem 2a，PDF p.5；SIS92 Berkeley UCB/ERL M92/41 元数据页；仅作 L25 提及工具的背景，不据此声称复现。

1.2.5 5. 多级逻辑与函数分解

关键问题：为什么两个 LUT 中间的线数由不同余函数的数量决定，而不是由原来的 minterm 数决定？

先修：m02、m03。核心目标：画 Boolean network 并识别 fanin/fanout/reconvergence；用等价类构造 h 与 g，使 f(x,y)=g(h(x),y)；计算最少编码位和有利分解条件；区分逻辑共享的 literal 节省与实际时序成本。

活动：补全 → 独立 → 迁移；章节手算与画图。来源：L25 2025-10-31；I.5，印刷 pp.37–43 / PDF pp.41–47；T25 Tutorial 4 P4，印刷 p.40 / PDF p.42；解答 PDF p.52；F089 内页 DMA 2022（文件名 21_retake_exam）；Problem 1d/1f，PDF pp.3–4；MULTILEVEL90 1990-02；§IV.D，印刷 p.272 / PDF p.9，kernel/co-kernel 定义与 Table 1；深入选学。

1.2.6 6. ROBDD：共享余函数的决策图

关键问题：为什么同一变量序下的最简决策图可以判等价，而换变量序后节点数会改变？

先修：m02。核心目标：按指定变量序从 cofactors 构造 OBDD；使用删除无效测试与合并同构子图两条规则；由到 1/到 0 的路径写 SOP/POS 并求 cofactor；用明确计数口径比较变量序与功能分解。

活动：补全 → 独立 → 迁移；改变顺序，观察 BDD 大小。来源：L25 2025-10-31；I.6，印刷 pp.44–47 / PDF pp.48–51；T25 Tutorial 4 P1–P3，PDF pp.39–41、44–51；F086 文件名 19_retake；卷内日期 2020-06-29；Problem 1f，PDF p.4；F087 2020 exam；Problem 1d，PDF p.4；BRYANT86 1986 原刊；作者注释电子版 §1–2，PDF pp.1–7；固定序 ROBDD 规范性与变量序限制。

1.2.7 7. FSM：状态建模、最小化与编码

关键问题：怎样证明两个状态可以合并，并把合并后的图变成可实现的触发器输入方程？

先修：m02。核心目标：将文字规格转为确定且完整的 Mealy/Moore 状态表；删除不可达状态并逐轮细分等价类；构造能区分两个状态的输入序列；完成编码、下一状态与输出方程并验证复位/非法码。

活动：补全 → 独立 → 迁移；章节手算与画图。来源：L25 2025-10-31；I.7，印刷 pp.48–56 / PDF pp.52–60；T25 Tutorial 4 P5，PDF p.42；解答 PDF pp.53–54；F087 2020 exam，Problem 1f/1h，PDF pp.6、8；F088 2021 exam，Problem 2c/2d，PDF pp.7–8。

1.2.8 8. 逻辑仿真：把变化放回时间轴

关键问题：布尔函数没有错，为什么仿真波形仍会出现毛刺、X或时间不前进?

先修：m02、m07。核心目标：区分稳态布尔等价、定时行为与模拟电气行为；按事件队列逐步画出门级波形，记录生成时刻与执行时刻；预测 transport 与 inertial 对窄脉冲的不同处理；解释三值逻辑、组合毛刺和零时间反馈不收敛。

活动：补全 → 独立 → 迁移；短脉冲与事件队列。来源：L25 II.1–3，印刷 pp.57–63；PDF pp.61–67；L25 II.4，印刷 pp.67–68；PDF pp.71–72；T25 Tutorial 5，印刷 pp.58–64；PDF pp.60–66；T25 Tutorial 6，印刷 pp.65–71；PDF pp.67–73；GHDL-SIM Simulation options：–stop-delta、–disp-time；2026-11-08 所见 7.0.0-dev 文档。

1.2.9 9. VHDL：读懂并发与赋值的真实含义

关键问题：为什么同一段 process 里两行看似顺序的赋值，得到的硬件与软件直觉不同?

先修：m08。核心目标：根据进程唤醒与 signal 更新规则手算运行结果；分别写出组合逻辑、锁存器和边沿触发寄存器；发现敏感列表缺项、未赋值分支和多驱动等错误；用可重复的输入与断言验证模型，而非只看波形。

活动：补全 → 独立 → 迁移；章节手算与画图。来源：L25 II.4，印刷 pp.64–68；PDF pp.68–72；T25 Tutorial 6，印刷 pp.65–71；PDF pp.67–73；IEEE1076 IEEE 1076-2019 官方元数据；仅核对规范身份、范围和发布日期；GHDL-STD Implementation of VHDL：VHDL standards；标准模式与部分实现限制；GHDL-SIM Simulation options：断言、时间与 delta 选项。

1.2.10 10. 故障模型与最小测试集

关键问题：测试通过能证明什么？怎样用少量输入检测一个明确定义的故障集合?

先修：m02、m03。核心目标：区分设计验证、制造测试、故障检测和故障定位；建立单 stuck-at 故障字典与 test-fault 关系；用集合包含判断等价、支配及删除方向；计算带明确定义分母的覆盖率并求小规模最小测试集。

活动：补全 → 独立 → 迁移；激活、传播与最小测试集。来源：L25 III.1，印刷 pp.69–78；PDF pp.73–82；T25 Tutorial 7，印刷 pp.72–75；PDF pp.74–77；F071 Testing slides：fault models 与制造测试引言，PDF pp.1–15。

1.2.11 11. ATPG：激活故障，再把差异送到输出

关键问题：看到一个内部 stuck-at 故障，怎样系统地构造会在输出暴露它的输入?

先修：m02、m10。核心目标：从 cofactor 推导布尔差分与 stuck-at 检测条件；完整完成一个 D 算法的激活、传播、证明与回溯过程；识别重汇合的自掩蔽，避免错误套用单路径链式法则；解释 PODEM 的主输入决策与 controllability 近似的边界。

活动：补全 → 独立 → 迁移；章节手算与画图。来源：L25 III.2，印刷 pp.79–89；PDF pp.83–93；L25 III.4，印刷 pp.93–99；PDF pp.97–103；T25 Tutorial 8，印刷 pp.76–81；PDF pp.78–83；T25 Tutorial 10，印刷 pp.87–94；PDF pp.89–96；F088 21_exam.pdf，Problem 4(c)，PDF p.12：global implication 学习准则与可控性；ROTH66 D 算法原始论文；出版信息与可索引摘要，全文抓取受限；GOEL80 PODEM 原始会议版本 FTCS-10，1980，pp.145–151；仅索引摘要，全文 403。

1.2.12 12. 故障仿真：给定测试到底检出了谁

关键问题：ATPG 给出一个向量后，怎样知道它是否顺便覆盖其它故障，又怎样避免重汇合误判?

先修：m08、m10、m11。核心目标：给定输入向量，先计算无故障值再找全部被检出故障；区分扇出干线与分支的传播和掩蔽；用局部可观察性构造无扇出区域的故障树；解释 serial、parallel、deductive、concurrent 的成本与模型限制。

活动：补全 → 独立 → 迁移；章节手算与画图。来源：L25 III.2，印刷 pp.85–89；PDF pp.89–93；L25 III.3，印刷 pp.90–92；PDF pp.94–96；T25 Tutorial 9，印刷 pp.82–86；PDF pp.84–88；AGRAWAL-FAULTSIM 作者课程 Lecture 7：Fault Simulation；四类方法为选学扩展。

1.2.13 13. 时序测试、Scan 与 BIST

关键问题：内部状态既不能直接设置也不能直接读出时，怎样使测试仍然可管理？

先修：m07、m09、m11、m12。核心目标：通过时间展开解释为什么时序故障需要输入序列；画出 scan 移位、capture、卸载的状态与资源时间线；计算明确装载重叠假设下的 scan 测试周期数；按明确定义递推手算 LFSR 与 GF(2) 签名，并解释 aliasing 限制。

活动：补全 → 独立 → 迁移；LFSR 的状态与周期。来源：L25 III.5，印刷 pp.100–107；PDF pp.104–111；T25 Tutorial 11，印刷 pp.95–97；PDF pp.97–99；OPENTITAN-LFSR prim_lfsr 文档：参数、锁死态保护和验证范围；master 于 2026-11-08 所见；XAPP052 Peter Alfke, XAPP052 v1.1, 1996-07-07；最大周期抽头与编号约定。

2 10 天路线与能力证据

2.1 Day 1 •先诊断，再修复布尔基础

净学习约 300 分钟；模块 m01、m02。

- 40 分钟无资料诊断：真值表 ↔SOP、cube 维数/距离、cofactor、一个简单事件表和故障激活；不会的写下卡点。
- 阅读 m01 30 分钟，m02 70 分钟；每读一个定义立即给一例和一反例。
- 90 分钟完成布尔示范、补全题、独立题；对照 T25 Tutorial 1。
- 用 35 分钟做 F087 1a 并解释每一步；最后 35 分钟建立错题与术语卡。

当天必须留下的能力证据

- 不看答案，从 3 变量真值表写 CSOP 并化简；再逐行验证 8 种输入。
- 能在 2 分钟解释 on-set、off-set、don’t-care 三者关系，区分 cube 维数与空间维数。

间隔复习：当晚回忆当天 3 个错误，重做一题，不抄整页笔记。

2.2 Day 2 •精确最小化：把步骤做稳

净学习约 300 分钟；模块 m03。

- 20 分钟空白纸重建 Shannon 展开、cube 维数和 cofactor。
- 80 分钟学习 Quine 合并、PI/EPI、覆盖条件、共识/吸收。
- 90 分钟完成一个 4 变量全流程示范 → 遮住关键列补全 → 独立变式。
- 70 分钟做 F086 1b（覆盖表，PDF p.2）与 1c（Quine，PDF p.3），分别记录合并错误和覆盖选择错误；F055 保留到 D9 再打开。
- 40 分钟在实验里先预测 PI 再运行；用枚举真值表复核。

当天必须留下的能力证据

- 提交全部 PI、覆盖表、MinSOP、代价口径和至少一次等价验证。
- 能解释 CompSOP 不等于 MinSOP，并给出启发式可能非最优的理由。

间隔复习：D1 错题；D2 知识次日、3 天后、7 天后复习。

2.3 Day 3 •启发式、共享逻辑与函数分解

净学习约 300 分钟；模块 m04、m05。

- 25 分钟闭卷重做昨日一题的覆盖条件。

- 80 分钟学习 cube removal、expansion、reduction 的合法性；分别删除与共同删除必须重新检验。
- 80 分钟学习 multiple implicant、因式分解与 LUT 分解；绑定变量和自由变量分开。
- 75 分钟完成 F087 1g 与 F086 1e；用不同 cofactor 数量计算中间位数。
- 40 分钟比较两种等价电路，统一 literal/门输入计数，并记录逻辑深度代价。

当天必须留下的能力证据

- 能够用恒真检验说明一个 literal 能否删除，失败时给具体输入反例。
- 画出 h(X)→Z→g(Z,Y) 并说明未用编码何时可以成为 don’t-care。

间隔复习：D1 间隔复习；D2 最易错一步。

2.4 Day 4 •ROBDD 与 FSM：图不是装饰

净学习约 300 分钟；模块 m06、m07。

- 20 分钟复习 cofactor 与分解等价类。
- 75 分钟构造 ROBDD，明确 0/1 边、变量顺序、合并同构子图和删除冗余测试。
- 85 分钟学习 Mealy/Moore 和划分细化，每轮用上一轮分组重新编码后继。
- 80 分钟做 F087 1d（ROBDD，PDF p.4）与 1h（状态最小化，PDF p.8）；画最小状态图再验证输入序列，F055 保留到 D9。
- 40 分钟把同一个函数换变量顺序；解释节点数与语义不变。

当天必须留下的能力证据

- 独立构造一个 ROBDD 并从路径恢复 SOP，用真值表对照。
- 完成 FSM 稳定划分，给出区分两个不等价状态的输入序列。

间隔复习：D2 复习覆盖；D3 复习合法删除条件。

2.5 Day 5 •事件仿真与 VHDL：掌握时间语义

净学习约 300 分钟；模块 m08、m09。

- 25 分钟画出昨日 FSM 的数据通路和状态存储。
- 80 分钟学习 event、transaction、同一物理时刻、delta、transport/inertial；先手算后运行实验。
- 75 分钟完成事件表：当前时间、信号状态、被激活元件、新事件、队列；再画波形。
- 70 分钟学习 VHDL signal/variable 与 process；独立完成 m09-q1、m09-q3、m09-q4，分别检查两拍赋值、遗漏敏感信号与锁存器错误；F088 保留到 D8。
- 50 分钟做 F087 2a/2b，对照原图检查初值与门延迟。

当天必须留下的能力证据

- 同一电路独立产生与实验一致的事件序列和最终稳定值。
- 说明为什么 signal 赋值后不能立即读到新值，指出不完整赋值/敏感列表的不同问题。

间隔复习：D2 的 3 天后复习；D4 图形题一题。

2.6 Day 6 •故障覆盖与布尔差分

净学习约 300 分钟；模块 m10、m11。

- 20 分钟闭卷写事件仿真的操作顺序。
- 65 分钟学习 stuck-at、fault detection/diagnosis、故障等价、覆盖率分母。

- 95 分钟学习 Boolean difference 和 AND-XOR，分别计算激活条件与传播条件。
- 80 分钟做 F086 4b（故障检测与最小测试集，PDF p.8）与 4c（布尔差分，PDF p.9）；把候选向量代入好/坏电路验证，F088 保留到 D8。
- 40 分钟整理输入端与内部节点故障差异，避免把不可控内部赋值当成合法输入。

当天必须留下的能力证据

- 给出最小检测测试集，并说明为什么它不一定能区分所有故障。
- 给定一个输入 stuck-at 故障，求全部检测向量并逐个验证。

间隔复习：D3 的 3 天后复习；D5 时序易错点。

2.7 Day 7 •D 算法、故障树和扫描设计

净学习约 300 分钟；模块 m11、m12、m13。

- 20 分钟复习 D 与 D-bar 以及各门的非控制值。
- 90 分钟手工执行 D 算法：激活 → 传播 → 证明可实现 → 蕴含 → 冲突回溯；保留搜索树。
- 75 分钟学习 fanout-free region、局部可观测性合成及重汇聚；完成故障树示范。
- 65 分钟做 F087 3c/3d；用故障枚举核对已找到向量，检查是否漏解。
- 50 分钟学习 scan-in/capture/scan-out、LFSR 与 MISR；深入工具实现可放入选学。

当天必须留下的能力证据

- 提交一棵含失败分支的 D 算法搜索树，明确失败限于路径还是全搜索。
- 用故障树列出一个测试向量检测的故障集；解释 scan 改善可控/可观测性的机制与代价。

间隔复习：D4 的 3 天后复习；D6 布尔差分一题。

2.8 Day 8 •第一轮整卷：按证据分配修复时间

净学习约 300 分钟；模块 m02、m03、m04、m05、m06、m07、m08、m09、m10、m11、m12。

- 15 分钟准备纸笔、计时器和允许的模拟资料；打开前 7 天保留未作答的 F088（21_exam.pdf），先隔离解答。若你在本计划开始前已看过整卷答案，本次标为复练，不把分数当盲测证据；可改用尚未看过解析的本站原创模拟卷。
- 60 分钟完整模拟，暂不看答案；无法开始的题写出已知和第一步后跳过。
- 75 分钟按官方答案逐小题复盘；把失分分为概念、算法、符号、时间和审题。
- 100 分钟只修复分值损失最大的两项：每项示范 1 题、独立 2 题、迁移 1 题。
- 50 分钟复习已到期卡片；重做一道旧错题，证明修复而不是记住答案。

当天必须留下的能力证据

- 保留计时作答纸和自评分；给每个错题标首个错误步骤及修复证据。
- 同类新题在限定时间达到至少 80% 评分点。该阈值是学习门槛，不是学校及格线。

间隔复习：D1 的 7 天后复习；D5 的 3 天后复习。

2.9 Day 9 •第二轮模拟：迁移与速度

净学习约 300 分钟；模块 m02、m03、m04、m05、m06、m07、m08、m10、m11、m12、m13。

- 20 分钟回顾昨日首个错误，不重看整章。
- 60 分钟做前 8 天保留未作答的 F055（DMA22_retake.pdf，卷内日期 2023-06-20）；保留最后 5 分钟检查。若此前已经做过或看过答案，改用尚未看过解析的本站原创模拟卷；两者均已接触则明确标为复练。

- 70 分钟批改并比较两轮的题型耗时与独立得分，不比较已经见过答案的分数。
- 100 分钟做迁移：更换变量顺序/初值/故障极性/编码位，观察哪些步骤保持不变。
- 50 分钟整理 2 页速查：算法步骤、符号约定、边界条件、来源页码。

当天必须留下的能力证据

- 写出每个高风险题型的 5 行解题程序，能在不看讲义时复述。
- 对一条错误规则构造反例，并解释为何之前看似可行。

间隔复习：D2 的 7 天后复习；D6 的 3 天后复习。

2.10 Day 10 •收口：可用的知识与稳定状态

净学习约 240 分钟；模块 m03、m06、m07、m08、m11。

- 40 分钟闭卷重建五张模板：QM 表、ROBDD、FSM 划分、事件表、D 算法树。
- 60 分钟混合小题，优先挑过去做错而最近没有看答案的题。
- 60 分钟只修复仍会重复的错误，停止扩展新主题；查看全部必学模块的能力记录。
- 40 分钟核对考试通知、时间地点、允许资料和计算器；打印索引与速查表。
- 40 分钟轻量口述、画图和复习；留出休息与睡眠，不用通宵补偿。

当天必须留下的能力证据

- 所有必学模块至少有一份独立作答证据；未通过项明确写“尚未掌握”，不由阅读进度代替。
- 能够解释机制、画图、计算、定位错误、比较方案和给验证方法。

间隔复习：D3 的 7 天后复习；D7 的 3 天后复习；考试前只看高频个人错因。

3 01 课程地图与数字设计的三种视角

先理解问题、表示与评价标准，再学习算法

关键问题：同一个电路为什么要有行为、结构、物理三种描述，综合、仿真和测试分别证明什么?

学习目标

- 画出 Y-chart 并给每个域举例
- 把需求逐步转成 RTL、门级结构和物理实现
- 区分功能正确、时序正确与制造测试
- 用统一口径比较面积、延迟和资源取舍

先修：无；**横向联系**：m03、m07、m08、m10。

3.1 你要掌握的主线：表示、变换、验证

这门 DMA 是 **Design Methodology and Automation，设计方法学与自动化**，不是计算机体系结构中传数据的 Direct Memory Access。整门课围绕一个问题：怎样把数字功能要求转成可制造、可验证、可测试的电路。第一部分“综合”寻找正确且成本好的结构；第二部分“仿真”研究状态怎样随事件变化；第三部分“测试”研究制造出的实例是否出现指定故障。

十天复习不宜逐页平均用力。先建立统一语言：布尔函数把输入映成输出；cube 把一组输入压缩表示；cofactor 固定部分输入；SOP 是一种表示；BDD 是另一种表示。QM、启发式优化、BDD 化简并非彼此无关的口诀，它们都在改变**表示**并保持允许输入上的**行为**。FSM 把当前状态加入输入；仿真把状态加入时间；测试把正常与故障行为作比较。

本学习站采用的能力顺序是：能解释 → 能手算 → 能画中间过程 → 能找反例 → 能比较 → 能设计验证。每章“已阅读”只记录接触过内容，真正通过需要提交独立题的过程。已有数电基础可以从诊断题开始；没有把握时先用真值表这一共同语言补足。这里不假定你已经学过 CPU、GPU 或某种编程语言。

3.2 Y-chart：域与抽象层级是两根不同坐标

Y-chart的三个方向表示观察同一对象的三种方式。**行为域**回答“输入什么会得到什么”，如算法、寄存器传输、布尔方程；**结构域**回答“由什么连接而成”，如 CPU/存储器、ALU/寄存器/MUX、门/触发器、晶体管；**物理域**回答“放在哪里、几何形状怎样”，如芯片/板、布局块、标准单元、掩模多边形。离中心远近表达抽象层级，不能把“结构域”当作“低层级”的同义词。

请在纸上画三条轴，每轴按 system → algorithm → RT → logic → circuit 排列对应对象。例如同在逻辑层：行为是$y = ab + \bar{a}c$，结构是 AND/OR/NOT 门网表，物理是单元与连线布局。综合通常从行为走向结构，同时逐步细化；分析从结构推导行为或性能；物理提取从布局获得电容、电阻等更具体信息。实际流程有反馈回路，不是一次向前永不回头。

可把它类比为“菜谱、厨具与工位布置”：菜谱说明目的与步骤，厨具连接说明实现，摆放决定移动成本。类比边界是数字电路有精确定义的并发与时序，菜谱通常没有形式化等价标准。理解类比后应立即回到“同一行为、不同结构、不同物理代价”的严格表述。来源：L25 PDF pp.9–10。

3.3 设计流程：每次转换都要带上约束

一个可复述的简化流程是：需求/行为规范 → 架构选择 → RTL → 逻辑优化 → 技术映射 → 布局布线 → 制造 → 测试。RTL 描述每个时钟边界上的寄存器更新与组合运算；逻辑优化寻找等价表达；技术映射把抽象逻辑匹配到真实单元库；物理设计处理放置和连线。L25 PDF p.8 展示的流程同时包括库信息、模块生成和测试生成，这些不是事后可有可无的附加物。

同一份布尔功能可映到标准单元，也可映到 FPGA 的 LUT 与寄存器。选用何种实现结构影响合法操作和成本。例如两级 SOP 的一个大 AND 不能假定对应一个任意扇入、零延迟的物理门；一个 6 输入 LUT 也不能无条件装下 7 输入函数。抽象算法必须保留“目标库/资源/时间预算”这组上下文。

每一步至少问三个问题：① 转换前后在什么输入假设下等价？② 评价的是 literal、门数、面积、最长路径还是功耗？③ 模型没覆盖什么？忽略第 ① 会优化错函数，忽略第 ② 会把不同方案的不同指标混比，忽略第 ③ 会把教学估算说成硬件预测。十天内优先练课程算法，不必为流片工具安装消耗主线时间。

3.4 贯穿案例：一位全加器的等价实现

问题：输入为a, b, c_{in}，输出和位s与进位c_{out}，必须满足$a + b + c_{in} = s + 2c_{out}$（本式是整数算术）。基线直接由真值表列出两个输出的 SOP。机制改写为$p = a \oplus b$，$s = p \oplus c_{in}$，$c_{out} = ab + pc_{in}$。另一种进位写法是$ab + ac_{in} + bc_{in}$。两式功能相同，最短表达不一定在所有库里最快。

证据先用八种输入穷举。例如 101 的整数和为 2，所以$s = 0, c_{out} = 1$；公式得到$p = 1, s = 0, c_{out} = 1$。对 111，$p = 0, s = 1, c_{out} = 1$，对应整数 3。成本比较必须声明模型：假设 XOR 延迟 2 个时间单位、AND/OR 各 1，且输入同到、无限扇出、忽略连线，使用 p 的 carry 最长路径为 XOR→AND→OR，共 4 单位；多数函数写法 AND→OR，共 2 单位（假设 3 输入 OR 可用）。这是教学估算，换单元库就可能变化。

适用条件：共享 p 可能节约逻辑，且 sum 本来需要 p；反例：carry 是关键路径时，共享 p 可能拖慢它。验证办法：功能做八行表，时序做按路径累计或库级分析，面积按同一库统计。如果库只允许二输入 OR，多数函数的 OR 树还需额外一级，比较要重算。不要把本段任意时间单位当作纳秒。

3.5 仿真、形式验证、制造测试：证据不能互相替代

仿真在模型和选定激励下计算行为，一次仿真“没有报错”只支持已测试的轨迹。组合电路若穷举所有有限输入且模型无遗漏，可以建立该模型上的完整功能证据；大电路通常不能穷举。形式等价检查则尝试证明所有允许输入上两个表示相同，例如构造$f \oplus g$并证明恒为 0。两者都依赖约束正确：把关键场景错误排除，结论也会失真。

制造测试是在实物上施加测试并观察输出，通常相对于 stuck-at 等故障模型报告覆盖率。逻辑功能正确的设计仍可能制造损坏；没有检测到 stuck-at 故障也不能证明不存在所有模拟、电源或时序问题。课程后半部分会教“激活故障、传播差异、在输出观察”，这与综合中的“覆盖全部 minterm”在数学上都有集合覆盖影子，但覆盖对象和物理含义不同。

十天验收建议：每天先闭卷回忆昨天的两个机制，再做一道未见过或改参数的题；晚上只记录错误原因与改正证据。对“概念没记住、算法步骤漏、符号看错、时间分配失误”分别处理。正式考试允许资料、时长与范围请以学校最新通知为准，本站旧试卷仅作练习依据；没有收到的通知不会被猜成事实。

3.6 练习：先作答，再查附录

3.6.1 m01-q1 ·补全·2 分

“布尔方程、门级网表、单元布局”分别属于哪一个域？三者是否必然处于不同抽象层？

3.6.2 m01-q2 ·独立·4 分

不用看正文，画从规范到制造测试的流程，标出至少两个需要反馈重做的地方。

3.6.3 m01-q3 ·独立·2 分

全加器输入 110 时，carry 和 sum 是什么？为什么不能用布尔 OR 代替整数求和来解释？

3.6.4 m01-q4 ·补全·2 分

在正文的理想门延迟模型下，XOR→AND→OR的路径延迟是多少个时间单位？

3.6.5 m01-q5 ·迁移·4 分

有人说“我运行了 1000 组随机向量，所以 20 输入组合电路已完全验证”。你会怎样补全他的证据？

3.6.6 m01-q6 ·迁移·3 分

用“问题—基线—机制—成本—反例—验证”简述全加器共享 p 的取舍。

3.7 本章来源

- L25：2025-10-31；Introduction，印刷 pp.3–7 / PDF pp.7–11
- F072：2023 Introduction slides，PDF pp.23–28；历史版本，仅用于设计流程

4 02 布尔空间、cube 与 Shannon 展开

把公式读成集合，把消元读成固定输入

关键问题：一个横杠到底代表多少点，固定一个变量以后为什么能重建整个函数？

学习目标

- 在 minterm、cube、集合、SOP/POS 之间互换
- 计算维数、覆盖点数、cube距离与包含关系
- 求 cofactor、Shannon 展开、Boolean difference
- 用严格包含关系判断 unate 与变量依赖

先修：m01；**横向联系**：m03、m04、m06、m11。

4.1 先统一符号：同一个“加号”有两种语义

本课程用+ 表示 OR，用并写或· 表示 AND，用$\bar{x}$ 或 ASCII `x'` 表示 NOT；$\oplus$ 是 XOR。布尔代数有$x + x = x$，所以不要套整数规则$x + x = 2x$。常用法则要能正反使用：吸收$x + xy = x$；分配$x(y + z) = xy + xz$；另一条分配$x + yz = (x + y)(x + z)$；De Morgan $\overline{xy} = \bar{x} + \bar{y}$。后一条意味着翻转整个门时也要翻转输入和门类型。

例：$f = ab + \bar{a}b$，提取 b 得$b(a + \bar{a}) = b$。每一步是集合并/交的等式，不是“把 a 消掉”的口诀。再看$ab + \bar{a}c + bc$：bc 是前两项的 consensus，可被吸收；但是$ab + \bar{a}c$ 一般不等于$b + c$，输入$a = 1, b = 0, c = 1$ 就是反例。学会找一个反例比盲目展开更快。

变量顺序必须显式写出，例如abc 的 101 是$a = 1, b = 0, c = 1$，对应十进制 minterm 5。输入空间有2^n 个点；所有完全指定单输出布尔函数有2^{2^n} 个，不是2^n。这是“输入有几种”和“可给每个输入安排多少种输出表”两个不同计数问题。来源：L25 PDF pp.13–19。

4.2 cube：少一个 literal，就多一个自由维度

cube是一组固定了部分比特的输入点。n 变量空间中，一个含 k 个不同且一致 literal 的乘积项维数为$d = n - k$，覆盖2^d 个 minterm。三变量 `1-0` 是$a\bar{c}$，覆盖 100 与 110，维数 1；`---` 是常量 1，覆盖全部 8 点，维数 3；`101` 是 minterm，维数 0。空集 0 不是通常的非空 cube，不能把相互矛盾的$a\bar{a}$ 算作“2 literal、维数 n−2”。

每位可写 0、1、横杠，故非空 cube 总数为3^n。恰好 d 维的 cube 数为$\binom{n}{d}2^{n-d}$：先选自由位置，再给其余位赋值。比如三变量的 1-cube 有$\binom{3}{1}2^2 = 12$，也等于立方体的 12 条边。四维的 1-cube 有 32 条边。这是 Quine 第一轮最多可能产生多少不同 1-cube 的来源。

两个 cube 的距离按本课口径是**发生相反固定值的位置数**，也等于从两集合各取一点的最小 Hamming 距离。例如 `1-0` 与 `0-1` 在 a、c 上冲突，距离为 2；`1-0` 与 `-10` 距离为 0，交集为 110。距离 0 不等于两个 cube 相同；也不等于一个包含另一个。包含检验是“更大集合的每个固定条件都被更小集合满足”：`110` 包含于 `1-0`，而不是反过来。

4.3 On/off/DC 与 SOP/POS 的转换

完全指定函数的 on-set F 是必须为 1 的点，off-set R 是必须为 0 的点。若有 don’t-care D，则 F、D、R 两两不交且并集为B^n；一个合法实现 g 满足$F \subseteq on(g) \subseteq F \cup D$。cube 中的横杠表示该输入变量自由，而真值表输出栏的星号表示规范未约束输出，二者虽然常用同一符号，语义不同。

原创示范：变量 abc，$F = \{1, 3, 5, 7\}$，即 c=1 的四点。CSOP 写成$\bar{a}\bar{b}c + \bar{a}bc + a\bar{b}c + abc$，全部合并得到$f = c$。off-set 为 0、2、4、6；CPOS 是对应四个 maxterm 的积。对输入 010，令该和项恰为 0 的式子是$(a + \bar{b} + c)$，因此 maxterm 中 0 对应正 literal、1 对应负 literal，正好与 minterm 相反。

一般求 POS 的稳妥方法：先求补函数$\bar{f}$ 的 SOP，再整体取反并应用 De Morgan。例如$\bar{f} = ab + \bar{a}c$，则$f = (\bar{a} + \bar{b})(a + \bar{c})$。不要把 SOP 中每个乘号换加号却忘记整体补关系。要检查时，挑一个 on-set 和一个 off-set 点代入；小函数可穷举全部点。canonical 表示全部 minterm/maxterm，complete SOP 表示全部 PI，这两个单词不是同义词。

4.4 完整示范：固定输入、重建函数、得到 MUX

取$f(a, b, c) = ab + \bar{a}c$。固定 a=1 时，$f_{a=1} = b$；固定 a=0 时，$f_{a=0} = c$。cofactor 是把变量替换为常数后得到剩余变量的函数，不是普通代数求导。Shannon 展开为：

$$f = af_{a=1} + \bar{a}f_{a=0} = ab + \bar{a}c.$$

证明只需分 a=1 和 a=0 两种情形；此时一个分支被屏蔽，另一个分支被选中。因此它正是 2:1 MUX：a 为选择，b 为 a=1 的数据，c 为 a=0 的数据。若画图，必须在边上标 0、1，不能只靠左右方位猜。连续固定不同变量可交换次序，例如$f_{a=0,b=1} = c = f_{b=1,a=0}$。

沿 b 展开也成立：$f_{b=1} = a + \bar{a}c = a + c$，$f_{b=0} = \bar{a}c$，所以$f = b(a+c) + \bar{b}\bar{a}c$。表达式变长并不表示推导错了；不同分解变量会导致不同的 BDD 大小和电路结构。把 cofactor 类比为程序中“把某个参数设为常量后部分求值”很有用，但布尔函数没有程序的副作用和执行顺序，不能把公式里的左右顺序当时间先后。来源：L25 PDF pp.19、24。

4.5 依赖、单调、量词与布尔差分

变量 a 的布尔差分为$\partial f/\partial a = f_{a=0} \oplus f_{a=1}$。对 MUX 例，它等于$b \oplus c$：当 b=c，改变选择 a 不影响输出；当 b≠c，改变 a 会翻转输出。**独立于 a** 等价于这个差分恒为 0；**依赖于 a** 只要求存在某个输入使差分为 1，即差分不恒为 0，不要求处处为 1。这一点在 PDF 文本抽取丢失否定符号时尤其容易看错。

positive unate 意味着$f_{a=0} \subseteq f_{a=1}$：固定其他输入后，a 从 0 到 1 不能把输出从 1 变 0。negative unate 方向相反。若两个方向都不成立，就是 binate。MUX 对 a 为 binate：取 b=1,c=0，a 升高输出升高；取 b=0,c=1，a 升高输出降低。它对 b 和 c 则是 positive unate。独立变量同时满足正、负 unate，不能因此认定定义矛盾。

Consensus/全称消元为$\forall a\, f = f_{a=0} f_{a=1}$；smoothing/存在消元为$\exists a\, f = f_{a=0} + f_{a=1}$。对 MUX 例分别得到 bc 与 b+c。前者问“不论 a 如何都为 1 吗”，后者问“能否选一个 a 使它为 1”。恒真证明需要两个 cofactor 都恒为 1；一个分支为 1 只证明部分输入成立。布尔差分会在故障传播中再次出现，cofactor 会在 BDD、启发式合法性和分解中反复使用。

4.6 你该怎样在纸上验证自己

每次转换先写变量序与假设，再写集合或真值表作独立检查。若题目要求某个 cube 属于函数，使用$c \subseteq f \Leftrightarrow f|_c \equiv 1$，这里$f|_c$ 表示固定 c 中所有 literal，使其为真。例如$c = ab$，就取 a=1,b=1。注意$f|_c = 1$ 要对剩余所有输入都成立，而不是随便再取一组值验证。

发现答案不同时，先检查输出补号、变量顺序和 CSOP/CompSOP 混淆，再检查代数。这三种抄写错误比算法本身更常见。PDF 文本中的上划线可能在提取时丢失，本网站统一重写了公式；对原卷图上的补号有疑问时应回看渲染页。能力验收是能在 10 分钟内把一个三变量函数写成真值表、求两个 cofactor、给出一条合法扩张和一条反例；做到后即可进入 QM。

4.7 练习：先作答，再查附录

4.7.1 m02-q1 ·补全·2 分

在 B^7 中，cube 1-0–11 的维数是多少?

4.7.2 m02-q2 ·独立·2 分

在 B^4 中，恰好 1 维的非空 cube 有多少个?

4.7.3 m02-q3 ·独立·4 分

对 f=ab+a′ c，求 f|a=0、f|a=1、关于 a 的差分，并给出 a 不影响输出的一类输入。

4.7.4 m02-q4 ·独立·3 分

求 1-0 与 -10 的距离和交集，并解释距离 0 为什么不表示相同。

4.7.5 m02-q5 ·迁移·3 分

有人从“f|a=1恒为 1”推出 f 恒真。构造最小反例，并给正确判据。

4.7.6 m02-q6 •迁移•4 分

对 f=ab+a′c 判断 a 是否 positive unate、negative unate 或 binate，必须给证据。

4.8 本章来源

- L25：2025-10-31；I.1–I.2，印刷 pp.9–20 / PDF pp.13–24
- T25：Tutorial 1，印刷 pp.3–12 / PDF pp.5–14

5 03 精确两级综合：从全部 PI 到最小覆盖

Quine–McCluskey、必要项、覆盖表、Petrick与成本口径

关键问题：怎样证明一个 SOP 正确而且在指定成本下最优，而不是只看起来更短?

学习目标

- 完整列出所有素蕴含项并解释为什么不能继续扩张
- 从覆盖表识别必要项，正确处理行列支配
- 建立选择变量方程，区分贪心结果与精确最优
- 对 don′t-care 与多输出共享分别计费并验证等价

先修：m02；**横向联系**：m04、m05、m10。

5.1 先分清四种“最简”

一个乘积项c是蕴含项（implicant），意思是c覆盖的每个输入点都允许输出 1。完全指定函数要求$c \subseteq F$；不完全指定函数允许$c \subseteq F \cup D$，但最终必须覆盖全部F。素蕴含项（PI）是不能再删除任何一个 literal 而保持合法的蕴含项。因此“素”是局部不可扩张，不是“必须选”。必要素蕴含项（EPI）至少独占一个 on-set 点：不选它就没人能覆盖那个点。

CompSOP 是所有 PI 的和；**irredundant cover** 是无法直接删去任何一项的覆盖；**MinSOP** 才是按目标成本最小的 SOP。L25 印刷 p.21 将 MinSOP 定义为最少 literal。某些教材采用“先项数，再 literal”的字典序目标，做卷子必须先读题，不能悄悄换口径。本课程默认报告“项数 / literal 数”两个数，并按题设选择目标。

为什么可把最优搜索限制到 PI? 若某解含非 PI 项，至少能删去一个 literal 扩张它，覆盖仍合法，成本不会增加。重复后得到 PI。因此至少存在一个由 PI 组成的最优解；这没有说全部 PI 都要保留。literal 计数是逻辑代理成本，不等于标准单元面积、门延迟或晶体管数量。来源：L25 PDF pp.25–27。

5.2 示范一：把所有合并做完，别漏 PI

以下是原创教学实例，变量顺序固定为abc，a 为最高位：$F = \{0, 1, 2, 5, 6, 7\}$，$D = \emptyset$。先列 0-cube 并按 1 的个数分组：0 个为 `000`；1 个为 `001,010`；2 个为 `101,110`；3 个为 `111`。第一轮只合并恰好一个固定比特不同的项：

```
000 + 001 → 00-  = P = a'b'   覆盖 {0,1}
000 + 010 → 0-0  = Q = a'c'   覆盖 {0,2}
001 + 101 → -01  = R = b'c    覆盖 {1,5}
010 + 110 → -10  = S = bc'    覆盖 {2,6}
101 + 111 → 1-1  = T = ac     覆盖 {5,7}
110 + 111 → 11-  = U = ab     覆盖 {6,7}
```

第二轮要求横杠位置相同、另一个固定比特恰好不同。`00-` 与 `11-` 相差两位，不能合并；`0-0` 与 `1-1` 也相差两位；`-01` 与 `-10` 亦然。因此六项全是 PI，没有 2-cube。每个已参与合并的父项都打勾，同一个新 cube 经不同路径生成时只保留一份；最终收集**所有轮次**中未被继续合并的项，不能只取最后一轮。

最常见错法是看到 `00-` 和 `0-0` 只有“一个位置看起来不同”，便写 `0--`。它实际上多覆盖了 `011`，而m_3 是 off-set，立即非法。验证手段就是把 cube 展开为点集，检查新增点，而非凭图形印象判断。算法依据：L25 PDF p.26；T25 Tutorial 2。

5.3 示范二：覆盖表与选择变量，不把项变量当输入

在六个 PI 的表中，m_0 可由 P 或 Q 覆盖，m_1 由 P 或 R，m_2 由 Q 或 S，m_5 由 R 或 T，m_6 由 S 或 U，m_7 由 T 或 U。没有只出现一次的列，所以没有 EPI。这不代表无解，只代表必须作选择。

为每个 PI 引入一个选择变量（P 表示“选 P”而不是输入值），覆盖条件为：

$$C = (P+Q)(P+R)(Q+S)(R+T)(S+U)(T+U) = 1.$$

这就是常称的 Petrick 形式：**每列求 OR，所有列求 AND**。展开并用$X^2 = X$、$X + XY = X$ 吸收，得到若干选择集合；逐个按 literal 成本比较即可。此例每个 PI 都覆盖两个点，所以至少要$\lceil 6/2 \rceil = 3$ 项。选择PST 覆盖六点，选择QRU 也覆盖六点，两者都恰好 3 项、6 literal，达到了下界，因而都最优。

得到两种 MinSOP：$\bar{a}\bar{b} + b\bar{c} + ac$ 与$\bar{a}\bar{c} + \bar{b}c + ab$。不要因标准答案选了第一种而否定第二种。也不要机械展开很大的选择式：每乘入一列就吸收包含别人的冗余乘积，或保留成本下界做分支剪枝。小题可以枚举，规模大时这一组合步骤本身会很贵。最后用 8 行真值表验证两式均在 0、1、2、5、6、7 为 1，在 3、4 为 0；正确性验证与最优性下界是两份不同证据。

5.4 EPI 与支配：删行删列方向要有理由

另取$F = \{0, 2, 3, 7\}$，PI 为$A = 0-0$、$B = 01-$、$C = -11$。点 0 只有 A 能盖，点 7 只有 C 能盖，故先选 A、C，覆盖全部 F，B 不必选。这个小例用于复核课程中的必要项概念。

列规则（约束）：设覆盖列m_i 的可选行集合为R_i。若$R_i \subseteq R_j$，覆盖更难的m_i 时必然覆盖m_j，因此可删去较容易的列m_j，保留m_i。背“谁支配谁”的名字容易混乱，最好在纸上写集合包含关系。

行规则（候选）：若行 A 的覆盖集合包含行 B，且$cost(A) \le cost(B)$，则在寻找至少一个最优解时可删 B。必须检查成本！一个含很多 literal 的大候选不一定优于一个便宜的小候选。若同成本同行覆盖，任选保留一个只会保留部分等价最优解；要列出所有最优解时应记录替代关系。

删除列后重新找 EPI，选择后再删除已覆盖列，循环至不能约简。若余表仍非空，贪心“选覆盖剩余点最多的行”是启发式，不是精确证明。抽象反例：宇宙为 1…6，A 盖 {1,2,3,4}，B 盖 {1,2,5}，C 盖 {3,4,6}，成本均为 1。贪心先选 A 后还需 B、C，共 3；而 B+C 只需 2。它说明覆盖策略的局限，并非声称这三行已是某特定布尔函数的完整 PI 表。

5.5 Don’t-care 与多输出：扩大候选，不扩大必须完成的任务

在 Quine 合并阶段使用$F \cup D$ 生成候选，在覆盖表中只为F 建必须覆盖的列；纯覆盖 D 的 PI 不值得单独选择。例：$F = \{1, 3\}, D = \{5, 7\}$，三变量函数可选$f = c$，只需 1 literal；但实现不用覆盖 D 也合法，$f = \bar{a}c$ 同样满足规范，只是不够便宜。若某个 don’t-care 是“正常运行不出现”，必须有可达性或输入约束依据，不能因为题目没列就随意把它当 D。

多输出共享优化要区分“函数表达式里的出现次数”和“实际创建的共享乘积项”。若$f_1 = ab + ac$、$f_2 = ab + ad$，分别计算输入 literal 是 4+4=8；实现时将$t = ab$ 共享，AND 平面只有ab, ac, ad，输入 literal 共 6，另有四个输出连接。这个成本统计只适合声明了 AND/OR 平面的模型，不应拿 6 去对比另一方案的总门输入数。

一个同时供给两个输出的乘积项必须蕴含$f_1 f_2$（若有 D 则逐输出检查允许集合）。单输出 PI 的独立最小化可能错过共享机会；有时缩小某个 cube，反而使它能同时服务多个输出，整体更省。来源：L25 PDF pp.35–37。先做题目明确的单输出成本，再处理共享，避免把最少 literal、最少乘积项和最少连接数混为一谈。

5.6 考场步骤、反证与迁移

把答案分成五段：① 写变量序、F/D和成本；② 给分组、每轮合并、未合并项；③ 列 PI 覆盖表并说明必要项；④ 给覆盖选择及成本；⑤ 用 off-set/真值表复核。即使最后选择错了，这种结构也让中间方法可检查。近年卷确实包含 Quine、覆盖表、cube 计数等题型，但旧题分布只能说明训练价值，不能证明本次补考权重。

迁移到真实综合时，课上两级最优只是一个局部目标：标准单元有扇入限制，扇出会带来负载，增加共享可能拉长路径；综合工具还要在多级网络上考虑时序和库映射。可以把 QM 想成“列出所有合法积木，再精确选覆盖”；这有助于理解 set cover，但硬件优化还有连线与延迟，并非纯集合游戏。

验收线：离开答案，在 25 分钟内做一题 3–4 变量 QM，能解释“漏 PI、错支配、把 D 当必盖、把 irredundant 当 minimum”四类错因。若失败，先回 m02 练 cube 展开和 cofactors；若已能独立写出完整表，可跳过示范，直接做本章独立题并隔一天换变量重做。不要以“我看懂了解析”代替这项能力证据。

5.7 练习：先作答，再查附录

5.7.1 m03-q1 ·补全·3 分

在三变量空间中，00- 覆盖哪些 minterm？能否与 0-0 合并成 0–?

5.7.2 m03-q2 ·补全·2 分

主示范中六个 PI 每个覆盖 2 个必需点；最少至少要几项?

5.7.3 m03-q3 ·独立·4 分

独立求 F={0,2,3,7}, D= 空 的全部 PI、EPI 与 MinSOP，变量 abc。

5.7.4 m03-q4 ·独立·4 分

F={1,3}, D={5,7}，变量 abc。为什么 f=c 合法？把 5、7 放进必须覆盖的列会造成什么概念错误?

5.7.5 m03-q5 ·迁移·4 分

有两行：A覆盖 {1,2,3}、成本 5；B 覆盖 {1,2}、成本 2。能仅凭 A 覆盖更多就删 B 吗?

5.7.6 m03-q6 ·迁移·4 分

比较 f1=ab+ac、f2=ab+ad 的分别实现与共享 ab 实现。报告你采用的成本口径，并提出一个共享可能变慢的原因。

5.8 本章来源

- L25：2025-10-31；I.3，印刷 pp.21–33 / PDF pp.25–37
- T25：Tutorial 2，印刷 pp.13–24 / PDF pp.15–26；Tutorial 3 P1、P4，PDF pp.27、29–33、38
- F086：文件名 19_retake；卷内日期 2020-06-29；Problem 1c，PDF p.3
- F088：2021 exam；Problem 1b，PDF p.3

6 04 启发式优化与恒真检验

每一次删项、扩张、缩小，都要证明仍然合法

关键问题：为什么“单独能删”不代表“可以一起删”，为什么先变复杂有时能通向更好的解?

学习目标

- 推导 expand、reduce、remove 的合法性条件
- 把 cube 包含转成 cofactor 恒真问题
- 解释配置图的可行解、邻域、局部与全局最小
- 对每步修改给出证明或反例而非只写是/否

先修：m02、m03；**横向联系**：m06、m05。

6.1 为什么不用穷举解决所有问题

QM先产生候选 PI，再解决覆盖选择。变量变多时，真值表、PI 数与选择组合都可能迅速增长。启发式方法不承诺枚举所有可能，而是在合法覆盖之间移动：把一份 cover 看成一个状态，literal 数等看成成本，把一次允许的修改看成边。这就是 configuration graph。顶点必须是合法实现；非法低成本表达没有资格参加比赛。

在只保留改进方向的图中，没有可继续改善的相邻解就是局部最优；全局最优要求所有可行解都没有更小成本。两者取决于邻域定义。只允许“直接删一整项”时的局部最优，可能通过先 reduce 再 expand 到达更好解。允许成本持平边时还要处理平台与循环，不能把“出度为 0”脱离课程画图规则随意推广。

可以类比整理行李：直接丢东西不可能减少必需品，但重新打包能让空间更省。类比边界是电路中每一次改动都有逐输入的功能约束，不能凭直觉宣布“差不多”。工业工具的搜索顺序、停止条件和成本模型更复杂；此章演示的是 L25 的局部操作与合法性论证，不声称完整实现 ESPRESSO。

6.2 三种修改的严格条件

设当前完全指定函数$f = c + h$，h 是除当前 cube c 外其余项的和。**Remove** 把 c 整项删掉，得到 h；合法当且仅当$c \subseteq h$，即$h|_c \equiv 1$。一定要先从 h 里拿掉 c，否则拿$f|_c$ 检查永远为真，会错误地证明所有项都可删。

Expand 删除 c 中一个 literal。若原项为$c = lq$，新项为 q，则增加的区域是$\bar{l}q$。完全指定函数上合法当且仅当$\bar{l}q \subseteq h$，也等价于$q \subseteq f$。后一个条件可用$f|_q \equiv 1$ 检查。若有 D，要求新区域不碰 off-set，已有实现值在 D 上改变仍合法；这时不要强求新旧完整真值表在 D 上相同。

Reduce 给 c 加一个 literal l，变成 cl，移走的区域是$c\bar{l}$。完全指定情况下必须有$c\bar{l} \subseteq h$；有 D 时只要被移走的必需 on-set 点仍被其他项覆盖。缩小会增加此项 literal 成本，却可能调整其他项的可扩张边界、制造新的共享或冗余。

三步的共性是“只检查新增/移走区域”，不是每次从头猜一个函数。做题时写清 c、h、新项与待证 cofactor，是最稳的得分结构。来源：L25 PDF pp.39–40。

6.3 完整示范：从三个点到两个大 cube

取三变量函数$f = abc + ab\bar{c} + \bar{a}bc$，on-set 为 {3,6,7}，没有 D。初始 cover 有 3 项、9 literal。

第一步扩张$abc \to ab$。新增区域是$ab\bar{c}$，它已由其他项覆盖；也可算$f|_{a=1,b=1} = c + \bar{c} + 0 = 1$，所以合法。覆盖变成$ab + ab\bar{c} + \bar{a}bc$，literal 暂时从 9 降到 8。

第二步删除$ab\bar{c}$。此时其余项 h 包含 ab，固定 a=1,b=1,c=0 时$h = 1$，合法。成本变成 2 项、5 literal。

第三步扩张$\bar{a}bc \to bc$。$f|_{b=1,c=1} = a + 0 + \bar{a} = 1$，所以合法。得到$f = ab + bc$，2 项、4 literal。若尝试再把 ab 扩张成 a，取 100 时新式为 1、原式为 0，失败；若扩张成 b，取 010 即失败。给反例比写“不是恒真”更有说服力。

再演示一次合法 reduce：把 ab 缩成$ab\bar{c}$。被移走的 abc 仍被 bc 覆盖，所以合法，得到$ab\bar{c}+bc$，5 literal。它本身没有改善成本，只展示允许的反向移动；不能把“reduce 有时帮助搜索”误写成“每次 reduce 必然改善”。最后穷举八个点验证所有合法中间步骤保持 {3,6,7}。

6.4 关键反例：分别可删，不能一起删

令$c_1=ab$、$c_2=ac$，其余项$h=a\bar{b}c+ab\bar{c}$，所以$f=c_1+c_2+h=a(b+c)$。单独删除c_1 时，其独有区域$ab\bar{c}$ 已在 h 中，公共区域 abc 仍由c_2 保留；因此合法。单独删除c_2 时同理，它独有的$a\bar{b}c$ 由 h 保留，公共区域 abc 仍由c_1 保留。

但若同时删去两者，只剩 h，输入 111 时 h=0 而原 f=1。因此结论是：**各自针对原 cover 的删除证明不能直接叠加。**如果顺序删除，做第二次判断时必须使用第一次修改后的 cover，重新构造“其余项”。同样的问题也会出现在分别删除两个 literal 的扩张操作中：各自增加的集合合法，不代表一次删除两者引入的更大交叉区域也合法。

这正是训练题喜欢要求“分别、同时”的原因；它考的是逻辑量词与当前状态。调试算法时保存每步 cover、成本与失败反例，能够定位究竟是修改逻辑错了，还是拿了过期的证明。把合法性写成纯函数并在小 n 上做穷举，是比观察最终式子更可靠的程序验证方式。

6.5 怎样高效做恒真检验

要证明$c\subseteq f$，固定 c 中全部 literal 使其为真，剩下的函数必须恒为 1。若它不是显然的$x+\bar{x}$，再任选剩余变量 x 分成两个 cofactor，两个都为 1 才通过。任一分支变成 0 或出现使输出 0 的输入，就给出了反例。递归求值树与 BDD 是同一个 Shannon 思想的两种组织方式。

若 f 对 x positive unate，则$f_{x=0}\subseteq f_{x=1}$，要证明恒真只需证明更小的$f_{x=0}$ 恒真；negative unate 时检查$f_{x=1}$。这是有前提的剪枝，不能凭表达式看起来“正项多”就套用。例：$f=x+y+\bar{y}$，两个分支都恒真；但$g=x+y$ positive unate，检查 x=0 后得到 y，不恒真，反例 x=0,y=0。

SOP 中出现包含某变量的正负项，不自动证明 binate，因为表达式可能冗余。函数属性要靠 cofactor 比较。对于条件受限输入，还需明确恒真是在整个B^n 还是允许域上；若假设输入约束 A，则要检验$A\Rightarrow(f=g)$，而不是无限制地删掉边界情况。

6.6 方案比较与验收

问题同为求成本小的合法 cover：基线是 QM 完整候选加精确覆盖；替代是启发式局部搜索。前者可在完成搜索时提供指定成本最优证据，但组合爆炸；后者可较早得到好解，且可在预算用尽时返回当前可行结果，但需要承认没有全局最优保证。公平比较应使用同一 F/D、同一成本、相同计算预算，并把“最优性缺口未知”写出来。

课程配置图只是某个邻域下的示意，不代表某工具的实测运行轨迹。要验证程序，先检查每个顶点覆盖 F、不碰 R，再检查每条边确实对应允许修改，最后检查成本变化。局部最优性还需检查全部允许邻居，而不只是运行中恰好访问过的邻居。独立题应能同时交出“判定结果、cofactor 过程、反例或恒真证明”。若仅会背三个操作名，尚未达到补考做题水平。

6.7 练习：先作答，再查附录

6.7.1 m04-q1 ·补全·2 分

测试是否可以从 f=c+h 中删去 c，应检查 f|c 还是 h|c？为什么？

6.7.2 m04-q2 ·独立·3 分

对 f=abc+ab c′ +a′ bc，证明 abc 可以扩张成 ab，并报告最终 ab+bc 的 literal 数。

6.7.3 m04-q3 ·独立·3 分

把 ab+bc 中 ab 缩小为 ab c′ 是否合法？给出被移走区域和覆盖者。

6.7.4 m04-q4 ·独立·4 分

在 f=ab+ac+ab′ c+abc′ 中，分别删除 ab、ac 是否可行？同时删除呢？

6.7.5 m04-q5 ·迁移·4 分

一个启发式程序只测试了 3 个邻居就宣布局部最优，你要求它补充什么？

6.7.6 m04-q6 ·迁移·4 分

设计一个针对 cube 删除程序的小规模验证器，不需写具体语言。

6.8 本章来源

- L25：2025-10-31；I.4，印刷 pp.34–36 / PDF pp.38–40；多输出操作 PDF p.35
- T25：Tutorial 3 P2、P3，PDF pp.28–29、34–37
- F087：2020 exam，Problem 1g，PDF p.7
- F088：2021 exam，Problem 2a，PDF p.5
- SIS92：Berkeley UCB/ERL M92/41 元数据页；仅作 L25 提及工具的背景，不据此声称复现

7 05 多级逻辑与函数分解

从提取公共表达式到压缩 bound variables 的信息

关键问题：为什么两个 LUT 中间的线数由不同余函数的数量决定，而不是由原来的 minterm 数决定?

学习目标

- 画 Boolean network 并识别 fanin/fanout/reconvergence
- 用等价类构造 h 与 g，使 f(x,y)=g(h(x),y)
- 计算最少编码位和有利分解条件
- 区分逻辑共享的 literal 节省与实际时序成本

先修：m02、m03；**横向联系**：m04、m06、m07。

7.1 两级最优并不是所有电路结构里的最优

SOP的两级结构是 “若干 AND 后接 OR”，它的好处是表示和覆盖关系清楚。多级逻辑允许中间子函数被后级重复使用。一个组合 Boolean network 是有向无环图：输入节点、内部函数节点、输出节点，边表示信号依赖。fanin 是直接前驱，transitive fanin 包含所有祖先；fanout 是输出被多少后继使用；reconvergence 是分叉的信号又在后面汇合。

例：$y = ab + ac + db + dc = (a + d)(b + c)$。展开 SOP 有 8 次输入 literal 出现，因式表达有 4 次。若库允许 OR、AND，每项的实现门和层数都要根据目标结构重数，而不是简单以 “字符串变短” 判断速度。共享中间信号$t = b + c$ 能减少重复逻辑，但驱动多个目的地会增加负载。拓扑图的边是功能依赖，不自带固定延迟。

来源 L25 PDF pp.41–42 将多级综合目标表述为更少门/literal、较好的 fanin/fanout。它没有保证每一次提因子都降低关键路径。将软件的公共子表达式消除类比到这里很直观，边界是软件常按指令顺序复用计算，而组合电路各级同时响应变化，共享的传播延迟与物理扇出必须另外评估。

7.2 函数分解：只向后级传递真正需要区分的信息

把输入分成 bound variables x 与 free variables y，希望写成$f(x,y)=g(h(x),y)$。h 只能看 x，输出 z；g 看 z 与原来的 y。对每个固定 x，f 变成关于 y 的余函数$f_x(y)$。若两个 x 值产生完全相同的余函数，它们对后级来说不可区分，可以用同一个 z 编码；若余函数不同，就绝不能共用编码，否则 g 无法还原差别。

设不同余函数数量为μ，至少需要$r=\lceil \log_2 \mu \rceil$ 个中间比特，因 r 比特最多编码2^r 种情形。这是对该固定分区、完全指定函数的最少信息宽度。要实现课程所说的“有利分解”，还要求$r<|x|$，即$\mu \le 2^{|x|-1}$。能够写一个 h 不表示必然减少逻辑：h=x 总是可以，却没有压缩。

这很像对后续处理足够的“摘要”：两份输入只要对所有后续 y 都产生同样反应，就能共用摘要。但摘要必须对**所有** y 保真，而不是对当前采样的 y 看起来相同。构造分解表时把每列视为一整个向量，不能只比较某一行。来源：L25 PDF pp.43–47。

7.3 完整示范：3 位 bound 输入压成 2 位

原创例子：$x=(a,b,c)$、自由变量为 d，定义$f=ab+ac+bc+d(a+b+c)$。固定 abc 后，只剩 d 的函数：000 对应常量 0；001、010、100 对应 d；011、101、110、111 对应常量 1。按 d=0、1 写向量分别是 (0,0)、(0,1)、(1,1)，因此$\mu=3$，最少$r=2$ 位，满足$2<3$。

选择编码如下：余函数 0 用 z1z0=00，余函数 d 用 01，余函数 1 用 11。于是可以构造：

$$z_1=h_1(a,b,c)=ab+ac+bc,\qquad z_0=h_0(a,b,c)=a+b+c.$$

后级$g(z_1,z_0,d)=z_1+z_0 d$。检查三类：00 时恒 0；01 时等于 d；11 时恒 1。编码 10 不会由 h 产生，所以 g 在这一内部码上可按 don’t-care 处理；本例 g(10,d)=1，是一种合法选择。若测试或系统接口允许外部强制 z=10，它是否仍可不关心必须重新看规范，不能把内部不可达默认变成所有模式下不可达。

画图时先画 a,b,c 进入 h1/h0，输出两根线 z1,z0，再让 z 与 d 一起进入 g。验证不要只画箭头：枚举 abc 的 8 值，并对 d=0/1 共 16 个输入检查$g(h(x),d)=f(x,d)$。一共 3 种余函数是证据，2 位是信息论下界，所给 h/g 是达到下界的构造。这三个结论一起才算完整答案。

7.4 编码选择、LUT 容量与 BDD 切割的联系

给等价类分配不同编码，不改变μ，但会改变 h 的表达式和 g 的表达式。正文编码让 z0 等于 OR、z1 等于 majority；若把常量 1 类改为 10，h/g 可能更复杂或更简单，要实际化简才知道。不同余函数必须有不同码；相同余函数允许分配多个码，只是通常不会减少最少位数，有时可让 h 更便宜。L25 PDF p.46 展示了这种编码取舍。

若目标是单输出 k-LUT，一块 LUT 能实现不超过 k 个输入的任意函数，但多输出 h 的每一位通常需要独立 LUT（除非器件明确支持特殊双输出结构）。本例 h1、h0 各需一个 3 输入函数，g 也是 3 输入函数，因此按普通单输出 3-LUT 模型需三块 LUT，不能把“h 画成一个盒子”误算成一块 LUT。原函数有四个变量，不能直接放进一个普通 3-LUT。

若 BDD 的变量顺序先 x 后 y，在 x 层后切开，每种可达的剩余子函数对应一个等价类。统计时要包括越过某些变量的边和直接到达的常量终端；只数某一横排画了几个圆圈可能漏掉余函数。BDD 提供分区探索方法，但选择能同时优化线宽、LUT 数与延迟的分区仍需成本评估。

7.5 深入选学：代数除法、kernel 与 co-kernel

本节帮助连接真实多级综合；十天主线先掌握前面功能分解，不要求把 kernel 枚举当作当前讲义新增必考内容。代数除法试图将一个 SOP 表示写成$f=pq+r$，在项集合和 literal 的形式结构上寻找可提取因子；它比允许全部布尔恒等变换的布尔分解限制更强。

原创例：$f = ab + ac + db + dc + e$。以 cube a 作除数，能被 a 整除的项给商$b + c$，余项是$db + dc + e$。商$b + c$ 没有共同的非平凡 cube 因子，称为 cube-free，因此是一个 kernel；a 是得到它的 co-kernel。同一个 kernel 也能由 co-kernel d 得到。将这两个共同商合并，可写$f = (a + d)(b + c) + e$。

kernel 是“用单个 cube 作代数除法得到的 cube-free 商”，co-kernel 是对应的 cube 除数；不要与 cofactor 混淆。cofactor 固定变量值，会把含相反 literal 的项消为 0，并不等于取某些项的除法商。比如$f = ab + \bar{a}c$，对 a=1 的 cofactor 为 b；但这只是一个巧合式相似，不是两种运算在所有表达式上相同。

原始扩展阅读：Brayton 等，Multilevel Logic Synthesis，Proceedings of the IEEE，1990，§IV.D（印刷 p.272 / PDF p.9）的 kernel 定义与代数方法；2026-11-08 核对该部分。该引用用于方法定义，不把论文历史实验数字当成本网站的性能实测。本文例题是自拟，计数与推导可独立核验。

7.6 公平比较与失败反例

案例问题是压缩前级传给后级的信息。基线直接传全部 x；机制按相同余函数分组；证据是完整分解表；成本包括 h 的逻辑、z 线数、g 的 fanin 和额外级数；适用条件是μ 足够小且新结构满足目标库约束。反例：若每个 x 产生不同余函数，$\mu = 2^{|x|}$，最少 r=|x|，没有输入宽度收益。即使 r 小，复杂 h 也可能让最长路径变差。

比较面积与延迟必须固定同一库、同一输入到达时间、同一输出负载和同一时序约束。站内用 literal 或理想 LUT 数的估算只回答该抽象模型的问题。能力验收是拿到一个分解表，能找等价列、算 r、编码、画 h/g、填 LUT 表并用原输入复算。若只会说“用 BDD 可以分解”，还无法完成课程的构造题。

7.7 练习：先作答，再查附录

7.7.1 m05-q1 ·补全·3 分

4个 bound 变量产生 5 种不同余函数，最少需几位中间编码?

7.7.2 m05-q2 ·独立·3 分

正文例子 abc=010、d=1 时，求 z1,z0 与输出，并用原式复核。

7.7.3 m05-q3 ·独立·3 分

为什么不能把余函数 d 和常量 1 编成相同的 z?

7.7.4 m05-q4 ·迁移·3 分

某人将正文 h=(h1,h0) 画成一个盒子，宣称总共只需两块单输出 3-LUT。找出资源计算问题。

7.7.5 m05-q5 ·独立·4 分

对 f=ab+ac+db+dc+e，写一组 kernel/co-kernel 和因式分解，并说明它不是 cofactor。

7.7.6 m05-q6 ·迁移·4 分

设计一个检验候选分解 h/g 是否正确的办法，并说明仅检查全部 z 编码为什么可能不够。

7.8 本章来源

- L25：2025-10-31；I.5，印刷 pp.37–43 / PDF pp.41–47
- T25：Tutorial 4 P4，印刷 p.40 / PDF p.42；解答 PDF p.52
- F089：内页 DMA 2022（文件名 21_retake_exam）；Problem 1d/1f，PDF pp.3–4
- MULTILEVEL90：1990-02；§IV.D，印刷 p.272 / PDF p.9，kernel/co-kernel 定义与 Table 1；深入选学

8 06 ROBDD：共享余函数的决策图

会建图、会约简、会从图读 SOP/POS，也会承认变量序的代价

关键问题：为什么同一变量序下的最简决策图可以判等价，而换变量序后节点数会改变?

学习目标

- 按指定变量序从 cofactors 构造 OBDD
- 使用删除无效测试与合并同构子图两条规则
- 由到 1/到 0 的路径写 SOP/POS 并求 cofactor
- 用明确计数口径比较变量序与功能分解

先修：m02；**横向联系**：m04、m05、m11。

8.1 图中的一个节点究竟表示什么

BDD的非终端节点表示“检查某个变量 x，若 0 走 low，若 1 走 high”。若子图代表函数 L 与 H，当前节点代表$\bar{x}L + xH$，就是 Shannon 展开。0、1 终端代表常函数。图不是门级网表的直接同义词，而是一种精确的布尔函数表示；若把每个节点实现成 MUX，可以形成一种电路，但实际库映射还可能改写结构。

OBDD 的 ordered 要求所有从根到终端的路径按同一个变量顺序前进，允许跳过不依赖的变量，不能在一条路径反向或重复测试。不是要求每层每条路径都必须有一个节点。例如顺序 a<b<c 时，某条路径可以从 a 直接到 c，但不能从 c 回到 b。若题目给虚实线，先写清哪个表示 0、哪个表示 1；本章文本始终标 low=0、high=1。

画图时用“每个节点标变量、每条边标值、终端标常量”三条纪律。输入原式中的同名变量在图中可以对应多个不同节点，因为它们的剩余子函数不同；只有子图结构相同才可合并。来源：L25 PDF pp.48–49。

8.2 两条约简规则与规范性的前提

第一条规则：若某节点 low=high，结果与该变量无关，删除此节点，所有入边指向共同子节点。这来自$\bar{x}g + xg = g$。第二条规则：若两个节点测试同一变量，low 相同且 high 相同，则它们代表相同子函数，合并为一个节点。要从底向上比较“变量、low、high”三元组，不能只看两个节点标签相同就合并。

把两条规则完全应用，得到 Reduced OBDD。**对于同一布尔函数、相同固定变量序、相同终端/边约定，ROBDD 在同构意义下唯一。**这使等价检查可以归结为图相同或共享管理器中的根节点相同；若变量序不同，形状不同不能证明函数不同。若在不同软件管理器中，节点数字编号本身也不能直接比较。

常量 0 和 1 的 ROBDD 只需对应终端；无须为每个未使用变量保留一层。教学中常见“两个子节点函数看起来差不多”不能当合并依据：必须证明它们等价，或者已经在统一序下约简成同一个子图。Bryant 1986 原始论文的作者注释版给出这一规范表示结果；电子版附有历史勘误，本站不用其中历史实验规模推断现代工具性能。

8.3 完整示范：MUX 的两种变量序

取$f = ab + \bar{a}c$。先用顺序 a<b<c。根 a 的 low 为 c、high 为 b；b 与 c 的 low=0、high=1。这已完全约简：三个不同非终端节点 a、b、c，共 3 个；若把常量终端也计入则共 5 个。不要在计数题里混用两个口径。

```
a: low → c ; high → b
b: low → 0 ; high → 1
c: low → 0 ; high → 1
```

换顺序 b<c<a。b=0 时剩下$\bar{a}c$，b=1 时剩下$a+c$。第一支在 c=0 为 0、c=1 为$\bar{a}$；第二支在 c=0 为 a、c=1 为 1。因此有：

```
b : low → cL ; high → cH
cL: low → 0  ; high → aN
cH: low → aP ; high → 1
aN: low → 1  ; high → 0
aP: low → 0  ; high → 1
```

两个 a 节点的 low/high 互换，不能合并。于是非终端 5 个，连终端共 7 个。此计数**不使用 complement edges**；若工具采用补边，统计可能不同，不能据此说工具或手算错。功能仍相同，穷举八种输入可证。选择变量放前面使 MUX 分解自然，是这个函数的结构证据；并非“把名称叫 selector 的变量放最前总最优”的普遍定理。

8.4 从图读函数、余因子与反例

到达 1 终端的每条根路径给出一个乘积项：遇到 0 边写负 literal，遇到 1 边写正 literal，跳过的变量不写。把所有此类路径相加得到合法 SOP，不保证是 MinSOP。对上面 a 优先的图，到 1 的路径分别为 a=0,c=1 与 a=1,b=1，得到$\bar{a}c+ab$。

到达 0 的路径描述补函数的 SOP：$\bar{f}=\bar{a}\bar{c}+a\bar{b}$。整体取反得到$f=(a+c)(\bar{a}+b)$，这是 POS。常见错误是把到 0 的路径直接“相加”当成 f，或者取反时忘记把 OR 变 AND。每条路径本质是一个部分输入约束，不要求所有变量都有 literal。

求$f|_{a=1}$ 时，在图中固定 a 选择 high，得到 b；求$f|_{b=0}$ 时，即使 b 不是根，也要在所有出现 b 的可达节点走 low 后重新约简，得到$\bar{a}c$。找到使函数为 0 的路径就是反例生成；找到使$f\oplus g=1$ 的路径，就是 f 与 g 不等价的具体输入。不要只写“图不同”，最好交出输入与两个输出。

8.5 用 BDD 证明恒真，连接包含与分解

要证明函数恒真，可以递归检查 cofactors，或把它构造成 ROBDD 后检查根是否约简到 1。原创例$h=(a+b)(a+\bar{b})+\bar{a}$：a=1 时 h=1；a=0 时$h=b\bar{b}+1=1$。根 a 的两支都为 1，删除根，最后只剩 1。也可以先用分配律得$(a+b)(a+\bar{b})=a$，再得 h=1；两种方法互为核验。

包含关系$c\subseteq f$ 可用$c\bar{f}=0$ 检查，也可固定 c 后证明$f|_c=1$。等价关系用$f\oplus g=0$。这些都是将复杂问题转成常量图判定，正是 m04 合法性检查与 BDD 的横向联系。

如果变量序先 bound x 后 free y，经过全部 x 判断后能够到达的不同剩余子图代表不同余函数。其数量μ 决定 m05 中间编码的下界。但路径可能跳过变量或直接到常量，需要统计**所有穿过切面的边所指的不同剩余函数**，而不是只数图某一行的节点。图压缩与函数信息压缩共享同一个“相同余函数可以复用”的思想。

8.6 成本、异常与可验证的模型边界

ROBDD不保证对所有函数都很小。某些变量序可能导致大量不同余函数，图膨胀；即使挑选变量序，某些函数仍有很大的表示。节点数较少也不等于实际门数、面积或延迟一定较少。构图算法还需要唯一表、缓存等管理结构，这些开销不会出现在手画图里。

一个可复现实验应固定函数、变量集和补边约定，只改变变量序；记录非终端节点数与完整真值表一致性。对常量函数期望 0 个非终端，对单变量函数期望 1 个，MUX 的上述两个顺序期望 3 与 5。这些是手算与边界验证，不是硬件性能基准。若节点数异常，先查没有删 low=high、没有共享相同三元组、错把边值交换、跨顺序使用缓存四类问题。

延伸来源：R. E. Bryant, Graph-Based Algorithms for Boolean Function Manipulation，IEEE TC C-35(8)，1986，pp.677–691；作者注释电子版 PDF 28 页，阅读 §1–2（PDF pp.1–7）及变量序讨论，2026-11-08 访问。本文只借用机制与定理，示范计数为本课程自己推导。验收时应能闭卷完成给定顺序构图、约简、读式和反例，而不是仅记住“BDD 很快”。

8.7 练习：先作答，再查附录

8.7.1 m06-q1 ·补全·2 分

某节点变量 x，low 与 high 都指向同一个子函数 g。约简后是什么？

8.7.2 m06-q2 ·独立·2 分

对 f=ab+a′c，顺序 a<b<c、无补边时有多少非终端节点？

8.7.3 m06-q3 ·独立·4 分

同一函数用 b<c<a 时有多少非终端节点？为什么 a 与 a′的节点不能直接合并？

8.7.4 m06-q4 ·独立·3 分

一张图到 0 的路径为 a=0,c=0 与 a=1,b=0。写原函数 POS。

8.7.5 m06-q5 ·迁移·3 分

两个经过约简的 BDD 形状不同，可以断言函数不等价吗？给出条件或反例。

8.7.6 m06-q6 ·迁移·4 分

设计 BDD 实现的三个边界测试和一个非平凡一致性测试。

8.8 本章来源

- L25：2025-10-31；I.6，印刷 pp.44–47 / PDF pp.48–51
- T25：Tutorial 4 P1–P3，PDF pp.39–41、44–51
- F086：文件名 19_retake；卷内日期 2020-06-29；Problem 1f，PDF p.4
- F087：2020 exam；Problem 1d，PDF p.4
- BRYANT86：1986 原刊；作者注释电子版 §1–2，PDF pp.1–7；固定序 ROBDD 规范性与变量序限制

9 07 FSM：状态建模、最小化与编码

相同当前输出不够，必须对所有未来输入都不可区分

关键问题：怎样证明两个状态可以合并，并把合并后的图变成可实现的触发器输入方程？

学习目标

- 将文字规格转为确定且完整的 Mealy/Moore 状态表
- 删除不可达状态并逐轮细分等价类

- 构造能区分两个状态的输入序列
- 完成编码、下一状态与输出方程并验证复位/非法码

先修：m02；**横向联系**：m05、m08、m13。

9.1 状态是过去信息的摘要，输出时间必须说清

组合电路输出只由当前输入决定；时序电路还依赖保存的状态。同步 FSM 写成$s_{t+1} = \delta(s_t, x_t)$。Mealy 输出为$y_t = \lambda(s_t, x_t)$，Moore 输出为$y_t = \lambda(s_t)$。本章约定每拍先用当前状态和本拍输入计算输出，再在时钟边界更新状态，避免把下一状态的输出提前算到本拍。

FSM 包含状态集 S、输入字母表 I、输出字母表 O、转移函数 δ、输出函数 λ 和初始状态。输入字母表不一定就是“低电平/高电平”：售货机里的一个字母可代表一种硬币事件，先编码才会成为比特。画 Mealy 图时输出写在边上 输入/输出；Moore 输出写在状态内。看到一个图先检查每个状态对每种允许输入是否有唯一转移。

类比记账：状态保存“迄今欠多少钱”，而不是保存全部投币历史。这解释了最小化为何可能；边界是状态还需保存一切会影响未来输出的信息，例如退币模式、超时状态。若两段历史有同样金额却处理退款方式不同，它们不能仅凭余额相同就合并。来源：L25 PDF pp.52–56。

9.2 示范一：从文字规格画出三状态售货机

原创规格：商品价格 3 单位，每次事件投入 1 或 2 单位；达到或超过 3 时当次出货、不找零，并回到余额 0。一次事件只允许一种硬币，暂不建模无硬币/取消/超时。状态 S0、S1、S2 代表未出货的余额 0、1、2。输出 v=1 表示本次出货。

```
S0: 投 1 → S1 / v=0 ; 投 2 → S2 / v=0
S1: 投 1 → S2 / v=0 ; 投 2 → S0 / v=1
S2: 投 1 → S0 / v=1 ; 投 2 → S0 / v=1
```

这里采用 Mealy 模型，出货发生在达到阈值的转移。若用 Moore，需要一个带出货输出的状态并定义何时回到 S0；转换会影响输出对应的周期，不能只把边上的 1 挪进一个圈而忽略时序。若添加找零输出 r，S2 接受投 2 时总额 4，需同时出货并 r=1；它和 S2 投 1 的转移终点相同，但输出不同，不能因终点相同就合并标签。

验证规格要覆盖边界：余额 2 投 1 刚好达到，余额 2 投 2 超额，出货后下一枚币属于新交易。若把输入 0 编成“投 1”，则 0 不是“未投币”；若真实系统允许 idle，需要扩展输入字母表或加 valid。完整描述这些边界，比凭图形看起来像售货机更重要。

9.3 等价的定义与分区算法

两个状态等价，当且仅当从它们出发，对**每一条有限输入序列**，产生相同输出序列。当前输出相同是必要但不充分条件。对完全指定确定性 Mealy 机，先按所有单步输入对应的输出向量分组，得到一等价分区Π_1；然后在每组内部比较每种输入的下一状态所在的旧分区。

若同组两个状态某个输入的后继落在不同旧组，就把它们分开。重复细分，直到新旧分区一致，才得到稳定等价类。每轮比较必须使用**上一轮统一完成的分区**，不能在一行刚拆开后就让后面的行使用更新结果而前面的行不重算。它通常仍可能走向稳定结果，但你写的“第 k 轮”证据会失去定义。

最小化前先从指定初始状态做可达性遍历，删去不可达状态；若题目要求全状态机器等价而非初始行为等价，则要说明是否允许这一步。Moore 机通常首先按当前状态输出分组；不要直接套 Mealy 的双输入输出向量。对不完全指定 FSM，涉及兼容性而不只是普通等价关系，本章算法范围明确为完全指定确定性机器。来源：L25 PDF pp.57–59。

9.4 示范二：一轮不够，完整细分并找区分序列

原创 Mealy 机初始 A，表中每项为“下一状态/输出”：

```
状态      输入 0      输入 1
A          B/0         C/1
B          A/0         C/1
C          D/0         A/0
D          D/0         B/1
E          E/1         E/1
```

从 A 可达 B、C、D；E 不可达，先移除。A、B、D 的输出向量都是 (0,1)，C 是 (0,0)，所以$\Pi_1 = \{\{A, B, D\}, \{C\}\}$。记这两个旧组为 K、L，则 A、B 的后继组签名是 (K,L)，D 的签名是 (K,K)，故第二轮拆出 D：$\Pi_2 = \{\{A, B\}, \{C\}, \{D\}\}$。

第三轮 A 与 B 在输入 0 时均回到 {A,B}，输入 1 时均到 {C}，输出相同；其他组为单状态，因此$\Pi_3 = \Pi_2$，停止。最小机的三个状态记 X={A,B}、Y={C}、Z={D}：X 在 0→X/0、1→Y/1；Y 在 0→Z/0、1→X/0；Z 在 0→Z/0、1→X/1。

为什么 A 与 D 虽然一步输出相同却不能合并？输入序列 11 就能区分：A→C→A 输出 10，而 D→B→C 输出 11。两拍差异就是第二轮拆分的具体证据。A 与 B 可合并的证据则是稳定分区条件，不是因为试了几条序列都相同。对输入序列 1011，从 A 原机输出 1011，新机从 X 也输出 1011；这是一条回归例，不能代替稳定分区证明。

9.5 从三状态图到二进制逻辑：别停在画圈

对最小机编码 X=00、Y=01、Z=10，使用两个触发器 q1,q0。对六个“合法状态 × 输入”列出下一状态与输出。X 输入 0→00，输入 1→01；Y 输入 0→10，输入 1→00；Z 输入 0→10，输入 1→00。输出在 X、Z 时等于 x，在 Y 时恒 0。

现在明确非法码 11 的策略：无论输入如何，下一拍回到 00，输出为 0。由完整八行表可得到一组实现：

$$D_1 = \bar{x}(q_1 \oplus q_0), \qquad D_0 = \bar{q}_1\bar{q}_0 x, \qquad y = \bar{q}_0 x.$$

复算 Y=01、x=0：D1=1、D0=0、y=0，确实到 Z；复算非法 11：XOR 为 0、D0 为 0、y 为 0，满足恢复策略。复位把 q1q0 设为 00。若把 11 当作 don’t-care 来化简，可能得到更短式子，但恢复行为不再保证；要先决定系统是否允许这个取舍。

三状态的二进制编码下界为$\lceil \log_2 3 \rceil = 2$ 个触发器；one-hot 通常用 3 个触发器，组合译码可能更简单。最少状态不必然等于映射后的最小面积或最短路径；编码、输出寄存方式和目标库都会影响结果。不得只比“触发器 2 对 3”便下完整性能结论。

9.6 验证、异常与考试验收

每份 FSM 解答应交四类证据：状态含义；完整转移/输出表；最小化或区分序列；编码后的方程与复位。测试要含初态、边界阈值、连续事件、每条边覆盖和非法码策略。边覆盖只能表明这些转移被执行过，不能自动覆盖所有长序列行为；与形式等价或分区证明的范围不同。

常见异常定位：输出整体错一拍，先查用了当前还是下一状态；相同输入产生两个后继，查边条件是否重叠；永远不离开初态，查输入编码/valid；最小化分组太粗，查只比较了单步输出；方程模拟与图不符，查状态码 bit 顺序、补号和非法码假设。纸上计算与程序都应保留状态轨迹，这使错误可定位到第一拍。

比较二进制与 one-hot 时固定同一规格、目标库、时钟约束、复位策略和输入激励；报告寄存器数、组合面积与关键路径，而不是引用抽象经验当实测。十天验收线：在 30 分钟内完成一个 4–8 状态表的稳定分区，并能写一个两拍区分序列；在额外 15 分钟内给三状态编码和方程，至少复算两行合法码与一个非法码。

9.7 练习：先作答，再查附录

9.7.1 m07-q1 ·补全·2 分

Mealy与 Moore 的输出函数分别依赖什么？本文的 y_t 使用当前还是下一状态?

9.7.2 m07-q2 ·独立·4 分

售货机余额 2 投入 2，商品价格 3 且新增找零功能，出货、找零和下一余额分别是什么?

9.7.3 m07-q3 ·独立·4 分

不看正文，给五状态示范删去不可达状态后的 Π1、Π2、稳定分区。

9.7.4 m07-q4 ·独立·3 分

写出区分示范中 A 与 D 的最短输入序列，并列输出序列。

9.7.5 m07-q5 ·补全·2 分

一个最小机有 5 个状态，普通二进制编码最少几个触发器?

9.7.6 m07-q6 ·迁移·4 分

对正文 D1=x′(q1 XOR q0)、D0=q1′q0′x、y=q0′x，验证非法码 11 的恢复，并说明把 11 直接当 DC 可能改变什么。

9.8 本章来源

- L25：2025-10-31；I.7，印刷 pp.48–56 / PDF pp.52–60
- T25：Tutorial 4 P5，PDF p.42；解答 PDF pp.53–54
- F087：2020 exam，Problem 1f/1h，PDF pp.6、8
- F088：2021 exam，Problem 2c/2d，PDF pp.7–8

10 08 逻辑仿真：把变化放回时间轴

事件队列、毛刺、延迟模型与 delta cycle

关键问题：布尔函数没有错，为什么仿真波形仍会出现毛刺、X或时间不前进?

学习目标

- 区分稳态布尔等价、定时行为与模拟电气行为
- 按事件队列逐步画出门级波形，记录生成时刻与执行时刻
- 预测 transport 与 inertial 对窄脉冲的不同处理
- 解释三值逻辑、组合毛刺和零时间反馈不收敛

先修：m02、m07；**横向联系**：m09、m12。

10.1 先分清你正在模拟哪一层

逻辑综合问“哪些输入产生哪些输出”；仿真还问“输出在何时变”。同一布尔函数可以由不同门网络实现，稳态真值表相同，传播延迟和毛刺却可能不同。课程采用离散事件、门级逻辑模型，时间单位可为 ns，也可为题目定义的基本时间单位。题目只写$\tau = 2$ 时，不要擅自把它解释成实际工艺的 2 ns。

两值模型只有 0/1。三值模型加入 X，表示在模型精度下无法确定。对 AND，$0 \wedge X = 0$，因为另一输入取任何值都无法改变输出；$1 \wedge X = X$。对 OR，$1 \vee X = 1$、$0 \vee X = X$。NAND/NOR 先算 AND/OR 再取反。X 不是一个真实的“第三个电压”，也不是能随意选成 0 或 1 的 don’t-care。多驱动、高阻和强弱驱动还需要更丰富的逻辑域，不能把课程三值表直接当成完整 std_logic 解析规则。

可用“消息队列”类比事件仿真：门收到输入变化才计算，输出变化在未来交付。边界是物理电路并没有中央队列；队列是模拟器为了重建因果关系采用的数据结构。模型也不直接预测亚稳态恢复时间、短路电流或实际脉冲幅度。

若题目给最小/最大延迟而非固定值，例如输入 10 ns 变化、门延迟范围 [2,4] ns，则输出可能在 12–14 ns 内变化。保守模型可在这一窗口标 X，窗口外标确定值；这不是说输出一定经历宽 2 ns 的模拟亚稳态，也不能从区间自动推出均匀概率分布。上升沿、下降沿及 pin-to-pin 延迟可能分别给出，必须按路径和边沿类型选参数。[L25 pp.60–61]

10.2 示范：事件怎样从一个门传到下一个门

设$n = \neg a$，$y = n \wedge b$。初态$a = 0, b = 1, n = 1, y = 1$，反相器延迟 2 ns，AND 延迟 3 ns，连线延迟 0；a 在 10 ns 变 1。把事件写成(s, v, t_{gen}, t_{exe})（依次是信号、新值、生成时间、执行时间），两时间之差就是本门延迟。

1. 10 ns 执行输入变化$a = 1$，只唤醒反相器。根据新输入，产生$(n, 0, 10, 12)$。队列中最早时刻为 12 ns。
2. 12 ns 执行$n = 0$，唤醒 AND，产生$(y, 0, 12, 15)$。此时 y 仍为 1，不能提前画成 0。
3. 15 ns 执行$y = 0$；若没有扇出需要计算且队列为空，本次传播结束。

因此输入到输出的路径延迟为$2 + 3 = 5$ ns。每一步都写“执行了什么、激活了谁、排入什么”，比只画最终折线更容易得步骤分。一个赋值事务不一定产生 event：把当前已经为 0 的信号再次更新成 0，是事务，但值没变化，所以不唤醒只对该信号 event 敏感的进程。

同一时刻可能有多个信号待更新，应先按语言规则完成该轮信号更新，再执行被唤醒的进程；不要任意采用纸面左右门的先后顺序。没有显式物理延迟的信号赋值也不是立刻修改所有读者看到的值，而进入当前时刻后续的 delta 轮次。当前时刻仍有待处理活动，就不能跳到下一个物理时间。

10.3 示范：3 ns 脉冲通过 5 ns 延迟

设输入初值 0，在 10 ns 变 1、13 ns 变 0，延迟 $d = 5$ ns。以下是单驱动、孤立输入脉冲、相同上升/下降延迟的教学模型。

Transport：10 ns 排入输出 1@15 ns；13 ns 排入输出 0@18 ns。15 ns 已有上升沿仍保留，输出在 15–18 ns 出现宽 3 ns 脉冲。它将每个变化按延迟平移，适合描述理想传输延迟；不意味着所有真实连线都完全保留任意窄脉冲。

默认 inertial：10 ns 暂排 1@15 ns；13 ns 发现输入已恢复，早先尚未执行的窄脉冲响应被取消；输出保持 0。可把它理解成“输入必须保持足够久才让响应生效”，但这只是帮助理解的类比。VHDL 的完整规则作用于信号 driver 的 projected output waveform，还涉及显式 `reject` 和同一进程多次赋值；不能把任意程序简化成事后删除所有短脉冲。

默认惯性拒绝时间等于该赋值的延迟；孤立脉冲宽度小于拒绝时间时被抑制。宽度等于拒绝时间的边界应按具体语言事务规则核对，本课简化单脉冲模型保留等宽脉冲。实验应至少测宽度 0、$d - 1$、d、$d + 1$ 和 $d = 0$，并明确时间离散粒度。

另一个易错点：transport 不等于“旧事务永不删除”。在同一 driver 上后来安排的波形，会替换从其首个新事务时刻开始的旧未来波形。例如同一进程同一时刻依次安排 1@10、2@20、3@30、4@15，最终是 1@10 和 4@15；20、30 时刻旧事务被覆盖。[T25 Tutorial 6, Problem 3]

10.4 静态冒险：从布尔函数回到门网络

考虑$y = ab + \bar{a}c$，令 $b = c = 1$。稳态上 $y = a + \bar{a} = 1$，但两条路径延迟不同。设 a 在 10 ns 由 1 变 0：直接 AND 输出 $p = ab$ 延迟 2 ns，在 12 ns 下降；反相器延迟 3 ns，后接 AND 延迟 2 ns，使 $q = \bar{a}c$ 到 15 ns 才上升。末级 OR 延迟 1 ns，采用 transport。

于是 12–15 ns 间 $p = q = 0$，y 在 13 ns 下降、16 ns 恢复，产生 3 ns 低脉冲。这叫静态 1 冒险：理论上应保持 1，实际门模型短暂变 0。它不是布尔代数出错，而是稳态代数没有编码不同路径的到达时间。若末级采用拒绝时间大于 3 ns 的 inertial，此教学模型会滤除脉冲，但不能因此宣布硬件永远无风险。

添加共识项 bc 可得$y = ab + \bar{a}c + bc$；该项逻辑冗余，却在 $b = c = 1$ 时提供始终为 1 的覆盖。在满足单输入变化及相应延迟模型条件下，它消除该静态冒险。代价是额外门、负载和面积；多个输入同时变化、复杂多级网络的动态冒险并不自动消失。同步系统通常也会通过时序约束让寄存器只在稳定窗口采样，但异步复位、时钟、锁存器使能等信号尤其不能随意带毛刺。

10.5 反馈、锁存器与“不前进的时间”

交叉耦合 NOR 锁存器可写$Q = \neg(R + Q_n)$、$Q_n = \neg(S + Q)$。$R = S = 0$ 时保持原状态；$S = 1, R = 0$ 置位；$R = 1, S = 0$ 复位；$R = S = 1$ 使两个输出都变 0，破坏互补关系。随后同时释放至 0，理想零延迟模型可能在同一物理时刻循环：$(0,0) \to (1,1) \to (0,0) \to \cdots$。每个箭头是 delta 推进，而不是经过了 1 ns。

现实电路可能受不对称延迟、噪声与亚稳态影响最终落到某状态；离散逻辑模型的 delta 振荡不是对真实振荡频率的预测。解决问题也不是随便增大模拟器循环上限：先检查非法激励、初值、组合反馈和缺少真实延迟的模型。

GHDL 提供`--disp-time` 观察时间与 delta 编号，`--stop-delta=N` 限制同一物理时刻的 delta 轮数。这是工具实现的调试选项，不是 VHDL 标准规定所有工具都必须同名支持的接口。[GHDL-SIM] 对课程波形题，先声明初态，再逐轮记录被唤醒进程；没有初态时不能擅自选一个看起来方便的稳定值。

10.6 从示范转为独立作答：固定五步

遇到一张波形图，先圈出模型：门延迟还是线延迟、transport 还是 inertial、二值还是三值、是否有初始状态。第二步预测“最后应是什么”，用稳态真值表做总检查。第三步按时间建立事件队列，给未来事务写出生成原因。第四步只在执行时刻改变波形，并处理反馈激活。第五步解释异常：是短脉冲抑制、路径差、X 传播还是零时间不收敛。

比较全电路周期求值与事件驱动时，固定同一网表、激励、逻辑域、延迟模型和输出精度。事件驱动减少无变化区域的计算，却付出队列、调度和数据结构成本；高切换活动下未必占优。编译技术与事件调度不是绝对互斥，不能把“编译型”直接等同于“永远不支持时间”。讲义中的二分是教学抽象，真实工具可组合多种技术。

学习证据应是一张别人可复算的事件表和一段异常解释。读完只能勾选“已阅读”；能在改变一个门延迟后独立重画波形，才记录“已掌握”。回到 T25 Tutorial 5 Problem 1 做 NAND 锁存器事件追踪，再做 Tutorial 6 Problem 2 的 driver 列表，不要凭记忆临摹答案曲线。

10.7 练习：先作答，再查附录

10.7.1 m08-q1 •补全•2 分

链路 NOT 延迟 4 ns、AND 延迟 6 ns，输入在 7 ns 变化且路径始终敏化。输出最早何时改变？只填时间。

10.7.2 m08-q2 •补全•4 分

初始为 0 的信号在 10 ns 变 1、12 ns 变 0，通过默认 inertial 延迟 3 ns。列出待执行事务如何变化，并解释输出。

10.7.3 m08-q3 •独立•3 分

在本章静态冒险例中，只把反相器延迟由 3 ns 改成 5 ns，其余不变并仍为 transport。输出低脉冲宽度是多少?

10.7.4 m08-q4 •独立•4 分

画一个三值 NAND 表，并用“对未知值的两种补全”解释 NAND(0,X) 与 NAND(1,X)。

10.7.5 m08-q5 •迁移•4 分

仿真日志停在 100 ns，delta 不断增加。提出两种原因、验证方法，并说明为什么不能直接据此报出真实电路振荡频率。

10.7.6 m08-q6 •迁移•4 分

设计一个最小实验，比较 transport 和 inertial；要求能揭穿“所有宽度都被抑制”和“所有脉冲都通过”两种错误实现。

10.8 本章来源

- L25：II.1–3，印刷 pp.57–63；PDF pp.61–67
- L25：II.4，印刷 pp.67–68；PDF pp.71–72
- T25：Tutorial 5，印刷 pp.58–64；PDF pp.60–66
- T25：Tutorial 6，印刷 pp.65–71；PDF pp.67–73
- GHDL-SIM：Simulation options：–stop-delta、–disp-time；2026-11-08 所见 7.0.0-dev 文档

11 09 VHDL：读懂并发与赋值的真实含义

从 signal、variable 到进程、综合和自检 testbench

关键问题：为什么同一段 process 里两行看似顺序的赋值，得到的硬件与软件直觉不同?

学习目标

- 根据进程唤醒与 signal 更新规则手算运行结果
- 分别写出组合逻辑、锁存器和边沿触发寄存器
- 发现敏感列表缺项、未赋值分支和多驱动等错误
- 用可重复的输入与断言验证模型，而非只看波形

先修：m08；**横向联系**：m07、m13。

11.1 把文件结构与硬件结构分开

`entity` 说明接口：模块名、输入输出、类型与可配置 generic。`architecture` 说明实现，可以是逻辑表达式、若干并发 process，也可以实例化子模块。不同 architecture 可实现同一 entity，configuration 或实例绑定负责选中具体实现。testbench 通常没有外部端口，在内部生成激励并观察待测模块。

architecture 中并列的语句是并发关系；process 内部语句按书写顺序执行。这里的“并发”是语言语义，并不要求宿主电脑为每个 process 分配一个 CPU 线程。普通进程运行到 `wait` 暂停；带敏感列表的进程在结尾有等效等待。不要同时给一个普通 process 写敏感列表又在其内部写 wait。没有任何等待的无限循环可能把模拟器卡在同一时刻。

课程教程保留了旧版 VHDL 写法，例如使用内部信号读取输出。VHDL-2008 允许读取 out 端口，`process(all)` 也是 2008 特性。考试若没有声明支持新语法，沿用课程的内部信号与显式完整敏感列表最稳妥。本文示例会标注版本，避免用一个版本

的限制解释另一个版本的代码。IEEE1076 来源只读取了公开元数据，完整标准条文需要购买或订阅；语义教学以 L25/T25 及 GHDL 文档交叉核验，不冒称查过未访问的条款号。

11.2 示范：一拍延迟来自哪里

看下面的时钟进程。`s,q,r,a` 均为 bit，初值 $s = q = r = 0$；`v` 是 process 局部变量。`clk` 使用 std_logic，并已通过 `use ieee.std_logic_1164.all;` 引入对应的 rising_edge 函数。每个上升沿调用一次：

```
process(clk)
  variable v : bit;
begin
  if rising_edge(clk) then
    s <= a;
    q <= s;
    v := a;
    r <= v;
  end if;
end process;
```

若第一个上升沿到来前 $a = 1$，执行 `s <= a` 只是安排 s 将来更新为 1，接下来的 `q <= s` 仍读到旧 $s = 0$。变量 `v := a` 立即把 v 变成 1，所以 `r <= v` 安排 $r = 1$。该轮 signal 更新后得到 $s = 1, q = 0, r = 1$；若第二个沿前 $a = 0$，之后为 $s = 0, q = 1, r = 0$。

这不是“signal 永远晚一拍”。零延迟 signal 在后续 delta 生效；此处 q 晚一拍，是因为它只在下一次时钟沿再次读 s。若是组合进程且对 s 敏感，可能在同一物理时间的下一 delta 重新求值。也不是“variable 不产生寄存器”：变量若跨时钟使用旧值，同样可能需要存储。判断硬件应看数据依赖和所有执行路径，而不是看到一个关键字就下结论。

11.3 组合逻辑：完整敏感、完整赋值

一个 2:1 多路选择器可写成：

```
process(a, b, sel) -- 兼容课程显式列表写法
begin
  if sel = '0' then
    y <= a;
  else
    y <= b;
  end if;
end process;
```

每条路径都给 y 赋值，所以输出仅依赖当前输入。若使用 VHDL-2008 模式，可改为 `process(all)` 减少遗漏风险。若写 `process(a,sel)` 却在分支里读取 b，当 sel=1 而只有 b 变化时仿真进程不会唤醒，输出可能保持旧值；综合工具却可能推导出正常 mux，产生仿真与综合差异。

若去掉 else，sel=1 时 y 不被赋新值，语言语义是保留旧值，综合通常需要锁存器。修复方法取决于设计意图：组合逻辑可先写默认 `y <= b;` 再由 if 覆盖；需要真正锁存器则应明确保持条件并检查时序。默认赋值不是随便填 0：它必须符合规格。

单个进程同一轮对同一 signal 安排同一目标时刻的多个赋值，后面的赋值会覆盖相应旧事务，例如 `y <= a; y <= b;` 最终安排 b。不同 process 同时驱动同一 signal 是多驱动问题，不适用“文件中最后一行胜出”。std_logic 有解析函数；bit 没有这种解析，不能依赖它来合并多个驱动。

11.4 寄存器、锁存器和延迟语句的边界

标准边沿寄存器结构是 `if rising_edge(clk) then q <= d; end if;`。异步复位写在进程外层条件，复位信号也进入敏感列表；同步复位只在时钟沿内检查。是否有复位、有效电平、优先级都来自规格。不能为了让波形好看擅自添加复位。

`if en='1' then q<=d; end if;` 若放在组合敏感进程中，描述电平敏感保持元件；相同 if 放在时钟沿内部描述带使能的触发器。画图时先找时钟事件判定，再找保持路径，这比背诵模板可靠。

`after 5 ns` 是仿真时间模型，不应据此推断综合器会自动造出一个精确 5 ns 延迟单元。`wait for` 常用于 testbench 时钟或激励，通常不属于目标 RTL 综合子集。可综合与否还依赖工具、目标器件和支持子集。编译通过仅表示语法和类型等检查通过，不表示行为符合需求，也不表示一定能映射到硬件。

std_logic 的 U 通常表示尚未初始化，X 可由冲突或运算不确定传播，Z 表示高阻；bit 只有 0/1，可能掩盖初始化类问题。教学小例可用 bit 简化推演，工程测试需明确类型与初值，不能把二值实验的成功当成所有未知态都已处理。

11.5 自检比“波形看起来正确”更可靠

先确定输入组合和预期结果，再写断言。对于零延迟组合 mux，可以在 testbench 中依次施加全部 a, b, sel 组合，在模型规定的稳定窗口检查输出。若施加输入后立即 assert，进程可能读到尚未更新的旧输出；可等待足够的 delta 稳定，或在纯零延迟教学 testbench 中等待 1 ns。这个 1 ns 是测试安排，不是电路真实延迟。

```
a <= '1'; b <= '0'; sel <= '0';
wait for 1 ns;
assert y = '1'
  report "mux selected wrong input" severity failure;
sel <= '1';
wait for 1 ns;
assert y = '0'
  report "mux selection failed" severity failure;
```

独立实验可以使用 `ghdl -a --std=08 file.vhd` 分析、`ghdl -e --std=08 tb` 展开、`ghdl -r --std=08 tb --assert-level=error --stop-time=100ns --wave=tb.ghw` 运行。代码中的 entity 名须与 tb 匹配；三步标准模式要保持一致。这些是可复现实验说明，网站里的演示不是 GHDL，也不声称支持任意 VHDL 程序。

验证至少包含正常选择、只有未选中输入变化、只有选中输入变化、sel 边界切换、初始状态。时序电路还需复位、保持、连续多拍和使能停止。将断言失败定位到“输入施加、更新轮次、预期表达式”三个环节，而不是先改预期值去迎合错误输出。

11.6 案例：一个 mux 为何三种描述都能成立

问题是实现同一个 $y = \bar{s}a + sb$。基线为门级结构描述：显式 NOT、AND、OR，方便观察路径和毛刺；行为式条件赋值简洁，适合说明功能；process 写法适合更复杂控制。统一比较前，要让三种描述使用相同类型、相同延迟假设与同样激励。一个有门延迟、另一个零延迟时，波形不同并不能证明功能不等价。

代价与适用条件：结构式暴露实现细节，修改成本高；行为式把技术映射交给综合器，更易维护，但不能仅从源码推断最终门数量和时序。反例是测试某路径冒险：零延迟条件赋值模型往往没有对应内部节点，不能用它证明门网络无毛刺。验证办法是先穷举稳态真值表检查功能，再对带延迟实现单独做事件仿真与时序检查。

完成本章的证据是：能口头说明“进程内顺序、进程间并发、signal 延后更新”三者如何同时成立；能在纸上预测两次时钟沿的 s/q/r；能指出一个遗漏敏感信号和一个未赋值分支。若这些已熟悉，可跳过语法速读，直接做后面的调试与迁移题。

11.7 练习：先作答，再查附录

11.7.1 m09-q1 •补全•4 分

本章 s/q/r 例中初始 s=0；连续两个上升沿前 a 依次为 1、0。写出每个沿后的 (s,q,r)。

11.7.2 m09-q2 •补全•2 分

进程同一次执行依次写 x <= 1 after 5 ns; x <= 2 after 5 ns;。同一 driver、同一目标时刻最终计划的数值是多少?

11.7.3 m09-q3 •独立•4 分

代码 process(a,sel) begin if sel= '0' then y<=a; else y<=b; end if; end process; 存在什么问题？给出能暴露问题的激励。

11.7.4 m09-q4 •独立•4 分

process(d,en) begin if en= '1' then q<=d; end if; end process;是组合门、锁存器还是边沿触发器？若规格要求组合 mux，该如何修改?

11.7.5 m09-q5 •迁移•4 分

设计一个反例说明“variable 立即生效，所以不可能综合成寄存器”错误。

11.7.6 m09-q6 •迁移•4 分

一个组合电路刚改输入就 assert 失败，但等 1 ns 后通过。给出两种可能解释和验证办法，不直接宣布电路正确。

11.8 本章来源

- L25：II.4，印刷 pp.64–68；PDF pp.68–72
- T25：Tutorial 6，印刷 pp.65–71；PDF pp.67–73
- IEEE1076：IEEE 1076-2019 官方元数据；仅核对规范身份、范围和发布日期
- GHDL-STD：Implementation of VHDL：VHDL standards；标准模式与部分实现限制
- GHDL-SIM：Simulation options：断言、时间与 delta 选项

12 10 故障模型与最小测试集

检测、诊断、故障等价与覆盖表

关键问题：测试通过能证明什么？怎样用少量输入检测一个明确定义的故障集合?

学习目标

- 区分设计验证、制造测试、故障检测和故障定位
- 建立单 stuck-at 故障字典与 test-fault 关系
- 用集合包含判断等价、支配及删除方向
- 计算带明确定义分母的覆盖率并求小规模最小测试集

先修：m02、m03；**横向联系**：m11、m12、m13。

12.1 测试不是设计验证的另一个名字

设计验证关心“设计是否符合规格”，制造测试关心“做出来的这个器件是否带有目标缺陷”。正确的 RTL 可能被制造成有断线的芯片，错误的 RTL 也可能制造得完全忠实。ATPG 通常假定无故障网表代表期望功能，再寻找可区分正常网表与故障网表的输入，因此不能自动发现规格本身错误。

物理缺陷复杂，课程使用逻辑故障模型降低分析成本。$x/0$ 表示该故障点永久取 0，$x/1$ 表示永久取 1。**单故障假设**表示一次分析只注入一个故障，不表示芯片上物理上绝不可能同时发生多个缺陷。若有 N 个明确定义的故障位置，最直接的 stuck-at 故障表有 $2N$ 项；这不是“所有真实失效模式数”。延迟缺陷、桥接故障、开路和模拟参数漂移可能需要其它模型。

扇出干线 stem 和分支 branch 要分开标号：stem 故障影响全部分支，单个 branch 故障只影响一条连接。在重汇合逻辑中，两者的检测条件可能不同。列故障数前先画出故障位置，特别说明是否包含主输入、输出、stem 和 branch；否则同一电路出现不同的“总故障数”可能只是计数口径不同。

12.2 完整示范：二输入 AND 的故障字典

设 $y = ab$，故障位置只有 a, b, y，共六个故障 $a/0, a/1, b/0, b/1, y/0, y/1$，输入排列为 $ab = 00, 01, 10, 11$。正常输出依次为 0,0,0,1。每次只改变一个故障点，然后与正常输出异或，得到检测关系。

```
输入      检测到的故障
00        y/1
01        a/1, y/1
10        b/1, y/1
11        a/0, b/0, y/0
```

例如 01 时 $a/1$ 把实际 AND 输入改成 11，故障输出 1 与正常 0 不同，所以被检测；同样输入下 $b/0$ 只让 00，输出仍 0，所以不可检测。做表时不要因为“故障点的值错了”就直接打勾，还要看错误是否到达可观察输出。

故障字典保存每个故障电路的响应；覆盖表保存“响应是否异于正常”。对于多输出，至少一个可观察输出不同时检测成立，可写$R(t,f) = \bigvee_j (y_j(t) \oplus y_j^f(t))$。覆盖表丢掉了差异发生在哪个输出等细节，足够用于检测覆盖，但做更精细诊断时可能需要完整响应向量。

此例是本课程自行构造的穷举基准，不是某道历年题的标准答案；你应能用四行真值表独立复算全部六列。

12.3 检测、等价、支配：先写集合，再看名称

定义 $F_t = \{f : R(t,f) = 1\}$ 为测试 t 检出的故障集合；$T_f = \{t : R(t,f) = 1\}$ 为能检出故障 f 的输入集合。若 $T_{f_1} = T_{f_2}$，两故障在该输入/观察域下检测等价。AND 例中 $a/0, b/0, y/0$ 均只有 11 能检出，故可保留一个代表用于生成检测测试。等价不表示物理缺陷位置相同。

若 $T_{hard} \subseteq T_{easy}$，任何检出 hard 的测试都能检出 easy；为覆盖所有故障，只要保留 hard 的测试要求即可。AND 例中 $T_{a/1} = \{01\} \subseteq T_{y/1} = \{00, 01, 10\}$，所以检出 $a/1$ 也保证检出 $y/1$。不同资料对“谁支配谁”的命名方向可能不同，考试答案写出集合包含关系和“保留谁”最可靠。

覆盖表中测试行也可支配：若同成本测试 t_A 检出的故障是 t_B 的子集，则可删掉 t_A 这个较弱选择。不要混淆测试行与故障列：删测试时保留覆盖更多的行；删故障要求时保留更难满足的列。带权成本或附加诊断约束时，要重新核查该删除是否仍保持目标。

检测是区分“正常与故障”；诊断是区分“故障 f1 与故障 f2”。如果两故障在所有可施加输入上的可观察响应都相同，增加相同观察域中的测试次数也无法区分它们，需要更多观察点或改变故障模型。

12.4 求最小测试集：与逻辑覆盖相同的数学骨架

给每个候选测试一个选择变量 $u_t \in \{0,1\}$。对每个待检故障 f，至少选择一个 $t \in T_f$，因此整体条件为

$$C = \prod_{f \in F} \left(\sum_{t \in T_f} u_t \right) = 1.$$

这个乘积中的和与积是布尔 OR/AND。展开并吸收后，选择变量数最少的乘积项对应最小测试集；它与 QM 后面的覆盖选择有直接联系。不能把“去掉任何一条就不完整”的不可冗余集合，误叫成“全局最少条数”。

AND 例中 $a/1$ 只能由 01 检测，$b/1$ 只能由 10 检测，$a/0$ 只能由 11 检测。因此 01、10、11 都必选，三条已经覆盖其余故障，最小测试集唯一为 $\{01, 10, 11\}$。这是严格下界加可行构造的证明，不只是贪心算法给出的一个结果。

大表可用贪心：每次选新增覆盖最多的测试，删去已覆盖故障后重新计算新增数量。有成本时可比较“新增覆盖/成本”。贪心快，但不保证最少，且并列选择可能影响后续结果。作答要写清并列处理与剩余集合，让别人能复现。小表尽量用必选项、支配、吸收或穷举核对最优性。

12.5 覆盖率的分母决定你到底声称了什么

以原始故障集合 F 为分母，故障覆盖率为 $|F_{detected}|/|F|$。在 AND 例中只用测试 11 检测 3/6=50%；用 01 与 11 检测 5/6，仍漏 $b/1$。若先做等价折叠得到四个代表，测试 11 检出 1/4=25% 的代表，这不能直接拿来与原始 50% 相比。报告要注明 collapsed 还是 uncollapsed，并在需要时按等价类大小还原。

若存在已证明不可测故障 U，某些工具另外报告以可测故障为分母的指标 $|F_{detected}|/(|F| - |U|)$。例如 100 个原始故障、90 个检出、10 个已证明不可测，原始覆盖 90%，可测域覆盖 100%。若 10 个只是 ATPG 超时，不能把它们自动放入“已证明不可测”。教材和工具对 fault coverage/test coverage 的命名可能不同，必须写出分子分母。

100% 单 stuck-at 覆盖只说明在该模型、故障位置与观察约束下所有故障被覆盖；不证明所有真实芯片都无缺陷，不证明所有延迟故障被检出，也不代表诊断分辨率 100%。这是证据边界，不是测试无用。工程价值在于以统一、可计算的模型指导测试质量和成本权衡。

12.6 案例：功能测试与结构测试怎样配合

问题是确认一个组合数据选择模块的制造正确性。基线只运行若干“常见功能输入”，成本低且容易理解，但内部某些逻辑长期不被激活。结构测试先列 stuck-at 位置，再用 ATPG 找专门的激活/传播输入；证据是覆盖表中哪些列从未被命中，而不是“输入看起来够多”。

成本包括额外测试向量、测试时间和故障仿真。适用条件是存在可访问的输入输出及可信无故障网表。反例是一个物理延迟故障：即使静态 stuck-at 测试全部通过，也可能只在工作频率下失败。因此设计验证、功能测试、结构故障测试和 at-speed 测试承担不同证据角色。

验证方案：固定同一故障集合，先统计基线向量集的覆盖；追加结构向量后重新故障仿真；同时保留每条新增向量的增量覆盖和总耗时。若比较两个方案，统一故障折叠规则、不可测处理、测试频率和覆盖统计范围。独立迁移时试着把 AND 换成 NAND，重新做表，观察等价类怎样改变，而不是直接背 AND 结论。

12.7 练习：先作答，再查附录

12.7.1 m10-q1 •补全•3 分

二输入 AND 例仅使用测试 01 和 11，原始六故障口径下覆盖率为百分之几？保留两位小数。

12.7.2 m10-q2 ·补全·3 分

若 T_A={t1,t3}、T_B={t1,t2,t3}，为保证两个故障都被检出，应保留哪一列要求？说明理由。

12.7.3 m10-q3 ·独立·4 分

重做 y=NAND(a,b) 的六故障覆盖表，给出一个最小测试集并证明至少需要这么多条。

12.7.4 m10-q4 ·独立·4 分

1000 个原始故障，900 个检出，50 个证明不可测，50 个超时。计算原始覆盖率以及排除已证明不可测后的覆盖率；为何不能再排除超时项?

12.7.5 m10-q5 ·迁移·4 分

两个故障总被同一组测试检出，是否就说明能定位到其中一个？如果不能，如何增加诊断能力?

12.7.6 m10-q6 ·迁移·4 分

测试 A 覆盖 {1,2,3,4}，B 覆盖 {1,2,5}，C 覆盖 {3,4,6}。贪心先选 A 后需几条？最优需几条？这个反例说明什么?

12.8 本章来源

- L25：III.1，印刷 pp.69–78；PDF pp.73–82
- T25：Tutorial 7，印刷 pp.72–75；PDF pp.74–77
- F071：Testing slides：fault models 与制造测试引言，PDF pp.1–15

13 11 ATPG：激活故障，再把差异送到输出

布尔差分、D 算法、重汇合与回溯

关键问题：看到一个内部 stuck-at 故障，怎样系统地构造会在输出暴露它的输入?

学习目标

- 从 cofactor 推导布尔差分与 stuck-at 检测条件
- 完整完成一个 D 算法的激活、传播、证明与回溯过程
- 识别重汇合的自掩蔽，避免错误套用单路径链式法则
- 解释 PODEM 的主输入决策与 controllability 近似的边界

先修：m02、m10；**横向联系**：m06、m12、m13。

13.1 布尔差分不是连续微积分

布尔差分用两个 cofactor 的异或定义：

$$\frac{\partial y}{\partial z} = y\big|_{z=0} \oplus y\big|_{z=1}.$$

它回答“其它条件不变时，翻转 z 会不会翻转 y”。结果为 1 表示在该输入条件下 z 的变化可观察，为 0 表示被屏蔽。这里没有无穷小，也没有正负导数，不能机械套连续微积分乘法法则。

对于内部信号 $z = z(x)$，先把 z 作为一个独立切断变量计算输出对 z 的差分，再把真实的 z 函数和其它信号约束代回。检测 $z/0$ 要求正常 z 为 1 且差分为 1；检测 $z/1$ 要求正常 z 为 0 且差分为 1：

$$T_{z/0} : z(x)\frac{\partial y}{\partial z} = 1, \qquad T_{z/1} : \overline{z(x)}\frac{\partial y}{\partial z} = 1.$$

第一项是激活，第二项是可观察性，缺一不可。多输出时将各输出的差分以 OR 合并。若故障点是 stem，切断变量必须替代其所有分支；若仅是 branch，就只替代该连接，不能用同一式子混算。

AND-XOR 表示便于差分：用 $u + v = u \oplus v \oplus uv$、$\bar{u} = 1 \oplus u$ 逐步替换，重复出现两次的同一项异或抵消。例如 $ab + ac = ab \oplus ac \oplus abc$，对 a 求差分得到 $b \oplus c \oplus bc = b + c$。不要把 OR 直接换成 XOR；当两项同时为 1 时，补上的乘积项正是修正两者差异所需。

13.2 完整示范：一个内部故障的两种解法

构造网络$n = ab$、$y = n + c$，目标 $n/0$。布尔法先求 $y_{n=0} = c$、$y_{n=1} = 1$，所以 $\partial y/\partial n = \bar{c}$。检测条件为 $ab\bar{c} = 1$，唯一完整输入为 $(a, b, c) = (1, 1, 0)$。正常 $n = 1, y = 1$；故障 $n = 0, y = 0$，差异确实到达输出。

用 D 记号重复同一过程：$D = (1, 0)$ 表示正常 1、故障 0；$\bar{D} = (0, 1)$ 表示正常 0、故障 1；0 和 1 分别表示两电路都为 0 或都为 1，X 表示未确定。先在 n 放 D（激活）；OR 的旁路输入必须取非控制值 $c = 0$ 才能传播，于是 $y = D$；最后把正常 $n = 1$ 向输入证明，AND 两输入需 $a = b = 1$。这个“激活 → 传播 → 证明”循环是手做 ATPG 的核心。

若目标改成 $n/1$，激活要求 $n = 0$，因此 $a = 0$ 或 $b = 0$，同时 $c = 0$。测试集合为 000、010、100。别把“a=0 或 b=0”错写成二者都 0；只写 000 虽然是一个有效测试，却没有列出全部测试。若题目只要求一个向量，可以写 $0X0$，并说明 X 为可任选输入位；这里的测试立方体 X 与仿真未知态 X 用途不同。

13.3 D 算法的每一张表都要有理由

五值运算可以理解为同时计算正常和故障两个布尔电路。例如 $D \wedge 1 = D$、$D \wedge 0 = 0$、$D \vee 0 = D$、$D \vee 1 = 1$、$\neg D = \bar{D}$。传播 AND/NAND 时把其它输入设 1，传播 OR/NOR 时设 0。X 不是可不加约束地穿过任何门：$D \wedge X$ 尚不能确定是否被阻断。

D-frontier 是含 D 或 $\bar{D}$ 输入、输出还未确定的门集合，它标出可能继续传播的边界；挑一个门并设置旁输入为非控制值。**J-frontier** 用于跟踪已要求输出值、但输入赋值尚未证明的门。对 AND 输出 1，两个输入都被强制为 1；对 AND 输出 0，有多个可选输入置 0，产生搜索分支。

每次新赋值后立即做前向/后向蕴含：能确定的值先传播，冲突就回溯，不要等整张图填满才检查。成功条件是至少一个可观察主输出出现 D 或 $\bar{D}$，且所有内部要求可由主输入一致实现。某条尝试路径失败只证明该选择失败；只有所有必要分支被完整排除，才能证明目标故障不可测。搜索预算耗尽应标成未解决，而不是不可测。

纸面作答建议每行记录“选择/强制、赋值、理由、尚存 D-frontier、冲突”。最终再单独跑一次正常与故障真值计算，避免记号在传播途中反相了一次却漏改 D 极性。

13.4 重点补充：global implication 与学习准则

令 $d = ab$、$e = ac$、$f = d + e$。从 $a = 0$ 出发，局部前向推导得到 $d = e = 0$，进而 $f = 0$。按逆否律，可靠结论是 $f = 1 \Rightarrow a = 1$，不是 $f = 0 \Rightarrow a = 0$；后者可被 $a = 1, b = c = 0$ 反驳。

为什么值得学习 $f = 1 \Rightarrow a = 1$？只从 OR 输出 1 反推，局部门规则不能决定 d 或 e 究竟谁为 1，往往会产生可选分支；全局关系提前强制 a=1，减少无意义搜索。课程强调优先学习那些不能仅靠一串局部蕴含恢复的关系，并要求所考虑的局部网络没有故障效应，避免把仅正常电路成立的关系强套到 D 值网络。

讲义 p98 的双分支重汇合示意中，前向赋值若满足局部可观察性 $y_z\,y_w = 1$，则其逆否方向是值得记录的候选。上述 a=0 例使 d=e=0，OR 对 d、e 的局部差分均为 1，符合该学习准则。做题时必须把正向赋值、局部差分、逆否结论和“不能由局部直接推出”的原因写全；准则不是把任何相关信号都任意连成蕴含。近年卷也单独考过这一项。[F088 Problem 4(c)]

13.5 重汇合：一条能走通的路并不等于整个故障能被看见

令 $u = ab$、$v = ac$、$y = u \oplus v$，a 是扇出干线。对于 $a/0$，若 $a = b = c = 1$，正常 $u = v = 1, y = 0$；故障 $u = v = 0, y = 0$。两条路径都有差异，但在 XOR 重汇合时互相抵消，形成 self-masking。只看任意一条路径会误判“已检测”。

直接化简 $y = a(b \oplus c)$，得到 $\partial y/\partial a = b \oplus c$，检测 $a/0$ 的条件为 $a(b \oplus c) = 1$。因此 $a = 1$ 且 $b \neq c$，例如 110 或 101，才有效。若目标只是 a 到 u 的单个分支故障，则 111 反而可以检测，因为另一支 v 仍为 1。故障位置不同，答案就不同。

对于无扇出、无相关旁路的单条路径，可用局部差分连乘，例如 $\partial y/\partial a = (\partial y/\partial n)(\partial n/\partial a)$；有多路径依赖时应保留所有变化关系，最稳妥的方法是直接比较两个 cofactor。布尔差分的 AND 规则含交叉项：

$$\partial_a(uv) = u\,\partial_a v \oplus v\,\partial_a u \oplus (\partial_a u)(\partial_a v).$$

式中的 u、v 取同一基准输入的值；丢掉最后一项会在两路都受 a 影响时出错。考场若不确定复杂规则，回到“翻转前输出 XOR 翻转后输出”总能重新推导。

13.6 PODEM 与可控性：把搜索空间和估算边界说清楚

PODEM 把显式决策限制在主输入：先选内部目标（例如让故障点取相反值，或让 D-frontier 门的旁输入非控制），再沿网表回溯到一个主输入赋值，执行蕴含；若不能继续传播则反转该输入尝试，仍失败就撤回。内部目标本身不是自由变量，必须由可实现的主输入驱动。这与直接在许多内部线作选择的 D 算法思路不同。该段为拓展概念；补考优先熟练课程 D 算法，原始 PODEM 全文本次访问受限，不引用其性能倍数。[GOEL80]

讲义用 $C_0, C_1 \in [0,1]$ 估计信号取 0/1 的容易程度，并假设主输入独立均匀。若 AND 输入独立，$C_1(out) = C_1(a)C_1(b)$；OR 有 $C_0(out) = C_0(a)C_0(b)$；NOT 交换 0/1 概率。两输入各为 0.5 时，AND 输出 1 为 0.25，OR 输出 1 为 0.75。这些是解析估算，不是实际芯片测量。

重汇合带来相关性：令 $b = a$，则 $a \wedge b = a$，真实 $P(1) = 0.5$，错误独立乘法却给 0.25。故课程概率递推对合适独立树结构精确，对相关网络仅作启发式，不能用来证明某输入不可达。它也不同于 SCOAP 的整数可控性成本；不要把两个体系的数值混在一起。

13.7 案例：代数 ATPG 与结构 ATPG 怎样交叉检查

问题是给一个小组合网络的内部 stuck-at 故障找测试。基线穷举全部 2^n 输入，结果直接且容易核对，但输入数增加后成本指数增长。布尔差分把问题变为布尔条件求解，适合表达式不大时手算；D 算法利用网表结构、传播路径和回溯，适合结构信息清楚的网络。

证据用同一故障列表和同一观察端口：对每个找到的测试，必须验证 $y_{good} \neq y_{faulty}$；在小输入范围穷举，检查“算法说不可测”的目标是否真的无检测输入。成本比较应记录搜索节点、失败回溯和运行时间，不能只比较输出向量数。一个算法生成较短测试集与单故障搜索较快是两个不同指标。

反例是有逻辑冗余的网表：某内部故障可能不可测，并不是算法故障。反过来，仿真只检查了十个向量而未命中，也不能证明不可测。实验中的随机尝试、穷举、D 算法或 BDD 求解要明确标名，不能把一个小网表的穷举工具称为工业 ATPG 实现。能力证据是你能为每次输入赋值说明它服务于激活、传播还是内部证明，并处理一次失败回溯。

13.8 练习：先作答，再查附录

13.8.1 m11-q1 ·补全·4 分

y=n+c、n=ab，目标 n/0。补全：激活条件 _，**传播条件** _，完整输入 ____。

13.8.2 m11-q2 ·补全·4 分

求 y=ab+ac 对 a 的布尔差分，并给出 a/1 的检测条件。

13.8.3 m11-q3 ·独立·4 分

y=(ab) XOR (ac)，检测 a 干线的 stuck-at-0，一共有多少个 3 位完整输入向量?

13.8.4 m11-q4 ·独立·4 分

用 D 算法分析 n=ab、y=n+c，目标 n/1。列出一个含 X 的测试立方体，说明 D 或 D-bar 的值如何传播。

13.8.5 m11-q5 ·迁移·4 分

一个算法因 D-frontier 为空而结束本分支。能否立刻宣布故障不可测？给出完整结论所需条件。

13.8.6 m11-q6 ·迁移·4 分

a 为均匀随机位，b=NOT a。按独立假设估计 P(a AND b=1) 是多少？真实值是多少？说明为什么该估算不能用来证明可控。

13.8.7 m11-q7 ·独立·4 分

网络 d=ab、e=ac、f=d+e。由 a=0 推出 f=0 之后，应学习哪个逆向蕴含？为什么不能写 f=0 推出 a=0？按课程准则解释学习价值。

13.9 本章来源

- L25：III.2，印刷 pp.79–89；PDF pp.83–93
- L25：III.4，印刷 pp.93–99；PDF pp.97–103
- T25：Tutorial 8，印刷 pp.76–81；PDF pp.78–83
- T25：Tutorial 10，印刷 pp.87–94；PDF pp.89–96
- F088：21_exam.pdf，Problem 4(c)，PDF p.12：global implication 学习准则与可控性
- ROTH66：D 算法原始论文；出版信息与可索引摘要，全文抓取受限
- GOEL80：PODEM 原始会议版本 FTCS-10，1980，pp.145–151；仅索引摘要，全文 403

14 12 故障仿真：给定测试到底检出了谁

可观察性、扇出干线与四类仿真方案

关键问题：ATPG 给出一个向量后，怎样知道它是否顺便覆盖其它故障，又怎样避免重汇合误判?

学习目标

- 给定输入向量，先计算无故障值再找全部被检出故障
- 区分扇出干线与分支的传播和掩蔽
- 用局部可观察性构造无扇出区域的故障树
- 解释 serial、parallel、deductive、concurrent 的成本与模型限制

先修：m08、m10、m11；**横向联系**：m13。

14.1 把问题方向与 ATPG 反过来

ATPG 的输入是故障，输出是可检测它的向量；故障仿真的输入是向量集，输出是它们检测的故障。两者配合：为一个难故障生成向量后，用故障仿真找到这条向量顺便覆盖的故障，从待生成列表中删去已检项，称 fault dropping。若目标是诊断、多次检测或统计各向量覆盖关系，不能不加记录就提前丢掉这些故障。

先计算无故障电路所有节点值，这是“激活哪些故障”的基础。某点正常值为 v 时，仅该点 stuck-at-$\bar{v}$ 被激活；与正常相同的 stuck-at-v 不会在此向量下产生局部差异。激活仍不等于检出：还要证明该差异能传播到某个观察输出。

课程的高效流程是：无故障前向仿真；对扇出干线显式检查故障影响；在合适的无扇出区域组合局部可观察性；最后对所有可观察节点加入与正常值相反的固定故障。抽取讲义文字时反相横线可能丢失，因此一定用“相反故障被激活”这一逻辑检查公式，而不要照抄缺横线的文本。[L25 pp.90–92]

14.2 完整示范：一个向量检出六个故障

用$u = ab$、$v = ac$、$y = u \oplus v$。故障位置仅选 a, b, c, u, v, y 六个节点，不另列 branch，共 12 个单 stuck-at 故障。施加 $(a, b, c) = 110$，无故障值 $u = 1, v = 0, y = 1$。逐点只需尝试相反固定值：

```
故障    故障传播后的 (u,v,y)   检出?
a/0     (0,0,0)              是
b/0     (0,0,0)              是
c/1     (1,1,0)              是
u/0     (0,0,0)              是
v/1     (1,1,0)              是
y/0     (1,0,0)              是
```

另外六个同值固定故障没有被激活。因此本向量检出集合是 $\{a/0, b/0, c/1, u/0, v/1, y/0\}$，原始覆盖为 6/12=50%。这张表提供了后续优化算法的穷举参考。

将输入改成 111 时 $u = v = 1, y = 0$。a/0 会同时把 u、v 改 0，y 仍 0，a 干线故障不可见；b/0 只改 u，c/0 只改 v，均可见。若独立加入 a 到 u 的 branch，那个 branch/0 也可见。这说明“同一逻辑值”“同一根源信号”“同一故障位置”是三个不同概念。

14.3 局部可观察性与故障树

在当前输入赋值下，AND 某输入的局部可观察性等于其它输入全部为 1；OR 对应其它输入全为 0；XOR 单个输入翻转总使输出翻转。若输出已经可观察，可以沿满足局部条件的边反向标记输入。无扇出单路径区域内可把条件连乘，形成连续的故障树。

例：$u = ab$、$y = u + c$，输入 110 时 $y = 1$。从输出 y 往回：OR 的 c=0，所以 u 可观察；AND 的 a=b=1，所以 a 和 b 都可观察。c 的旁输入 u=1，对 OR 是控制值，所以 c 不可观察。于是可观察节点为 $\{a, b, u, y\}$；它们的正常值均 1，所以检出 $a/0, b/0, u/0, y/0$。这与直接故障注入一致。

遇到扇出干线不能简单把“各分支可观察”做 OR。上节 111 例中，a 两分支分别翻转都能改变 y，但同时翻转互相抵消；需要模拟 stem 故障后合并全部受影响路径。故障树应连续接到观察点，不能在网表中随意标几个看似敏化的局部门就宣称检测成立。

做课程 Tutorial 9 时，可用三种颜色分别标正常节点值、局部敏化边和 stem 故障引起的变化；最终故障集合只从已证实可观察节点导出。颜色是辅助，交卷仍需保留明确的 0/1 及故障标号，避免只画箭头而没有可复核证据。

14.4 四种方案：共同目标，不同共享方式

串行 serial：每个故障单独改网表并模拟，与正常响应比较。实现最直观，可作为小网表正确性参考；若 F 个故障、T 个向量、每次求值成本 G，粗略工作量为 $O(FTG)$，实际还受事件活动影响。

并行 parallel：把同一节点在多个故障电路中的逻辑值塞进机器字位中，用位运算一次计算多个电路。最简单 w 位实现留 1 位给正常电路，最多 $w-1$ 位给故障；64 位可放 63 个故障。多值逻辑、不同延迟事件及故障分批会增加额外开销，所以“63 倍加速”只是理想计算上限类比，不是实测结论。

演绎 deductive：只做一次正常逻辑仿真，同时在每个节点传播“哪些故障会让该节点异于正常”的集合；门操作变为集合并、交、差或对称差。收益来自共享正常求值，成本来自故障集合大小与集合运算。其简单规则通常针对两值、单故障、零延迟模型。

并发 concurrent：正常事件共享，某故障与正常电路状态发生分歧时才维护局部故障活动，恢复一致的部分可以合并。它利用大量故障电路与正常电路相同的区域，却付出动态事件和状态管理成本。上述后三种属于扩展视野；补考核心仍是 L25/T25 的可观察性与扇出处理方法，不能因扩展名词而忽略手算。

14.5 选学示范：集合传播如何保留重汇合相关性

设 L_a 表示能使当前输入节点 a 与正常值不同的故障集合。对于二值单故障 AND：若正常输入全为 1，任一输入翻转都令输出变 0，因此输出传播集合为输入集合的并；若存在正常 0 输入，必须把所有 0 输入都翻为 1，并且不能把任何正常 1 输入翻为 0，因此用“0 输入列表的交集，减去 1 输入列表的并集”。再加入门输出本身与正常值相反的 stuck-at 故障。

对于 XOR，导致奇数个输入变化的故障才传播；二输入时是对称差 $L_a \triangle L_b$，即只属于一个集合的故障。若同一 stem 故障同时出现在两输入列表，它被抵消。这正是上节重汇合自掩蔽的集合版本。普通并集会错误地保留这个故障。

例如正常 XOR 输入为 1、1，$L_u=\{f,g\}$、$L_v=\{f,h\}$，则输出传播集合为 $\{g,h\}$；f 同时翻转两路，不改变输出。正常输出为 0，加入本地输出 stuck-at-1 后得到 $\{g,h,y/1\}$。每个集合元素代表**同一个故障身份**，不能因它沿两条路径出现就当成两个独立随机事件。

这里是为理解数据共享而构造的小例，不是完整工业故障仿真器。时序电路、X/Z、延迟故障、桥接、多故障组合都需要扩展模型与算法；未经实现和验证不能声称这些情形也被支持。

14.6 怎样比较工具，又怎样验证自己的手算

问题是加速覆盖统计，基线采用串行故障注入。选择另一算法时，先统一网表、故障位置、折叠口径、向量序列、初态、逻辑域和时序模型，再比较总时间、内存峰值与完全相同的检出集合。少计算了一些故障或提前丢弃数据后的“更快”，不构成同任务加速证据。

最有价值的微型验证集包括：无故障、输出固定故障、未激活故障、AND 控制值屏蔽、XOR 两路抵消、stem 与 branch 分离，以及不可测冗余故障。每个都能用手算或穷举给出答案，再让优化算法对齐。单看总体覆盖率相同不够：两个工具可能各错一列却碰巧总数相等，必须比较故障 ID 集合。

学习流程先预测某节点是否可观察，再改变一个输入，观察事件与故障集合如何变化，解释新检出或被掩蔽的原因，最后解决改变扇出位置的变式。每次复习从一张空网表重新算；只有看过解答的勾选不计为独立证据。若写程序，故障列表和网表必须分离，让同一组输入可用于正常与逐故障电路，减少误注入的风险。

14.7 练习：先作答，再查附录

14.7.1 m12-q1 ·补全·3 分

当前节点 x 的正常值为 0，x/0 与 x/1 中哪一个被激活？被激活是否必然被检出？

14.7.2 m12-q2 ·补全·3 分

w 位并行故障仿真保留 1 位模拟正常电路。w=32 时一次最多装多少个故障实例？

14.7.3 m12-q3 ·独立·4 分

u=ab，y=u+c。输入 (a,b,c)=110，故障位置 a,b,c,u,y。列出全部被检出单 stuck-at 故障。

14.7.4 m12-q4 ·独立·3 分

XOR 两输入的故障列表为 {f1,f2,f3} 与 {f2,f3,f4}，不计本地输出故障，输出列表是什么？

14.7.5 m12-q5 ·迁移·4 分

两种算法报告覆盖率都是 90%，是否足够证明它们等价？设计一个更强检查。

14.7.6 m12-q6 ·迁移·4 分

用 a 分两路接 XOR 两输入，构造一个“每条分支可观察，但 stem 不可观察”的反例；再说明增加一个 branch 故障位置后的区别。

14.8 本章来源

- L25：III.2，印刷 pp.85–89；PDF pp.89–93
- L25：III.3，印刷 pp.90–92；PDF pp.94–96
- T25：Tutorial 9，印刷 pp.82–86；PDF pp.84–88
- AGRAWAL-FAULTSIM：作者课程 Lecture 7：Fault Simulation；四类方法为选学扩展

15 13 时序测试、Scan 与 BIST

把状态变得可控可见，再理解压缩的代价

关键问题：内部状态既不能直接设置也不能直接读出时，怎样使测试仍然可管理？

学习目标

- 通过时间展开解释为什么时序故障需要输入序列
- 画出 scan 移位、capture、卸载的状态与资源时间线
- 计算明确装载重叠假设下的 scan 测试周期数
- 按明确定义递推手算 LFSR 与 GF(2) 签名，并解释 aliasing 限制

先修：m07、m09、m11、m12；**横向联系**：m01、m10。

15.1 先看一段状态序列，而不是一个输入向量

组合电路的输出只由当前输入决定；时序电路还依赖状态。写成 $q_{t+1} = F(q_t, x_t)$、$y_t = G(q_t, x_t)$ 后，测试问题自然变成：先把电路带到能激活故障的状态，再让故障效应经过若干拍传播到输出。时间展开把每拍组合逻辑复制一层，前一层寄存器 D 连接后一层 Q。

注意同一个永久 stuck-at 故障在每个时间帧都存在，不能只在某一层注入后就让它自动恢复。正常与故障电路的状态也可能逐渐分歧，要分别追踪。把故障效应传播到寄存器 D 并不一定已经检测，通常还需 capture 并继续观察 Q 或输出。

小例：$q_{t+1} = x_t$、$y_t = q_t$，目标为寄存器输入连接 D/0，初态 $q_0 = 0$。在 $t = 0$ 施加 $x_0 = 1$，此时 y_0 仍 0；下一有效沿后正常 $q_1 = 1$，故障 $q_1 = 0$，才在 y_1 出现差异。若读输出过早，会误判没有检测；若不知道初态，必须考虑初始化序列或扫描装载。

15.2 Scan 怎样让寄存器成为可访问的边界

全扫描将寄存器改成可在正常 D 输入与串行 scan 输入之间选择的单元。scan_enable=1 时，寄存器串成移位链，从 scan_in 逐位设置内部状态并把旧状态从 scan_out 移出；scan_enable=0 时执行正常逻辑，在功能时钟边沿 capture 响应。对组合 ATPG 而言，寄存器输出可看作伪主输入、寄存器输入可看作伪主输出。

```
装载状态（shift） -> 施加主输入 -> 功能捕获（capture）
          -> 卸载响应，同时装载下一个状态（shift）
```

“扫描把时序电路变成组合问题”是测试抽象：真实电路仍有寄存器、移位时钟、scan 模式和时序约束。扫描链本身也可能有缺陷，capture 可能需要一拍或多拍，at-speed 测试还要明确 launch/capture 协议，不能从普通单拍模型直接外推。

全扫描提升可控性与可观察性，代价是 mux、布线、面积、时序负载和大量 shift 周期；部分扫描只连接部分寄存器，成本较低，但剩余状态仍需时序 ATPG。多链缩短最长链，通常需要更多并行 I/O 或压缩结构。高 shift 切换活动可能带来功耗问题，不能只用“链越多越好”做设计结论。

15.3 手算测试时间：先声明有没有重叠

设单链长度 N，有 P 组测试，每组 capture 占 c 个周期；shift 和 capture 用同一频率 f。采用常见重叠方式：初次装载 N 周期，之后每次 capture 后卸载旧响应时同时装载下一向量，最后还要卸载最后响应。总周期为

$$C = N(P + 1) + Pc, \qquad t = C/f.$$

例：N=1000、P=10、c=1、f=10 MHz。初装 1000，10 次 capture 共 10，中间 9 次重叠 shift 共 9000，最终卸载 1000，合计 11010 周期，时间 1.101 ms。若每组均单独装载再单独卸载且不重叠，则是 $P(2N + c) = 20010$ 周期，不能混用两个公式。

若改成 4 条完全平衡的 250 位链，且 4 位输入输出能并行传送、频率和 capture 条件相同，用最长链长度 250 替换 N，得到 2760 周期、0.276 ms。这个数字是教学解析估算；并未包含模式切换、链不平衡、压缩初始化、ATE 协议和功耗降频。比较资源预算时必须把多出的 I/O 或解压缩硬件列出来。

边界检查：P=0 时没有测试，不应硬套公式仍要求装载；N=0 退化为无扫描状态，只剩 capture；P=1 时必须保留首次装载和最终卸载，得到 2N+c。实际工具可能把最后响应采样定义不同，所以作答开头写清时间线。

15.4 LFSR：先固定比特顺序，再谈多项式

LFSR 在 GF(2) 上工作：加减都是 XOR，没有进位。为避免抽头图的左右约定混乱，本例直接定义 3 位状态 $q_2q_1q_0$ 表示 $s(X) = q_2X^2 + q_1X + q_0$，每步执行 $s' = Xs \bmod p(X)$，取 $p(X) = X^3 + X + 1$。因此状态递推为 $(q_2', q_1', q_0') = (q_1, q_0 \oplus q_2, q_2)$。

从 001 开始逐拍为：

```
001 -> 010 -> 100 -> 011 -> 110 -> 111 -> 101 -> 001
```

共访问 7 个非零状态，周期 $2^3-1=7$；000 会一直保持 000。最大周期需要对应**本原多项式 primitive polynomial**、合适实现和非零种子。仅“不可约 irreducible”并不充分：$X^4+X^3+X^2+X+1$ 在 GF(2) 不可约，却使从 0001 开始的余式序列为 0001、0010、0100、1000、1111，然后回到 0001，周期只有 5 而非 15。这是对讲义 p105 简略表述的重要补充。

Fibonacci 与 Galois、XOR 与 XNOR、左移与右移可能使用不同抽头写法和锁死态。OpenTitan 的参数化实现明确区分 XOR 全零与 XNOR 全一的寄生态，并设计重置种子保护；这是具体工程实现，不能把其中的保护自动算进课堂普通 LFSR。该工程资料的验证范围也与自己只遍历 3 位实例不同。[OPENTITAN-LFSR]

15.5 签名分析与 aliasing：压缩不会凭空创造信息

把响应位流表示成多项式 $a(X)$，用生成多项式 $p(X)$ 相除，余式作为 n 位签名。约定比特串最高位先读，初始余式 0，不额外补零；例 $a=1101$ 对应 X^3+X^2+1，$p=1011$ 对应 X^3+X+1。GF(2) 除法只需一次 XOR：1101 XOR 1011 = 0110，因此签名为 110，即 X^2+X。

错误响应 0110 也得到签名 110。两响应之差为 1011，恰是生成多项式的倍数，所以被压缩掩蔽。这叫 aliasing：响应已经不同，最终签名却相同。它不同于故障根本没激活、没传播到响应，后两种属于测试图样覆盖不足。

若长度 L 的所有非零错误序列等可能，且压缩映射满秩到 n 位签名，漏检比例为

$$P_{alias}=\frac{2^{L-n}-1}{2^L-1}\approx 2^{-n}\quad (L\gg n).$$

此式包含分布和映射假设，不是任意确定故障的保证。上面指定错误 1011 的掩蔽概率就是 1；对均匀抽取的 4 位非零错误且 n=3，比例为 1/15，不能直接精确写成 1/8。MISR 将多路响应并行注入签名寄存器；具体递推还依赖输入映射、初值、时钟和多项式，必须按电路定义手算。

15.6 BIST 的取舍：图样生成、响应压缩与控制缺一不可

BIST 把图样发生器、待测电路、响应压缩和测试控制放在片上。基线为外部设备送入确定性向量、逐响应比较；BIST 可减少外部数据传输并支持系统内自检，但增加硬件、切换功耗和设计验证成本。一个 LFSR 本身不是完整 BIST：还需正确的模式切换、时钟/复位、测试时长、黄金签名和 pass/fail 判定。

伪随机向量对某些随机模式难测故障很低效。若单次独立随机输入检出某故障的概率为 p，K 次独立试验未检出的概率是 $(1-p)^K$；LFSR 序列确定且相关，这个公式只能作为明确独立假设下的估算，不能直接证明真实 LFSR 覆盖。可用加权随机、确定性 top-up 向量、测试点插入等办法改善覆盖，各有额外成本。

反例是所有测试图样都未激活某个故障，此时增加 MISR 位数也无济于事；相反，图样已导致错误响应，却恰被签名压缩掩蔽，则改图样、改变签名设计或扩大签名才相关。验证应分别报告故障覆盖、响应压缩 aliasing 模型、总测试周期与硬件代价。最大周期 LFSR 并不等于密码学安全随机数发生器，确定性结构不提供那种安全保证。

本章结束应能画出 shift/capture 状态时间线、列出至少一条 LFSR 完整周期、完成一次 GF(2) 长除法，并用一句话区分“未激活”和“签名碰撞”。这些是独立能力证据，背出 Scan、BIST 缩写不算完成。

15.7 练习：先作答，再查附录

15.7.1 m13-q1 •补全•3 分

单链 N=100，P=5，capture 每组 1 周期，允许卸载与下一组装载重叠。包括首装与末卸，总周期是多少?

15.7.2 m13-q2 ·补全·3 分

按本章 3 位 LFSR 递推，当前状态 100，下一状态是什么？请填写三位字符串并解释 XOR。

15.7.3 m13-q3 ·独立·4 分

按最高位先读、初始余式 0、不补零的约定，1101 除以 1011 的 3 位签名是什么？再找一个不同的 4 位响应具有同签名。

15.7.4 m13-q4 ·独立·4 分

为什么“不可约多项式 + 非零种子一定获得 2^n−1 周期”不正确？用本章 4 位例子说明。

15.7.5 m13-q5 ·迁移·4 分

8 位签名寄存器是否保证每一种故障只有 1/256 的漏检概率？若不是，给出反例与正确适用条件。

15.7.6 m13-q6 ·迁移·4 分

比较一条 1000 位链与四条 250 位链测试 10 组向量：capture 各 1 周期、10 MHz、均允许重叠。说明时间收益及至少三项未计成本。

15.8 本章来源

- L25：III.5，印刷 pp.100–107；PDF pp.104–111
- T25：Tutorial 11，印刷 pp.95–97；PDF pp.97–99
- OPENTITAN-LFSR：prim_lfsr 文档：参数、锁死态保护和验证范围；master 于 2026-11-08 所见
- XAPP052：Peter Alfke, XAPP052 v1.1, 1996-07-07；最大周期抽头与编号约定

16 交互实验的模型、手算与边界

16.1 把真值表压成最小 SOP

先预测：先写下你预计的 PI 数、EPI 数与最优 literal 数。再运行核对。

模型：最多 4 变量、单输出、两级 SOP。先穷举 QM 合并，再按总 literal 最少、项数最少的字典序选择覆盖；保留最多 32 个并列解。

参数与假设：a 为最高位。On-set 与 don’t-care 不相交。DC 可用于合并，却不是必须覆盖的列。门延迟、反相器共享、负载和工艺映射不计成本。

手算校验：n=3，On={1,3,5,7}，DC={}：001、011、101、111 合为–1，即 c，1 个 PI、1 个 literal。

边界校验：空 On-set → 0；全部为 1 → 1（0 个 literal）；On 与 DC 重叠必须拒绝。

迁移问题：用 On={0,1,2,5,6,7}，解释为什么没有 EPI 仍然存在最优覆盖。再加入 DC={3,4}，说明为什么可以化成 1。

限制：不是完整 Espresso 或工业综合器；literal 最小不保证实际面积/时序最优。

来源：L25、T25。

16.2 改变顺序，观察 BDD 大小

先预测：比较 a,c,b,d 与 a,b,c,d，先预测哪一种能更早合并相同余因子。

模型：真值表递归 Shannon 展开；删除 low=high 的节点；通过 (变量,low,high) 唯一表共享节点。计数仅包括非终端节点。

参数与假设：四个变量 a,b,c,d；全图统一顺序；不使用补边，不进行动态变量重排；0、1 终端不计数。

手算校验：默认 f=(a↔c)(b↔d)，On={0,5,10,15}。a,c,b,d 顺序有 6 个非终端节点；a,b,c,d 有 9 个。数值由本模型穷举验证。

边界校验：空 On-set 只剩终端 0；全 On-set 只剩终端 1；f=d 只有一个非终端节点。

迁移问题：改为奇校验函数 On={1,2,4,7,8,11,13,14}，尝试两种顺序，解释为什么并非所有函数都对顺序同样敏感。

限制：仅教学规模；构建阶段仍枚举真值表，不能把这个实现的运行时间当工业 BDD 性能。

来源：L25、BRYANT86。

16.3 短脉冲与事件队列

先预测：默认 w=2ns、d=3ns，分别画出 transport 与 inertial 波形，再执行。

模型：单个非反相门，输入在 1ns 上升，在 1+w ns 下降；transport 传递两个事务，简化 inertial 拒绝宽度小于 d 的脉冲。显示事件、delta、挂起事务与队列占用。

参数与假设：初值全部 0，门延迟常数 d。采用 w<d 拒绝、w=d 通过的边界定义；d=0 时更新发生在下一 delta。输入只包含这一段脉冲。

手算校验：transport：t=4ns 输出 1、t=6ns 输出 0。inertial：t=3ns 输入下降时撤销计划 t=4ns 的上升；输出始终 0。

边界校验：w=d 边界通过；d=0 不会增加物理时间但 delta 加 1；w≤0 拒绝。

迁移问题：令 w=3ns，d=3ns，观察阈值两侧。再解释两个零延迟串联赋值为何需要多个 delta 而不是延长物理时间。

限制：不是 IEEE VHDL 完整波形事务/拒绝时间实现，也不模拟模拟电压、亚稳态、完整多驱动解析或真实电路毛刺宽度。队列占用是教学事务数，不是 CPU 性能预测。

来源：L25、F073、GHDL-SIM。

16.4 激活、传播与最小测试集

先预测：在计算前先求 n/0 的测试：激活要求 n=1，传播经过 OR 要求 c=0，因而 abc=110。

模型：固定网络 n=a∧b，y=n∨c；在 a,b,c,n,y 五条声明信号线上枚举单 stuck-at-0/1，共 10 个故障。穷举 8 个向量的无故障与有故障输出。

参数与假设：输入位顺序 abc，单故障，确定二值逻辑，输出仅观察 y。每个输入仅出现一次，因此此网络没有单独列出的扇出分支故障。

手算校验：测试 110 检出 a/0、b/0、n/0、y/0，共 4/10=40%；100 检出 b/1、n/1、c/1、y/1。联合后仍需检查剩余故障。

边界校验：不选向量覆盖率 0%；全部 8 个向量覆盖 10/10。相同检测向量集合才是本模型下等价。

迁移问题：尝试找到 4 个向量覆盖全部 10 个故障，并证明 3 个不够。提示：a/1、b/1、n/0 各有一个不同的唯一检测向量；c/0 还需要 c=1。再解释删去冗余故障后的分母为何不能与原 10 故障分母混用。

限制：不包含桥接、时延、瞬态、多故障，也不将故障覆盖率等同于实际芯片缺陷检出率。

来源：L25、T25、F071。

16.5 LFSR 的状态与周期

先预测：seed=0001 时先手算五拍，再预测零种子和 mask=1 的行为。

模型：4 位右移 Fibonacci LFSR。显示 s3s2s1s0；f= 各 mask 置 1 位置对应状态位的 XOR；next=(state»1)|(f«3)。输出旧 s0。追踪直至状态重复。

参数与假设：位方向、抽头和反馈式均显式定义，不能只凭一个多项式字符串套另一资料的移位图。默认 mask=3，即 f=s0⊕s1。

手算校验：默认 mask=3：0001→1000→0100→0010→1001；周期 15。对应本约定输出序列满足 u(t+4)=u(t+1)⊕u(t)，特征多项式 x^4+x+1。

边界校验：全零 seed 锁定零，周期 1；mask=1 且 seed=1 只轮转，周期 4。不可约并不自动等于本原。

迁移问题：改 mask 并记录周期，找一个短周期反例。说明 MISR 的 2^-r 混叠近似还需要什么错误分布假设。

限制：这是确定性小型递推，不验证某个实际 MISR 的别名概率、不代表随机测试覆盖率。

来源：L25、T25。

16.6 接受坏解的温度

先预测：Δ=2、T=2、r=0.4，该移动是否接受？先算 p。

模型：最小化代价，Δ=Cnew−Cold。Δ≤0 接受；否则 p=exp(−Δ/T)，若均匀随机数 r∈[0,1) 满足 r<p 则接受。T=0 时不接受变差。

参数与假设：代价与温度同一归一化尺度；此实验显示单步接受规则，非完整布局或逻辑综合的退火过程。r 为可手动固定的随机样本以便复现。

手算校验：p=e^-1≈0.367879，小于 0.4，因此拒绝。保持 r 不变，提高 T 至 4，p≈0.606531，因此接受。

边界校验：Δ≤0 必接受；T=0 且 Δ>0 拒绝；r 不能等于 1；负温度不合法。

迁移问题：比较 T=0.5、2、10；再说明高接受率为什么不能证明更优最终解，提出同预算、多 seed 的验证办法。

限制：不声称退火保证在有限考试步数或有限运行时间内得到全局最优。

来源：L25、F006。

17 机制、成本与方案比较

17.1 同一函数：精确 QM 与启发式覆盖

17.1.1 问题与基线

给定同一单输出函数及相同 On/Off/DC 集，求覆盖 On 且不覆盖 Off 的 SOP。基线是全部最小项组成的 CSOP。固定代价为 literal 总数；若相同再比较项数。不能用一种算法的项数和另一种算法的 literal 数直接比较。

17.1.2 机制与证据

精确方法先生成完整 PI，再求全局最小覆盖。启发式通常先取必要项，再按局部覆盖收益选项，或在合法 expand/reduce/remove 之间迭代。本网站 QM 实验以独立全 cube 枚举和精确覆盖 oracle 校验全部 3 变量函数以及带 DC 的全部 6561 种三值规范；这是有限教学规模上的穷举证据，不是算法任意规模的性能测量。

17.1.3 成本与适用条件

精确法给出指定代价下的最优性，但候选与覆盖搜索可能指数增长。启发式可在较大问题上寻求可用解，成本取决于具体规则、起点和停止条件；本站没有测量工业工具运行时间。小变量考试题适合把全过程写完整，大规模综合则必须考虑预算。

17.1.4 反例与验证办法

PI 集合不是最小覆盖。没有 EPI 不代表无解；DC 不要求覆盖。用 On={0,1,2,5,6,7} 的六环覆盖表说明局部第一选择可能造成不同后续搜索。验证至少包含：每个 On 被覆盖、每个 Off 未被覆盖、成本口径一致、最优性有穷举或下界依据。比较运行时间时需同一输入、同一机器、相同停止条件与多次统计，不能凭一次迭代次数宣布更快。

17.2 同一函数：两级 SOP 与因式分解

17.2.1 问题与基线

实现 f=ab+ac+db+dc。固定二值组合功能、只有二输入 AND/OR、每门单位延迟、不计导线/负载/反相器。基线用四个二输入 AND 形成乘积，再用三只 OR 做平衡树。

17.2.2 机制与证据

利用分配律得到 f=(a+d)(b+c)。先两个 OR，再一个 AND。本例可对 16 种输入逐行证明等价；SOP 有 8 次 literal 出现，因式形式 4 次。按上述统一库，基线 7 门、最长 3 门延迟；分解后 3 门、最长 2 门延迟。所有数字都是明确教学模型下的手算，不是标准单元面积或 ns 实测。

17.2.3 成本与适用条件

因式分解复用结构，往往减少 literal 与门数；更一般的网络会增加扇出，改变逻辑深度、线长和重汇合。若门库提供大扇入复杂门，基线的计数与深度会改变，不能沿用 7 与 3。

17.2.4 反例与验证办法

面积更少不自动保证更快。例如提取一个被许多路径共享的项，可能增大该节点负载。先进行等价验证，再在相同门库、负载、输入转换时间和约束下做技术映射与静态时序比较。本课的 literal 代价是抽象代理，不能外推功耗或芯片面积。

17.3 同一函数：ROBDD 两种变量顺序

17.3.1 问题与基线

表示 f=(a↔c)(b↔d)。固定同一个函数、无补边的约简有序 BDD、相同归约规则，统计非终端节点。基线顺序 a,b,c,d。

17.3.2 机制与证据

基线需要先记住 a 和 b 的取值，再和 c、d 核对。交错顺序 a,c,b,d 更早完成第一对相等判断，使余函数共享更快出现。教学模型穷举得到基线 9 个非终端、交错 6 个；两者仍表示同一个 16 行真值表。原始 Bryant 论文讨论固定顺序下规范表示和变量顺序的重要性，本站小例不代表该论文性能实验。

17.3.3 成本与适用条件

好的顺序可以降低表示大小，但寻找好顺序也有成本。固定顺序下的规范性不能被误读为任意变量顺序的图完全相同。规模增大后某些函数仍有指数级最坏情况。

17.3.4 反例与验证办法

奇校验函数对本例几种顺序的节点数可能不变，不能将“交错总更好”当规律。对每个输入沿低/高边走到终端，确认与函数一致；检查同一变量在所有路径的相对顺序一致、low≠high、无重复同构节点。性能评估需固定补边机制、动态重排策略、缓存以及统计终端是否计入。

17.4 同一测试任务：串行与并行故障仿真

17.4.1 问题与基线

给定固定网表、固定向量集和固定 10 个单 stuck-at 故障，返回每个向量检测哪些故障。基线分别运行无故障电路和每个故障电路。这里比较的是完成同一检测矩阵的计算组织，不能随意改变故障分母。

17.4.2 机制与证据

串行组织对每个故障注入、传播、比较。位并行把多个故障机的同名信号打包成机器字；若一个 32 位字中保留 1 位正常机，则最多 31 位表示故障机。10 故障可放入一批。这是位宽容量计算，不是实测 31 倍加速。真正时间还受字宽、数据搬运、故障注入、检测后丢弃策略以及波形复杂度影响。

17.4.3 成本与适用条件

并行法适合相同逻辑操作同时作用于许多故障机；串行法结构直观，便于逐故障调试。复杂多值逻辑可能需要额外位平面；带时间的故障仿真还需处理不同故障机不同事件，不能直接套零延迟位并行性能。

17.4.4 反例与验证办法

若只看最终覆盖率而不逐项比检测矩阵，两个错误实现可能都得到 100%。应以简单电路的独立穷举结果作 oracle，逐向量、逐故障核对，再比较相同输入和统计范围内的墙钟时间与内存。重汇合处不能简单合并故障集合，必须计算差异会否互相抵消。

17.5 同一状态机：二进制编码与 one-hot

17.5.1 问题与基线

实现同一个 4 状态、相同输入输出及复位语义的 FSM。基线用 2 个状态触发器二进制编码。这里先固定状态转移，不改变可达行为，再比较编码。

17.5.2 机制与证据

二进制用 2 位表示 4 个状态；one-hot 用 4 位，每个合法状态恰有一位为 1。触发器数量的 2 与 4 是编码定义下的精确计数。one-hot 可能简化部分状态判断与转移逻辑，但逻辑级数、面积和最大频率由具体转移图及门库决定，本课程不虚构百分比收益。

17.5.3 成本与适用条件

二进制节省状态位；one-hot 增加寄存器和时钟负载，可能使某些转移译码更直接。若状态位便宜、时序紧张且状态图稀疏，可考虑 one-hot。若寄存器与时钟功耗重要，必须评估二进制。

17.5.4 反例与验证办法

最少状态数不必对应最少门或最快实现；编码也会引入不同数量的非法状态组合。验证须同时检查：复位落在合法状态、所有可达输入序列输出等价、非法状态恢复政策一致、综合约束与目标技术相同。只比较寄存器数不足以声称总面积最优。

18 历年卷地图与模考

18.1 先问自己“哪一步不会”，再安排学习

挂科后的复习很容易变成不断重看讲义。一个更可检查的起点是拿白纸做短诊断：给定一个三变量函数，写 on-set、cofactor 和一个化简；给定覆盖表，选最小覆盖；给定 FSM 表做一轮划分；给定两门电路做一条事件；给定 stuck-at 故障写激活条件。每题只花 5 分钟，不会就写出卡在哪一步。

把卡点分成四层。第一层是符号：反相、变量权重、cube 的星号、信号值与故障编号。第二层是操作：合并、cofactor、划分、排队、D 传播。第三层是停止条件：哪些项已成为 PI、划分是否稳定、队列是否空、搜索是否穷尽。第四层是边界：don’t-care 能否覆盖、启发式是否最优、指定路径无解是否等于整个故障不可测。补最早出错的一层，后面的计算才有意义。

诊断是调度工具，不是能力判决。一次做不出表明需要一条示范路径；一次看懂答案也不等于能够独立完成。

18.2 每个题型都走“示范—补全—独立—迁移”

示范阶段逐行追问为什么。例如 Quine 合并时，不只读“00-1”，而要说出哪两项只有一个固定位不同，以及为何其余星号位置相同。补全阶段遮住关键一列，自己填 PI 列表或新事件；保留题干和前一步，不马上重做整题。

独立阶段换一道未见过的同结构题，不打开答案、不复制示范中的数字。作答后按量表检查：前提、方法、过程、结果、验证。开放题采用自评，站点不把关键字匹配冒充可靠自动评分。

迁移阶段只改一个条件：变量顺序改了，函数应不变但 BDD 大小可能变；故障极性反转了，激活条件会变；门延迟变了，事件次序可能变；相邻两个 cube 分别可删，并不意味着一起可删。能够说出“不变的是什么、变化的是什么”，才说明学到了机制。

18.3 60 分钟训练卷如何分配时间

五份历史样本均标 60 分钟、60 分，可以先用“每分约 1 分钟”作为训练标尺。正式考试若时间或分值不同，要按通知调整。建议前 2 分钟扫卷、按题号标记熟悉程度，最后保留 5 分钟检查；剩余时间按分值分配。不是每个问题都值得同样多的时间。

先写能够确定的入口：QM 的 minterm 二进制、BDD 的两条 cofactor、FSM 的初始输出分组、仿真的稳定初值、ATPG 的激活值。入口正确且过程清楚，后续即使没有收敛，也比只写一个猜测的答案容易核查。

某个分支卡住超过预算的一半，保留中间结果、标明待查处并跳到可独立完成的小题。不要在覆盖表上无限试组合，也不要为了一个 D 算法分支吞掉整卷测试题时间。历史试卷声明小题可独立解答，但具体题目仍要看是否共享定义。

最后检查五件事：变量顺序；反相横线；cover 是否漏掉 on-set 或误含 off-set；时刻是否加了门延迟；D 向量是否真的让好坏电路输出不同。参考答案与自己的变量排列不同时，先统一表示再比较。

18.4 错题记录要留下可验证的修复

每条错题只需记录六项：题目来源与页码、自己的原答案、首次错误步骤、错误类型、正确规则、下一次验证时间。不要只记“粗心”，而要写“把 cofactor 当成乘法保留了绑定变量”“删除 cube 时仍把被删项放在 h 内”“事件表按输入顺序处理，未按执行时刻排序”。这种描述才对应可练习的动作。

复盘先找首次错误，不把由同一错误连带产生的十个错误当成十个知识缺口。规则修复后，立刻做一个微小反例，再隔一天做同类新题。若只会背原答案，把状态或变量改名；若改名就不会，说明依赖视觉记忆，需要重新解释机制。

间隔复习默认 1、3、7 天，剩余时间不足时可改为 1、2、4 天。每次复习优先在白纸上重建步骤，再打开记录核对。阅读、复习提醒到期、真正通过独立题是三种不同状态。

18.5 怎么使用这批资料而不被文件数量淹没

主线按 2025-10-31 正式 Lecture Notes 和 Tutorial Notes 走。手册中标 L25/T25 的是课程基准；标 F 编号的文件可在来源索引查到原文件名和阅读范围。2025 课堂手写页帮助理解老师板书步骤，求生笔记用于考前回忆，历年卷用于验收速度和迁移。

先读一个模块相关页，做一道题，再回原资料查未懂的定义。不要连续读完多个版本的同一章来获得“学了很多”的感觉。Script 三份合订本把同类内容重新排版，并不是多出三套独立知识。

做题时必须看 PDF 原页。文本提取可能丢失反相横线、门图和箭头方向。F089 虽然文件名写 exam，正文含标准答案；F055 文件名带 DMA22，封面实际日期为 2023-06-20。先确认版本再用它计时或评分。

18.6 把新知识接到熟悉的计算机知识上

如果你熟悉 CPU/GPU/存储：把 cube 看成对输入地址的一组位掩码，把 QM 看成把相邻且等价的地址区间合并；边界是 cube 不要求连续十进制地址，它对应布尔维度，而非一般数值区间。把 ROBDD 看成共享的条件决策图；边界是规范性需要固定变量顺序及归约规则，不是任意 if 树都有唯一结构。

把事件仿真队列类比成按时间排序的任务队列；边界是模拟时间和墙钟时间不同，同一时刻可能还有 delta 迭代，排队操作不能任意改变信号语义。把 scan 类比成给内部状态开调试读写通道；边界是扫描模式、功能模式和 capture 时序必须分开，不能把 scan 位移当作正常工作状态转移。

如果这些背景并不熟悉，可以直接从真值表、门电路和纸笔事件表开始。类比只用于建立入口，所有计算最后回到布尔语义和题目明确的模型。

18.7 2019_retake 标签卷（卷面日期 2020-06-29）

来源 F086；卷面日期 2020-06-29；60 分钟 / 60 分；可用于限时作答，先隔离答案。

- 1a（5 分）：cube、multiple implicant 与 CompSOP 概念。对应 m02、m03、m05；PDF p.2。
- 1b（4 分）：覆盖表的启发式选择。对应 m03；PDF p.2。
- 1c（5 分）：Quine 算法求全部素蕴含项。对应 m03；PDF p.3。
- 1d（3 分）：NAND-only 与 NOR-only 代数转换。对应 m02；PDF p.3。
- 1e（3 分）：函数分解矩阵及中间编码位数。对应 m05；PDF p.4。
- 1f（3 分）：从 ROBDD 求 SOP、POS、CompSOP。对应 m06、m03；PDF p.4。
- 1g（4 分）：cube 图和 on-set。对应 m02；PDF p.5。
- 2a（4 分）：通过恒真检验删除 literal。对应 m04；PDF p.6。
- 2b（4 分）：FSM 分组最小化。对应 m07；PDF p.6。
- 3a（5 分）：事件驱动仿真。对应 m08；PDF p.7。
- 4a（3 分）：全局蕴含、可测性和 AND-XOR。对应 m10、m11；PDF p.8。
- 4b（5 分）：故障覆盖、检测与区分。对应 m10；PDF p.8。

- 4c（4 分）：AND-XOR 与输入 stuck-at 的布尔差分。对应 m11；PDF p.9。
- 4d（4 分）：内部信号的布尔差分。对应 m11；PDF p.9。
- 4e（4 分）：故障路径及 D 算法。对应 m11；PDF p.10。

18.8 2020 正考（2020-12-03）

来源 F087；卷面日期 2020-12-03；60 分钟 / 60 分；可用于限时作答，先隔离答案。

- 1a（6 分）：布尔函数计数、真值表、CSOP 与表示。对应 m02、m03；PDF p.2。
- 1b（5 分）：覆盖条件、MinSOP 与 EPI。对应 m03；PDF p.3。
- 1c（2 分）：POS 到 CompSOP。对应 m03；PDF p.3。
- 1d（5 分）：给定顺序构造 ROBDD。对应 m06；PDF p.4。
- 1e（6 分）：多输出共享 implicant 的成本。对应 m05；PDF p.5。
- 1f（4 分）：售货机 FSM 读图。对应 m07；PDF p.6。
- 1g（4 分）：分别/同时删除 cube 与扩张。对应 m04；PDF p.7。
- 1h（6 分）：FSM 最小化和最小状态图。对应 m07；PDF p.8。
- 2a（5 分）：事件驱动仿真表。对应 m08；PDF p.9。
- 2b（2 分）：带触发器的波形。对应 m08；PDF p.10。
- 3a（3 分）：AND-XOR、零布尔差分与回溯原因。对应 m11；PDF p.11。
- 3b（4 分）：故障覆盖与删除测试后的漏测。对应 m10；PDF p.11。
- 3c（5 分）：结构故障路径与 D 算法。对应 m11；PDF p.12。
- 3d（3 分）：局部可观测性组合与故障树。对应 m12；PDF p.13。

18.9 2021 正考（2021-12-10）

来源 F088；卷面日期 2021-12-10；60 分钟 / 60 分；可用于限时作答，先隔离答案。

- 1a（6 分）：代数恒等、on-set、cofactor、真值表。对应 m02；PDF p.2。
- 1b（7 分）：CSOP、覆盖条件和启发式比较。对应 m03；PDF p.3。
- 1c（4 分）：Resolution layer algorithm。对应 m03；PDF p.4。
- 1d（4 分）：状态编码与状态/输出函数。对应 m07；PDF p.4。
- 2a（6 分）：配置图、恒真检验与 cube 扩张。对应 m04；PDF p.5。
- 2b（5 分）：多输出共同 implicant 的收益。对应 m05；PDF p.6。
- 2c（4 分）：售票机 Mealy FSM 设计与找零变式。对应 m07；PDF p.7。
- 2d（4 分）：1/2/3-equivalent 状态划分。对应 m07；PDF p.8。
- 3a（3 分）：含触发器的事件仿真。对应 m08；PDF p.9。
- 3b（3 分）：补全 VHDL 电路描述。对应 m09；PDF p.10。
- 4a（2 分）：ATPG 冲突与分区降低复杂度。对应 m11、m13；PDF p.11。
- 4b（3 分）：布尔差分求 x/0 与 x/1 全部向量。对应 m11；PDF p.11。
- 4c（6 分）：学习全局蕴含与 D 算法。对应 m11；PDF p.12。
- 4d（3 分）：组合局部可观测性构造故障集。对应 m12；PDF p.13。

18.10 21_retake 标签答案卷（卷面日期 2022-05-06）

来源 F089；卷面日期 2022-05-06；60 分钟 / 60 分；含答案，不宜用于未见题盲测。

- 1a（7 分）：cube、共同 implicant、Quine 最坏情况。对应 m02、m03、m05；PDF p.2。
- 1b（3 分）：启发式覆盖与真正 MinSOP。对应 m03；PDF p.2。
- 1c（3 分）：含 don’t-care 的最小化。对应 m02、m03；PDF p.3。
- 1d（3 分）：函数分解与 LUT 存储值。对应 m05；PDF p.3。

- 1e（5 分）：删除 cube / literal 的恒真条件。对应 m04；PDF p.4。
- 1f（5 分）：ROBDD 分解条件和 cofactor。对应 m06、m05；PDF p.4。
- 1g（3 分）：cube 图与 MinSOP。对应 m02、m03；PDF p.5。
- 2a（3 分）：FSM 状态表与寄存器数量。对应 m07；PDF p.6。
- 2b（3 分）：FSM 等价划分。对应 m07；PDF p.6。
- 3a（5 分）：事件驱动仿真。对应 m08；PDF p.7。
- 4a（3 分）：全局蕴含、可测性、AND-XOR。对应 m10、m11；PDF p.8。
- 4b（5 分）：检测集与故障区分集。对应 m10；PDF p.8。
- 4c（4 分）：输入故障的布尔差分。对应 m11；PDF p.9。
- 4d（4 分）：内部信号的布尔差分。对应 m11；PDF p.9。
- 4e（4 分）：结构路径与 D 算法。对应 m11；PDF p.10。

18.11 DMA22_retake 标签卷（卷面日期 2023-06-20）

来源 F055；卷面日期 2023-06-20；60 分钟 / 60 分；可用于限时作答，先隔离答案。

- 1a-g（8 分）：cube、on-set、CSOP/MinSOP、Quine 计数与 ROBDD 唯一性。对应 m02、m03、m06；PDF p.2。
- 1h（5 分）：五变量 Quine-McCluskey。对应 m03；PDF p.3。
- 1i（3 分）：覆盖表与启发式选择。对应 m03；PDF p.3。
- 1j（3 分）：ROBDD 与简化 SOP。对应 m06；PDF p.4。
- 1k（3 分）：Shannon 展开和二输入门实现。对应 m02、m06；PDF p.4。
- 2a-b（2 分）：literal removal 恒真检验。对应 m04；PDF p.5。
- 2c（3 分）：单独与共同 cube removal。对应 m04；PDF p.5。
- 2d（4 分）：ROBDD 到 LUT 分解表。对应 m05、m06；PDF p.6。
- 2e（2 分）：分解后不可达编码形成 don’ t-care。对应 m05；PDF p.6。
- 2f（6 分）：四轮状态等价和最小 Mealy 图。对应 m07；PDF p.7。
- 3（6 分）：完整事件表和波形。对应 m08；PDF p.8。
- 4a-c（3 分）：故障模型、fanout stem、D 算法失败边界。对应 m10、m11、m12；PDF p.9。
- 4d（3 分）：故障覆盖条件。对应 m10；PDF p.9。
- 4e（3 分）：AND-XOR 与布尔差分。对应 m11；PDF p.10。
- 4f-g（6 分）：结构路径与指定路径 D 算法。对应 m11；PDF p.11。

18.12 原创综合模拟 A：从机制到验证

60 分钟，60 分。本卷是针对课程目标编写的原创练习，不是历年原题、不预测补考试题。，建议先用 60 分钟独立作答，再展开参考答案。学习过程中可以分模块完成，此时不要把用时作为整卷速度证据。，未特别说明时使用完全指定的组合布尔函数，+ 表示 OR，乘积表示 AND，横线表示 NOT。，开放题按量表自评；等价表示与不同正确算法都可得分。只要前提、过程、结果和验证正确，不要求与参考文字一致。

18.12.1 mock-01 ·布尔表示与 cofactor ·6 分

变量顺序为(a, b, c)，a 是最高位。给定$f = ab + \bar{a}c$。

1. 列出全部 on-set（用十进制 minterm 编号）。
2. 写出$f_{a=0}$、$f_{a=1}$，并用 Shannon 展开重建f。
3. 解释为什么f 不是$b + c$，给出一个反例输入。

18.12.2 mock-02 ·Quine 到最小覆盖·8 分

变量顺序(a, b, c, d)。给定完全指定函数$f = \Sigma m(0, 1, 2, 3, 8, 9, 10, 11, 14, 15)$。

用 Quine 合并求全部素蕴含项，说明哪些是本质素蕴含项，给出 MinSOP。优化次序先最少乘积项，再最少 literal；不能只报最终化简式。

18.12.3 mock-03 •删除项与扩张的边界•5 分

给定$f = ab + \bar{a}c + bc$。

1. 证明bc 可以删除。
2. 在删除bc 后，能否分别从ab 中删除b 或删除a？每种情况用合法性条件或一个反例说明。
3. 一次操作合法是否自动意味着连续多个操作都合法？

18.12.4 mock-04 •ROBDD 的大小与语义•5 分

继续使用$f = ab + \bar{a}c$。不用补边（complemented edges），只用普通 0/1 分支。

1. 按$a \prec b \prec c$ 画 ROBDD，并给出非终端节点数。
2. 改为$b \prec c \prec a$，给出不同的非终端节点总数并解释。
3. 变量顺序改变时，函数是否改变？如何验证？

18.12.5 mock-05 •函数分解与编码成本•5 分

取绑定变量$X = (x_1, x_2)$，自由变量$Y = (y_1, y_2)$。四个绑定赋值对应 cofactor 依次为：

X	00	01	10	11
$f(X, Y)$	y_1	y_2	y_1	$y_1 + y_2$

求不同 cofactor 的数量、最少中间编码位数。采用$y_1 \mapsto 00$、$y_2 \mapsto 01$、$y_1 + y_2 \mapsto 10$，求 z_1, z_2 和一个正确的 $g(Z, Y)$。这是否必然减少总门数或 LUT 数？

18.12.6 mock-06 •FSM 稳定划分与反例序列•9 分

Mealy 机的转移表如下，单元格为“后继/输出”，复位状态为 E。

状态	x=0	x=1
A	B/0	C/1
B	A/0	D/1
C	C/1	A/0
D	D/1	B/0
E	A/0	E/1

逐轮划分等价状态直到稳定，给出最小状态表与二进制编码需要的寄存器位数。从 E 输入 101，写出输出序列。再找一个能区分 E 与 A 的输入序列。

18.12.7 mock-07 ·窄脉冲与待处理事务·6 分

网络为$e = \neg a$（门延迟 2 ns），$y = e \wedge b$（门延迟 3 ns）。初始稳定值 $a = 0, b = 1, e = 1, y = 1$。输入事件：1 ns 时 $a \leftarrow 1$；2 ns 时 $a \leftarrow 0$。

1. 在纯 transport 延迟模型下列出 e、y 的全部值变化时刻。
2. 2 ns 时 e 的当前值还是 1，是否可以因此不安排未来的 $e = 1$ 事务？解释。
3. 在“输入目标值连续稳定达到门延迟才改变输出”的简化惯性模型下，输出有何不同？

18.12.8 mock-08 ·检测与诊断的区别·5 分

可选测试的故障检测集合为：$t_0 = \{f_0, f_1\}$，$t_1 = \{f_1, f_2\}$，$t_2 = \{f_2, f_3\}$，$t_3 = \{f_0, f_3, f_4\}$，$t_4 = \{f_1, f_4\}$。预算相同，每次测试成本 1。

求最小检测测试集并证明其最小性。只保留 t_3 时，针对这 5 个模型故障的覆盖率是多少？所求最小检测集能否诊断区分这 5 个故障？

18.12.9 mock-09 ·内部故障、布尔差分与 D 传播·7 分

电路为 $n = a \wedge b$，$y = n \vee c$，信号顺序 (a, b, c)。

1. 为内部节点 $n/0$ 求全部测试向量，同时写出激活、传播、输入可实现条件，并用 D 符号标注传播。
2. 为输入端 $a/1$ 求全部测试向量。用好电路和坏电路输出确认结果。

18.12.10 mock-10 ·扫描测试时间估算·4 分

单条 scan chain 有 12 个触发器，要运行 100 个测试 pattern。每个 pattern 一次 capture；scan shift 时钟为 10 MHz，capture 也按一个同长度时钟周期计。相邻 pattern 允许“扫描输出上一个结果”和“扫描输入下一个向量”重叠，第一组需要先装载，最后一组需要完全读出。忽略模式切换等开销。

求总周期数和时间。若完全不重叠是多少？给出一项本估算忽略的真实成本。

19 附录 A：练习参考答案与自评量表

数值题仅核对答案值，不自动判断推导。文字题、图形题与设计题必须按评分点自评；看到答案后做对，应另找变式，不能算首次独立能力证据。

19.1 课程地图与数字设计的三种视角

19.1.1 m01-q1 ·2 分

分别属于行为域、结构域、物理域。可以都在逻辑层对应同一设计，域与抽象层级不同。

评分点：

- 三个域对应正确
- 指出可处于同一抽象层

19.1.2 m01-q2 •4 分

规范 → 架构 →RTL→ 逻辑优化 → 映射 → 布局布线 → 制造 → 测试。例：时序不满足时回到映射/逻辑/架构；可测性不足时回到插入扫描链或逻辑设计。反馈应说明发现的证据。

评分点：

- 主干顺序合理
- 库与约束进入流程
- 两个反馈点
- 反馈原因具体

19.1.3 m01-q3 •2 分

110的整数和为 2，因此 carry=1、sum=0。布尔 OR 的 1+1=1，而整数加法的 1+1=2；这两个记号环境必须区分。

评分点：

- 输出正确
- 解释布尔与整数加法差异

19.1.4 m01-q4 •2 分

2+1+1=4个时间单位。只适用于正文固定的门延迟模型。

评分点：

- 正确相加
- 报告时间单位和假设

19.1.5 m01-q5 •4 分

20输入共有 2^20=1,048,576 组；1000 随机向量不是穷举且可能重复。可做完全穷举或形式等价；还需说明输出检查器、约束与模型。不能只增加随机数量就称完整证明。

评分点：

- 指出未穷举
- 计算输入空间
- 提出有效验证方法
- 检查约束和判定器

19.1.6 m01-q6 •3 分

问题是实现相同全加器；基线为独立 SOP；机制是共享 a XOR b；成本可能减少重复逻辑但增加 carry 路径；carry 关键时可能不利；用八行真值表、统一库面积和最长路径验证。

评分点：

- 六项分析齐全
- 功能与性能证据分开
- 不把例子数字说成硬件实测

19.2 布尔空间、cube 与 Shannon 展开

19.2.1 m02-q1 ·2 分

有 4 个固定 literal、3 个横杠，维数 d=7−4=3，覆盖 8 个点。

评分点：

- 维数正确
- 区分维数与覆盖点数

19.2.2 m02-q2 ·2 分

先选 1 个自由位置，有 4 种；余下三位各有 0/1，共 8 种；结果 32。

评分点：

- 组合式正确
- 结果正确

19.2.3 m02-q3 ·4 分

cofactors分别是 c、b，差分 b XOR c。b=c 时差分为 0，因此 a 不影响输出；函数整体仍依赖 a。

评分点：

- cofactors 正确
- 差分正确
- 条件 b=c
- 局部不敏感与整体独立区分

19.2.4 m02-q4 ·3 分

没有相反固定值，距离 0；交集为 110。前者还含 100，后者还含 010，所以集合不同。

评分点：

- 距离 0
- 交集 110
- 给出集合不同证据

19.2.5 m02-q5 ·3 分

取 f=a：a=1 分支恒为 1，a=0 分支为 0。正确判据是两个 cofactor 都恒为 1。

评分点：

- 有效反例
- 两分支正确判据
- 说明量词是全部输入

19.2.6 m02-q6 •4 分

b=1,c=0时 a:0→1 使 f:0→1；b=0,c=1 时使 f:1→0。两种单调方向都被反例否定，因此 a 是 binate。

评分点：

- 两组输入正确
- 变化方向正确
- 结论 binate
- 不只凭表达式同时出现 a 与 a′ 下结论

19.3 精确两级综合：从全部 PI 到最小覆盖

19.3.1 m03-q1 •3 分

00- 覆盖 {0,1}；0-0 覆盖 {0,2}。二者并集是 {0,1,2}，0–还包含 3；若 3 是 off-set 就非法。横杠位置必须相同且只有一个固定比特不同。

评分点：

- 正确展开 cube
- 指出多覆盖点 3
- 给出合法合并条件

19.3.2 m03-q2 •2 分

至少 ceil(6/2)=3 项。下界只是必要条件；还需展示能覆盖六点的三项解，才能证明最优。

评分点：

- 给出 3
- 说明下界与可行构造缺一不可

19.3.3 m03-q3 •4 分

PI为 0-0、01-、-11。0 仅被 0-0 覆盖，7 仅被 -11 覆盖，二者是 EPI。最优 f=a′c′ +bc，共 2 项、4 literal。01- 不必选。

评分点：

- 全部 PI 无遗漏
- EPI 的独占点正确
- 最终式和成本正确
- 验证 off-set 无新增

19.3.4 m03-q4 •4 分

c的 on-set 为 {1,3,5,7}，包含 F 且不触及 off-set，因此合法。D 允许取 0 或 1，不是必须取 1；把 D 设为必盖会错误缩小合法解空间。本例碰巧 c 盖住全部 D，不代表一般必须如此。

评分点：

- 检查 F 被覆盖
- 检查没有 off-set
- 解释 D 非必需约束
- 指出本例与一般情况的区别

19.3.5 m03-q5 •4 分

不能。行支配还要求更大覆盖者的成本不高于被删者。可能另有便宜行覆盖 3，使 B 加其他行比 A 更优。应给出反例，如 C 只盖 {3}、成本 1，则 B+C 成本 3。

评分点：

- 拒绝错误删除
- 写成本条件
- 构造反例
- 区分行与列支配

19.3.6 m03-q6 •4 分

不共享按两个 SOP 的输入 literal 出现次数计为 8；共享 AND 平面 ab、ac、ad 共 6 个输入 literal，输出连接 4 个。共享 ab 的扇出增加，线负载或缓冲可能增加延迟。只有库映射和时序评估才能决定实际面积/速度。

评分点：

- 两种 literal 成本正确
- 注明输出连接另计
- 解释共享负载
- 不把逻辑成本当硬件实测

19.4 启发式优化与恒真检验

19.4.1 m04-q1 •2 分

应检查 h|c 是否恒为 1。f 已含 c，因此 f|c 天生为 1，不能说明其他项能替代 c。

评分点：

- 选择 h|c
- 解释错误检查为何恒真

19.4.2 m04-q2 •3 分

固定 a=b=1 得 c+c′ =1，扩张合法；最终 ab+bc 有 2+2=4 literal。

评分点：

- cofactor 正确
- 恒真结论
- 成本正确

19.4.3 m04-q3 •3 分

被移走的是 abc，它由 bc 覆盖，故合法。修改后为 ab c′ +bc，虽然等价但 literal 从 4 增至 5。

评分点：

- 移走区域正确
- 覆盖者正确
- 区分合法与成本更好

19.4.4 m04-q4 ·4 分

分别都可行。删 ab 后 abc 由 ac 盖、abc′ 已存在；删 ac 对称。一起删会漏 111；取 111 即反例。

评分点：

- 分别结论正确
- 两个覆盖理由
- 同时删除否定
- 111 反例

19.4.5 m04-q5 ·4 分

先定义允许邻域和成本；需检查所有允许的改善邻居不存在，或给等价完备判定。只访问了三个邻居仅说明未在这三处发现改进。若预算有限应报告停机条件和当前可行成本。

评分点：

- 邻域定义
- 完整性要求
- 区分搜索失败与最优证明
- 报告预算/停止条件

19.4.6 m04-q6 ·4 分

枚举全部输入，先算原函数和候选删后函数；对每个必需 on-set 确保候选为 1，对 off-set 确保为 0；若存在 DC 不要求其与原实现一致。返回首个反例，记录本次修改后的 cover。

评分点：

- 穷举输入
- F/R 两种检查
- 正确处理 D
- 给反例和当前 cover

19.5 多级逻辑与函数分解

19.5.1 m05-q1 ·3 分

ceil(log2 5)=3位，因为 2 位只能区分 4 类，3 位能区分 8 类；3<4，满足课程宽度压缩条件。

评分点：

- 编码位数正确
- 容量理由
- 有利分解判断

19.5.2 m05-q2 ·3 分

只有一个 bound 输入为 1，所以 z1=0,z0=1；g=0+1×1=1。原式 ab=ac=bc=0，d(a+b+c)=1，输出也为 1。

评分点：

- 中间码正确
- 后级输出正确
- 原式复算

19.5.3 m05-q3 •3 分

在 d=0 时两者输出分别为 0 与 1；相同 z 且相同 d 会要求 g 同时输出两个值，矛盾。

评分点：

- 给 d=0 反例
- 说明 g 确定性矛盾
- 全体 y 的等价要求

19.5.4 m05-q4 •3 分

h有两个独立输出，在普通单输出 LUT 模型中 h1、h0 各需一块；g 再一块，总共 3 块。只有目标器件明确提供可用的双输出结构时才能按其规则重新计数。

评分点：

- 指出 h 的两个输出
- 三块结论
- 模型依赖边界

19.5.5 m05-q5 •4 分

kernel可为 b+c，co-kernel 可为 a 或 d；f=(a+d)(b+c)+e。kernel 来自对特定 cube 的代数除法商；cofactor 是将变量固定后的函数，需处理所有项。

评分点：

- kernel 正确
- co-kernel 正确
- 分解正确
- 与 cofactor 区分

19.5.6 m05-q6 •4 分

枚举全部原输入 x,y，比较 f(x,y) 与 g(h(x),y)。仅检查 z 会忽略 h 是否把每个 x 正确映射；而不可达 z 上的 g 值可能是允许的 DC，不应误判。

评分点：

- 原输入枚举
- 检查组合等式
- 包含 h 的正确性
- 处理不可达内部码

19.6 ROBDD：共享余函数的决策图

19.6.1 m06-q1 •2 分

直接用 g 替代该节点，因为 x′ g+xg=g，函数不依赖此处的 x。

评分点：

- 约简结果
- Shannon 等式理由

19.6.2 m06-q2 ·2 分

根 a 加 b、c 两个节点，共 3 个非终端；若计 0/1 终端则是 5。

评分点：

- 正确构造
- 计数口径明确

19.6.3 m06-q3 ·4 分

5个。b 根、两个 c 节点、两个 a 节点；a 与 a′ 的 low/high 分别为 (0,1) 和 (1,0)，三元组不同，无补边约定下不能合并。

评分点：

- 节点数正确
- cofactor 分支正确
- a 节点不能合并理由
- 明确无补边

19.6.4 m06-q4 ·3 分

补函数为 a′ c′ +ab′，原函数为 (a+c)(a′ +b)。逐项取反、和变积。

评分点：

- 补函数 SOP
- De Morgan 正确
- 最终 POS 正确

19.6.5 m06-q5 ·3 分

不能无条件断言。变量序可能不同；本章 MUX 在两种顺序下分别 3/5 非终端但功能相同。同一固定序、相同边/终端约定的 ROBDD 才有规范性。

评分点：

- 否定无条件结论
- 给顺序反例
- 列规范性前提

19.6.6 m06-q6 ·4 分

常量 0/1 应只有终端；f=a 应只有一个非终端；low=high 节点应被删除。非平凡用 MUX 遍历 6 种变量序，穷举 8 组输入均与原函数一致，核对 a 优先 3 与 b,c,a 顺序 5 的无补边计数。

评分点：

- 常量边界
- 单变量或冗余边界
- 全部输入一致性
- 计数约定明确

19.7 FSM：状态建模、最小化与编码

19.7.1 m07-q1 ·2 分

Mealy依赖当前状态与当前输入；Moore 只依赖当前状态。本文 y_t 在更新前按 s_t 计算，下一状态为 s_(t+1)。

评分点：

- 两种函数正确
- 当前/下一时间约定正确

19.7.2 m07-q2 ·4 分

出货 1、找零 1 单位、下一余额 0。若不找零规格，则找零应为 0；不要混用两个规格。

评分点：

- 出货正确
- 找零正确
- 下一状态正确
- 说明规格变化

19.7.3 m07-q3 ·4 分

E 不可达。Π1={{A,B,D},{C}}；Π2={{A,B},{C},{D}}；Π3=Π2，稳定。

评分点：

- 可达性正确
- Π1 正确
- Π2 正确
- 稳定条件正确

19.7.4 m07-q4 ·3 分

11。A 经 C 返回 A 输出 10；D 经 B 到 C 输出 11。单输入 0 或 1 时输出相同，因此两拍已经是最短。

评分点：

- 输入序列
- 两条轨迹/输出
- 解释最短性

19.7.5 m07-q5 ·2 分

ceil(log2 5)=3。3 位能表示 8 种码，剩余 3 码仍需决定非法状态行为。

评分点：

- 结果正确
- 未使用编码策略

19.7.6 m07-q6 •4 分

q1=q0=1 时 D1=0、D0=0、y=0，下一拍回 X=00。若 11 作为 DC 化简，任何行为都可能被允许，不再保证恢复；要根据复位、故障模型与安全要求选择。

评分点：

- 代入三式正确
- 恢复结论
- DC 影响
- 假设与要求一致

19.8 逻辑仿真：把变化放回时间轴

19.8.1 m08-q1 •2 分

按生成与执行分开：NOT 在 11 ns 更新，AND 再过 6 ns，因此 17 ns。此处无其它路径、惯性取消或反馈。

评分点：

- 先算第一门 11 ns
- 再相加得到 17 ns

19.8.2 m08-q2 •4 分

10 ns 先排 1@13；12 ns 恢复 0 发生在待执行上升沿之前，宽 2 ns 小于拒绝时间 3 ns，上升响应被取消，输出保持 0。不能把输出画成 13–15 ns 的脉冲；那是 transport 结果。

评分点：

- 写出 1@13
- 识别宽 2<3
- 说明取消
- 输出保持 0

19.8.3 m08-q3 •3 分

p 在 12 ns 下降；q 在 10+5+2=17 ns 上升；y 在 13 ns 下降、18 ns 上升，低脉冲宽 5 ns。

评分点：

- 分别计算两路
- 得到输出 13 与 18
- 宽度 5

19.8.4 m08-q4 •4 分

NAND(0,0)=NAND(0,1)=1，因此 NAND(0,X)=1；NAND(1,0)=1 而 NAND(1,1)=0，因此 NAND(1,X)=X。完整对称表三行分别为 [1,1,1]、[1,0,X]、[1,X,X]，列序 0,1,X。

评分点：

- 表完整
- 控制值 0 使输出 1
- 非控制值 1 保留不确定
- 不把 X 当 don’t-care

19.8.5 m08-q5 •4 分

可能是零延迟组合反馈或非法锁存器激励后的对称状态。查看 100 ns 前后的输入、初值、敏感列表与驱动者，逐 delta 追踪循环，必要时采用有依据的门延迟模型。delta 不消耗物理时间，既没有可量化周期，也没有模拟亚稳态电气参数。

评分点：

- 给出组合反馈
- 给出非法激励/初值
- 逐 delta 定位
- 指出无物理频率

19.8.6 m08-q6 •4 分

固定延迟 5 单位和初值 0，分别输入宽 4、5、6 的孤立脉冲，再测宽 0 与延迟 0。transport 应保留所有非零宽度；本实验惯性规则应滤除 4、保留 5 和 6。记录队列插入/取消与波形两份证据；复核实际 VHDL 时要注明标准、reject 设置与边界事务规则。

评分点：

- 固定模型参数
- 测试小于等于大于边界
- 测零边界
- 双重记录并注明限制

19.9 VHDL：读懂并发与赋值的真实含义

19.9.1 m09-q1 •4 分

第一沿 (1,0,1)，第二沿 (0,1,0)。q 读取本轮执行前的 s；r 读取本轮立即更新的 v。

评分点：

- 第一沿正确
- 第二沿正确
- signal 读取旧值
- 变量立即赋值

19.9.2 m09-q2 •2 分

最终为 2。第二条赋值覆盖该 driver 同一目标时刻的旧事务；这不能推广成多个进程中的“最后写入者胜出”。

评分点：

- 结果 2
- 限定单 driver

19.9.3 m09-q3 •4 分

敏感列表缺 b。先置 sel=1、b=0 使输出 0；保持 a 与 sel 不变，把 b 改 1，进程不会唤醒，仿真输出可能仍 0。修复为 process(a,b,sel)，或在 VHDL-2008 模式使用 process(all)。

评分点：

- 发现缺 b
- 构造只有 b 变化
- 解释不会唤醒
- 给出版本正确修复

19.9.4 m09-q4 •4 分

它描述 en 高时透明、低时保持的锁存器，没有时钟沿条件。若规格是 en?d:r，补 else q<=r；若规格是 en?d:0，补 else q<= ‘0’ 。缺省值必须由规格决定。

评分点：

- 认出锁存器
- 指出保持分支
- 指出没有边沿
- 按明确规格补赋值

19.9.5 m09-q5 •4 分

在时钟进程内先令 q<=v，再执行 v:=d；v 需要保存上一次时钟的值，通常需要寄存器。立即生效描述的是本次 process 执行的变量语义，不排除跨时钟保留状态。

评分点：

- 时钟进程
- 先读取旧 v
- 再赋新 v
- 区分语义与硬件存储

19.9.6 m09-q6 •4 分

可能只是零延迟 signal 尚未经过 delta 更新，也可能真实模型含非零传播延迟或组合反馈。检查 driver、模型延迟与 delta 日志；在模型规定的稳定窗口比较全部关键输入，另外检查中间毛刺或不收敛。等 1 ns 一次通过不覆盖其它输入，也不证明物理时序。

评分点：

- 考虑 delta 未更新
- 考虑非零延迟/反馈
- 定义采样窗口
- 指出一次通过的限制

19.10 故障模型与最小测试集

19.10.1 m10-q1 •3 分

01 覆盖 a/1 与 y/1；11 覆盖 a/0、b/0、y/0，合计 5 项，漏 b/1；覆盖率 5/6≈83.33%。

评分点：

- 集合并集去重
- 指出漏 b/1
- 分母为六

19.10.2 m10-q2 ·3 分

保留 A 这一较难要求。任意检测 A 的输入也属于 T_B，因而检测 B；反向不成立，t2 只检出 B。以集合关系解释可避免支配命名方向歧义。

评分点：

- 包含关系 T_A⊂T_B
- 保留 A
- 给出反例 t2

19.10.3 m10-q3 ·4 分

00:{y/0}；01:{a/1,y/0}；10:{b/1,y/0}；11:{a/0,b/0,y/1}。a/1、b/1 和 a/0 分别强制 01、10、11，故最少三条，集合 {01,10,11} 达到下界。

评分点：

- 四行覆盖表
- 必选 01 与 10
- 必选 11
- 下界与可行解一致

19.10.4 m10-q4 ·4 分

原始 900/1000=90%；可测域口径 900/(1000−50)≈94.74%。超时表示未完成搜索，不是不可测证明；把它们排除会夸大结果。

评分点：

- 原始 90%
- 950 分母
- 94.74%
- 超时不等于不可测

19.10.5 m10-q5 ·4 分

不能。若在当前输入和观察域下可观察响应完全相同，它们是不可区分故障；覆盖只证明与正常不同。可增加内部观察点、利用更多响应位或不同测试模式，并重新建立响应字典。需注意多输出下单个检测位相同不必意味着完整响应向量相同。

评分点：

- 检测与诊断区分
- 限定观察域
- 提出额外观察
- 注意多输出完整响应

19.10.6 m10-q6 ·4 分

贪心 A 后剩 5、6，须再选 B 和 C，共 3 条；B∪C 覆盖全部六故障，仅 2 条。单条最多覆盖 4<6，故两条是最优。最大新增覆盖贪心不保证全局最小。

评分点：

- 贪心三条
- B 与 C 两条
- 一条不够的下界
- 解释贪心局限

19.11 ATPG：激活故障，再把差异送到输出

19.11.1 m11-q1 ·4 分

激活 n=1 即 a=b=1；传播要求 c=0；输入 (a,b,c)=110。正常 y=1、故障 y=0。

评分点：

- 激活 ab=1
- 传播 c=0
- 输入 110
- 输出差异核验

19.11.2 m11-q2 ·4 分

y=a(b+c)。a=0 时 y=0，a=1 时 y=b+c，所以差分 b+c。a/1 需要正常 a=0，检测条件为 a 取 0 且 (b OR c)=1；例如 010、001、011。

评分点：

- 两个 cofactor
- 差分 b+c
- 激活 a=0
- 至少 b 或 c 为 1

19.11.3 m11-q3 ·4 分

a 必须 1，b 和 c 必须不同，因此 110 与 101，共 2 个。111 中两路故障效应在 XOR 抵消。

评分点：

- 激活 a=1
- b≠c
- 列出 110 与 101
- 排除 111

19.11.4 m11-q4 ·4 分

可选 a=0,b=X,c=0，即 0X0。正常 n=0、故障 n=1，所以 n=D-bar；OR 旁输入 c=0 使 y=D-bar。a=0 证明 n=0，b 不受约束。

评分点：

- 激活 n=0
- 正确使用 D-bar
- 传播 c=0
- 证明并说明 X

19.11.5 m11-q5 •4 分

不能立即宣布。D-frontier 为空可能只是当前主输入或传播选择将所有路径屏蔽；需回溯其它尚未尝试决策。只有在算法假设成立、搜索完整且全部可行分支被排除时才可给不可测结论；超时是未解决。

评分点：

- 局部分支与全局分开
- 需要回溯
- 完整性条件
- 超时不等于不可测

19.11.6 m11-q6 •4 分

独立估算 0.5×0.5=0.25；真实 a 与 b 互补，AND 恒 0，概率 0。两个信号源于同一变量，完全相关。可控性启发式的非零数值不能证明该内部值实际可实现。

评分点：

- 估计 0.25
- 真实 0
- 相关性来源
- 不作可达性证明

19.11.7 m11-q7 •4 分

正确为 f=1 推出 a=1。f=0 推出 a=0 是逆命题，a=1,b=c=0 给出反例。前向 a=0 时 d=e=0，OR 对两输入局部差分均 1，符合双分支学习准则；反推 f=1 时局部规则只能给 d=1 或 e=1 的选择，全局关系直接约束 a。

评分点：

- 正确逆否
- 反例
- 局部差分均 1
- 减少可选分支

19.12 故障仿真：给定测试到底检出了谁

19.12.1 m12-q1 •3 分

x/1 被激活，因为故障值与正常 0 不同。若下游被控制值屏蔽或重汇合抵消，差异仍到不了输出，所以不保证检出。

评分点：

- 相反值 x/1
- 激活与检测分开
- 举出屏蔽机制

19.12.2 m12-q2 •3 分

最简单二值位并行表示可装 32−1=31 个。多值编码或其它额外状态会改变容量；这不是 31 倍实测速度保证。

评分点：

- 31 个
- 限定模型
- 不等于速度倍数

19.12.3 m12-q3 ·4 分

正常 a=b=u=y=1,c=0。a/0、b/0、u/0 均使 y 降 0；y/0 直接检出。c/1 把 c 升 1 但 y 原本已 1，不改变输出；同值故障均不激活。因此集合 {a/0,b/0,u/0,y/0}。

评分点：

- 正常值正确
- 三条传播故障
- 输出故障
- 排除 c/1

19.12.4 m12-q4 ·3 分

对称差为 {f1,f4}。f2 与 f3 各自同时翻转两输入，XOR 输出不变。

评分点：

- 使用对称差
- f1,f4 保留
- f2,f3 抵消

19.12.5 m12-q5 ·4 分

不够，可能检出的故障身份不同。固定同一网表、向量、故障列表、折叠/分母与模型，逐向量比较检出 ID 集合；对差异故障用串行注入和手算定位。若使用 fault dropping，还要明确首次检出与完整向量故障矩阵的区别。

评分点：

- 总数不够
- 统一任务口径
- 比较 ID 集合
- 独立基准复核

19.12.6 m12-q6 ·4 分

y=a XOR a 恒 0。stem 的任意固定故障同时改变两路，y 仍 0，所以不可观察。若只有一条 branch 固定为 0，a=1 时另一输入仍 1，y 从 0 变 1，可检测；branch/1 则 a=0 可检测。

评分点：

- 明确 y=a XOR a
- stem 同时改两路
- 差异抵消
- branch 独立可检

19.13 时序测试、Scan 与 BIST

19.13.1 m13-q1 ·3 分

C=N(P+1)+P=100×6+5=605 周期。逐段为首装 100、中间 4×100、末卸 100 以及 5 次 capture。

评分点：

- 首装末卸
- 重叠四段
- 公式与结果

19.13.2 m13-q2 ·3 分

下一状态 011。q2=1,q1=0,q0=0，所以新状态 (q1,q0 XOR q2,q2)=(0,1,1)。

评分点：

- 比特顺序正确
- 反馈 XOR 正确
- 结果 011

19.13.3 m13-q3 ·4 分

余式 110。1101 XOR 1011=0110，故响应 0110 也有同余式。两个响应差值是生成多项式 1011 的倍数。

评分点：

- GF(2) 减法为 XOR
- 余式 110
- 另一个响应 0110
- 倍数解释

19.13.4 m13-q4 ·4 分

最大周期需要本原多项式；不可约只保证相关商环构成域，不能保证 X 的乘法阶就是 2^{n-1}。$^{p=X}4+X^{3+X}2+X+1$，余式序列 0001→0010→0100→1000→1111→0001，周期 5 而不是 15。

评分点：

- 区分不可约与本原
- 列正确序列
- 周期 5
- 指出目标 15

19.13.5 m13-q5 ·4 分

不是。固定错误响应若与正常响应之差是生成多项式倍数，则每次都发生 aliasing。2^-8 是足够长且错误序列按假设均匀分布、映射条件成立时的近似；另外图样未激活或未传播的故障不属于签名 aliasing 问题。

评分点：

- 否定逐故障保证
- 倍数反例
- 均匀分布等假设
- 区分覆盖不足

19.13.6 m13-q6 •4 分

单链 11010 周期 =1.101 ms；四链 2760 周期 =0.276 ms。后者要求 4 路并行输入输出或额外压缩硬件；未计链不平衡、模式切换、ATE 带宽、布线和 shift 功耗/降频等。保持相同故障覆盖和 capture 协议后才可比较。

评分点：

- 单链结果
- 四链结果
- I/O 或压缩资源
- 至少三项限制及统一覆盖

19.14 原创模拟卷参考答案

19.14.1 mock-01 •布尔表示与 cofactor

依次代入000到111，输出为$0,1,0,1,0,0,1,1$，所以$F=\{1,3,6,7\}$。

$f_{a=0}=c$，$f_{a=1}=b$。Shannon 展开为$f=\bar{a}f_{a=0}+af_{a=1}=\bar{a}c+ab$。

反例取$(a,b,c)=(1,0,1)$：$f=0$，但$b+c=1$。这是带选择条件的 MUX，不能把两路数据直接 OR。

- on-set 完整且变量权重正确：2 分。
- 两个 cofactor 各 1 分：2 分。
- Shannon 公式与条件一致：1 分。
- 给出实际使两函数不同的反例：1 分。

19.14.2 mock-02 •Quine 到最小覆盖

先用四位二进制写 minterm：0000、0001、0010、0011、1000、1001、1010、1011、1110、1111。按 1 的个数分组，每次只合并一个固定位不同且其余位相同的 cube。

第一轮可得 000-、00-0、-000、00-1、-001、001-、-010、-011、100-、10-0、10-1、101-、1-10、1-11、111- (其中同一结果只保留一次)。第二轮得到 00–、-00-、-0-0、-0-1、-01-、10–、1-1-。第三轮得到 -0–。1-1-不能再合法扩张。

全部 PI 为$p_1=\bar{b}$ (-0–) 和$p_2=ac$ (1-1-)。p_1 覆盖 0、1、2、3、8、9、10、11；p_2 覆盖 10、11、14、15。minterm0 只能由 p_1 覆盖，14 只能由 p_2 覆盖，故二者均为 EPI。覆盖条件强制选择二者，$f=\bar{b}+ac$，2 项、3 个 literal。

复核：$b=0$ 的 8 个输入全部为 1；$b=1$ 时只有 $a=c=1$、d 任意的 2 个输入为 1，共 10 个，与给定集合一致。

- 二进制及合法逐轮合并：3 分。
- 完整 PI 且没有把非素项留作 PI：2 分。
- 给出独占 minterm 说明两个 EPI：1 分。
- MinSOP 与成本：1 分。
- on-set/真值表复核：1 分。

19.14.3 mock-03 •删除项与扩张的边界

令 $h=ab+\bar{a}c$。把 $bc=1$ 代入，得到 $h_{bc}=a+\bar{a}=1$，所以 $bc\subseteq h$，可删除。也可以使用共识定理。

删除 b 把 ab 扩为 a。输入 100 原函数为 0 而扩张后为 1，非法。删除 a 把 ab 扩为 b。输入 010 原函数为 0 而扩张后为 1，非法。

每次修改之后的其余覆盖可能已经变化，后续步骤必须对当前函数/当前 cover 重新检查。分别合法不推出一起合法；例如两个重复项各自都能从原 cover 中删除，但不能同时全部删掉。

- 写出 h 并以 cofactor 恒真/共识证明删除：2 分。
- 两个非法扩张的有效反例：各 1 分。
- 说明连续操作需要重新检查并有解释：1 分。

19.14.4 mock-04 •ROBDD 的大小与语义

顺序 a, b, c：根为 a。根的 0 边到 c 节点，1 边到 b 节点；b, c 各自 0 边到终端 0、1 边到终端 1。共 3 个非终端节点。

顺序 b, c, a：根 b 的 0 分支为 $\bar{a}c$，1 分支为 $a + c$。前者的 c 节点 0 边到 0、1 边到 $\bar{a}$ 节点；后者的 c 节点 0 边到 a 节点、1 边到 1。普通边约定下 a 和 $\bar{a}$ 节点的两个孩子相反，不能合并。共 1 个 b、2 个 c、2 个 a 节点，合计 5 个非终端节点。

两图语义相同，8 种输入求值必须完全一致。固定变量顺序的归约图具有规范性；这个性质并不保证不同顺序具有相同大小。若允许补边，节点计数口径会改变，不能与本题混比。

- 第一张图的边方向与标签正确：2 分。
- 第二张结构与 5 节点计数：2 分。
- 同函数验证及固定顺序条件：1 分。

19.14.5 mock-05 •函数分解与编码成本

不同 cofactor 有 3 种，因此至少需要 $\lceil \log_2 3 \rceil = 2$ 位中间编码。

由给定映射：$h(00) = 00, h(01) = 01, h(10) = 00, h(11) = 10$。所以 $z_1 = x_1 x_2$，$z_2 = \bar{x}_1 x_2$。编码 11 不可达。

一个可用的组合函数为$g = \bar{z}_2 y_1 + (z_1 + z_2) y_2$。代入 00 得 y_1，01 得 y_2，10 得 $y_1 + y_2$。编码 11 的值由实现选择，本例输出 y_2，并不违反原函数约束。

原来绑定变量 2 位，编码后仍 2 位，没有减少此接口宽度。分解增加 h 逻辑，是否节省门/LUT 还取决于目标库、LUT 输入数、共享机会和深度限制，不能仅凭“进行了分解”宣称改进。

- 3 类与 2 位：1 分。
- 两个编码函数：2 分。
- g 在可达编码上正确，解释 11：1 分。
- 说明成本/资源假设，否定必然节省：1 分。

19.14.6 mock-06 •FSM 稳定划分与反例序列

按当步输出向量，第一轮 $P_1 = \{\{A, B, E\}, \{C, D\}\}$。按 P_1 对后继编码：A/B 的后继组对是 (第一组, 第二组)，E 是 (第一组, 第一组)，所以 E 被分离。C/D 仍具有相同后继组对。

第二轮 $P_2 = \{U = \{A, B\}, V = \{C, D\}, W = \{E\}\}$。使用 P_2 再计算：U 在 0 下到 U/0、在 1 下到 V/1；V 在 0 下到 V/1、在 1 下到 U/0；W 在 0 下到 U/0、在 1 下到 W/1。各组内签名仍相同，划分稳定。

最小状态	x=0	x=1
U	U/0	V/1
V	V/1	U/0
W	U/0	W/1

3 个状态二进制编码需 2 位。全部原状态从 E 可达，不能以不可达为由删掉 E。从 E 输入 101：E –1/1→ E –0/0→ A –1/1→ C，输出 101。

用输入 10 区分 E 和 A：E 产生 10；A 先 1 进入 C 输出 1，后 0 在 C 输出 1，产生 11。因此输出向量相同只是第一轮等价，不足以说明所有未来行为一致。

- 首轮按输出正确分组：2 分。
- 第二轮正确细化并验证稳定：2 分。
- 最小状态表/图正确：2 分。
- 2 位寄存器和输出 101：1 分。
- 有效区分序列及两条输出：2 分。

19.14.7 mock-07 ·窄脉冲与待处理事务

transport 保留每次输入变化所产生的延迟事务。1 ns 的输入上升安排 $e=0$ 于 3 ns；2 ns 的输入下降安排 $e=1$ 于 4 ns。3 ns 时 e 变 0，安排 $y=0$ 于 6 ns；4 ns 时 e 变 1，安排 $y=1$ 于 7 ns。因此 e 在 [3,4) ns 为 0，y 在 [6,7) ns 为 0，其余仍 1。

不能因为 2 ns 时 e 当前为 1 就省略 4 ns 的恢复事务：队列里已经有 3 ns 的 $e=0$。只比较当前输出会丢失未来恢复，使仿真错误地永久停在 0。应区分事务（计划写值）与 event（信号值实际改变）。

简化惯性模型中 $a=1$ 只维持 1 ns，小于反相器的 2 ns 延迟，在 3 ns 到来前目标已经返回原值；待处理脉冲被取消。e、y 都保持 1。这是本题明示的稳定窗口模型，不宣称覆盖 VHDL 全部惯性事务删除语义。

- e 的 3/4 ns 变化：2 分。
- y 的 6/7 ns 变化：2 分。
- 说明不能按当前值丢恢复事务：1 分。
- 惯性无输出脉冲及模型范围：1 分。

19.14.8 mock-08 ·检测与诊断的区别

选 t_1 和 t_3，并集为全部 5 个故障。任何单个测试最多覆盖 3 个，不可能单测覆盖全部，所以 2 个测试达到最小数量。枚举其他两两并集会发现它是本表唯一的两测试全覆盖解。

只有 t_3 时覆盖率为 $3/5=60\%$，分母是题目给出的 5 个单故障模型，不能解释为 60% 的真实物理缺陷。

按 (t_1,t_3) 写检测签名：$f_0=(0,1)$，$f_1=(1,0)$，$f_2=(1,0)$，$f_3=(0,1)$，$f_4=(0,1)$。虽然所有故障至少被检测一次，仍有重复签名，不能唯一诊断。

- 给出 {t1,t3}：1 分。
- 下界证明不能 1 个测试：1 分。
- 60% 及正确分母：1 分。
- 签名或等效理由证明不能区分：2 分。

19.14.9 mock-09 ·内部故障、布尔差分与 D 传播

内部 $n/0$ 要激活，正常 $n=1$，故 $a=b=1$。输出对 n 的布尔差分为 $(1+c)\oplus(0+c)=1\oplus c=\bar{c}$，故需 $c=0$。唯一向量为 110。D 表示好值 1/坏值 0，n 处为 D；OR 另一输入 c 取非控制值 0，输出 y 也为 D。好电路输出 1，故障电路输出 0。

输入 $a/1$ 需要正常 $a=0$。$y_{a=1}=b+c$，$y_{a=0}=c$，差分 $(b+c)\oplus c=b\bar{c}$。因此 $b=1,c=0$，唯一向量 010。好电路 n=0、y=0；故障电路 a 固定 1 导致 n=1、y=1，产生 $\bar{D}$。没有满足激活条件的向量，即使传播条件成立也不能检测该故障。

- n/0 激活与输入可实现条件：1 分。
- 正确差分与 c=0：1 分。
- 唯一 110 与 D 传播：1 分。
- 好坏输出验证：1 分。
- a/1 激活和差分：2 分。
- 唯一 010 且好坏输出正确：1 分。

19.14.10 mock-10 •扫描测试时间估算

重叠时：首次装载 12 周期；100 次 capture 共 100 周期；100 次读出共 1200 周期（前 99 次读出同时装载下一组）。总数 $12 + 100 + 1200 = 1312$ 周期。10 MHz 的周期是 0.1 μs，因此总时间 131.2 μs。

不重叠时每组装载 12、capture1、读出 12，共 25 周期，100 组为 2500 周期，即 250 μs。以一个 pattern 检查边界：两模型都应得到 25 周期，这能查出漏掉初次装载或最后读出的公式。

本模型忽略 scan enable 切换、压缩/解压结构、capture 与 shift 不同时钟、功耗限制造成的停顿及测试机 I/O 带宽。测试周期估算不等于真实芯片测试时间测量。

- 重叠的 1312 周期及完整装载/读出边界：2 分。
- 131.2 μs 与不重叠 2500 周期/250 μs：1 分。
- 明确至少一个现实开销或边界检查：1 分。

20 附录 B：术语中英对照

- **寄存器传输级 / Register-transfer level (RTL)**：用寄存器状态与寄存器之间运算描述同步设计。
- **技术映射 / Technology mapping**：把逻辑网络变换为目标库可实现的单元结构。
- **设计空间 / Design space**：在行为、结构、物理视图及不同抽象层上的候选实现集合。
- **立方项 / Cube**：固定部分变量而让其余变量自由的输入点集合。
- **余因子 / Cofactor**：把指定变量固定为常数后的剩余函数。
- **不关心集合 / Don’t-care set**：规范允许实现输出取 0 或 1 的输入集合。
- **单极性 / Unate**：对某变量在全部其他输入固定时只呈一个单调方向。
- **布尔差分 / Boolean difference**：同一变量两个 cofactor 的 XOR，表示翻转该变量是否改变输出。
- **素蕴含项 / Prime implicant (PI)**：合法且不能继续删除 literal 扩张的 cube。
- **必要素蕴含项 / Essential prime implicant (EPI)**：至少覆盖一个不被其他 PI 覆盖的 on-set 点。
- **覆盖选择式 / Petrick expression**：每个必盖列写候选行之和，再把各列相乘的布尔选择约束。
- **不可冗余覆盖 / Irredundant cover**：不能直接删去其中任一项而保持覆盖的解；不保证全局最小。
- **扩张 / Expand**：删除 literal，让 cube 覆盖更多点；新增点不得违反规范。
- **缩小 / Reduce**：添加 literal，让 cube 覆盖更少点；移走的必需点必须仍被覆盖。
- **配置图 / Configuration graph**：以可行 cover 为节点、允许修改为边描述搜索空间的图。
- **恒真检验 / Tautology check**：判断函数是否对全部允许输入恒等于 1。
- **绑定变量 / Bound variables**：只通过前级 h 压缩后进入后级 g 的输入集合。
- **自由变量 / Free variables**：直接进入后级 g 的输入集合。
- **余函数等价类 / Cofactor equivalence class**：使剩余自由变量函数完全相同的一组 bound 输入。
- **核 / Kernel**：由 SOP 对 cube 作代数除法获得的 cube-free 商。
- **重汇合 / Reconvergence**：同一信号分叉后又在某个后继逻辑中汇合。
- **有序二叉决策图 / Ordered BDD (OBDD)**：所有根到终端路径遵循同一变量序的布尔决策图。
- **约简有序二叉决策图 / Reduced OBDD (ROBDD)**：删除无效测试并合并相同子图后的 OBDD。
- **规范表示 / Canonical representation**：在指定约定下同一对象具有唯一形式（这里指同构意义）。
- **补边 / Complement edge**：用边标记表示对子函数取反的实现技巧；会改变节点统计口径。
- **Mealy 状态机 / Mealy FSM**：输出取决于当前状态和当前输入的有限状态机。
- **Moore 状态机 / Moore FSM**：输出只取决于当前状态的有限状态机。
- **状态等价 / State equivalence**：从两个状态出发，对所有有限输入序列输出均相同。
- **分区细化 / Partition refinement**：根据输出和后继组差异反复拆分状态集合直到稳定。
- **独热编码 / One-hot encoding**：通常每个状态占一个触发器，合法状态仅一位为 1 的编码。
- **事务 / transaction**：driver 中计划在某时刻执行的一次信号赋值；不一定造成值变化。
- **事件 / event**：信号当前值发生变化，可唤醒敏感进程。
- **delta 周期 / delta cycle**：同一物理仿真时刻内部的更新/求值轮次。
- **惯性延迟 / inertial delay**：按 driver 事务规则拒绝短脉冲的延迟机制。

- **传输延迟 / transport delay**：主要表达变化平移的延迟机制；仍存在未来波形替换规则。
- **静态冒险 / static hazard**：稳态应保持不变，却因路径延迟出现短暂错误电平。
- **接口体 / entity**：定义模块端口与 generic，不规定内部实现。
- **结构体 / architecture**：一个 entity 的实现描述，可行为式或结构式。
- **信号赋值 / signal assignment**：<= 安排 driver 事务；不会立即改变当前进程读取的 signal。
- **变量赋值 / variable assignment**：:= 立即改变变量的当前值；是否需要硬件存储取决于用法。
- **敏感列表 / sensitivity list**：决定哪些信号 event 可重新激活该进程。
- **自检测试平台 / self-checking testbench**：自动产生激励并用断言比较预期结果的仿真环境。
- **单固定型故障 / single stuck-at fault**：一次只把一个指定逻辑位置固定为 0 或 1 的模型。
- **故障字典 / fault dictionary**：各输入下正常及各故障电路的响应集合。
- **检测 / fault detection**：观察到正常电路与某故障电路响应不同。
- **诊断 / fault diagnosis**：通过响应区分不同故障假设。
- **故障折叠 / fault collapsing**：用等价或支配关系减少需要单独处理的故障要求。
- **不可冗余测试集 / irredundant test set**：删掉任何一个测试就失去当前覆盖完整性；未必全局最小。
- **布尔差分 / Boolean difference**：变量 0/1 两个 cofactor 的 XOR，描述翻转敏感性。
- **故障激活 / fault activation**：让正常电路故障点取与固定故障相反的值。
- **故障传播 / fault propagation**：使正常/故障差异到达可观察输出。
- **证明赋值 / justification**：把内部所需值落实为相容的主输入赋值。
- **D 前沿 / D-frontier**：输入含故障效应而输出尚不确定的门集合。
- **自掩蔽 / self-masking**：同一故障沿多条路径重汇合后输出差异抵消。
- **故障丢弃 / fault dropping**：达到目标检测条件后把已检故障移出后续待仿真集合。
- **局部可观察性 / local observability**：当前其它输入固定时，翻转一个门输入能否翻转门输出。
- **扇出干线 / fanout stem**：分叉前的公共信号位置，故障同时影响各分支。
- **扇出分支 / fanout branch**：分叉后的单条连接，其故障仅作用于该支路。
- **演绎故障仿真 / deductive fault simulation**：共享正常逻辑求值并传播故障列表的算法。
- **对称差 / symmetric difference**：属于两个集合中的恰好一个的元素集合。
- **时间帧展开 / time-frame expansion**：用多份组合逻辑和跨帧寄存器连接表示时序行为。
- **扫描链 / scan chain**：测试模式下串接寄存器以设置和读出内部状态。
- **捕获 / capture**：在功能模式的指定时钟边沿把逻辑响应存入寄存器。
- **线性反馈移位寄存器 / LFSR**：用 GF(2) 线性反馈更新状态的移位寄存器。
- **多输入签名寄存器 / MISR**：将多个响应位并行压缩进签名状态的寄存器结构。
- **签名混叠 / aliasing**：不同响应序列压缩后得到同一签名。
- **内建自测试 / BIST**：片上生成图样、执行测试、压缩并判定响应的结构。

21 附录 C：来源与阅读边界

这里只收录课程引用的来源。完整 101 份原始文件的索引、哈希、抽查记录请见网站下载 source-index.json。原始 PDF 不附入公开分发包。

21.1 T25 •2025 Tutorial Notes —Design Methodology and Automation

类型：用户提供的课程/习题/试卷材料；版本：Issue October 31, 2025；发布日期：2025-10-31；访问/核验日：2026-11-08。

阅读范围：2025-10-31 版本；Tutorial 1–11 按主题交叉核对，题目与解题机制定位见各章。正文印刷页 +2=PDF 页。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.2 L25 •2025 Lecture Notes —Design Methodology and Automation

类型：用户提供的课程/习题/试卷材料；版本：Issue October 31, 2025；发布日期：2025-10-31；访问/核验日：2026-11-08。

阅读范围：2025-10-31 版本；I 逻辑综合、II 仿真、III 测试的课程主体各章；全文目录与章节引用定位。正文印刷页 +4=PDF 页。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.3 F006 •Day 2_DMA_Heuristic_Opt_2018.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 Day 2_DMA_Heuristic_Opt_2018.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：已做文件清单与文本提取；具体精读范围以章节引用为准，不声明全部页面精读。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.4 F055 •DMA22_retake.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 DMA22_retake.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：整卷题目与分值逐小题核对；日期和是否含答案见历年卷地图。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.5 F071 •#DMA - Slides - Testing.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 #DMA - Slides - Testing.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：已做文件清单与文本提取；具体精读范围以章节引用为准，不声明全部页面精读。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.6 F072 •#Intro (DMA - Folien 01 - Lecture Introduction 202311).pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 #Intro (DMA - Folien 01 - Lecture Introduction 202311).pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：已做文件清单与文本提取；具体精读范围以章节引用为准，不声明全部页面精读。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.7 F073 •#Logic Simulation (DMA - Folien 10 - Logiksimulation 11-09).pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 #Logic Simulation (DMA - Folien 10 - Logiksimulation 11-09).pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：已做文件清单与文本提取；具体精读范围以章节引用为准，不声明全部页面精读。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.8 F081 •#19_retake_solution.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 #19_retake_solution.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：已做文件清单与文本提取；具体精读范围以章节引用为准，不声明全部页面精读。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.9 F082 •#20_solution.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 #20_solution.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：已做文件清单与文本提取；具体精读范围以章节引用为准，不声明全部页面精读。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.10 F084 •#21_solution_update.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 #21_solution_update.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：已做文件清单与文本提取；具体精读范围以章节引用为准，不声明全部页面精读。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.11 F085 •#DMA22_retake_solution.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 #DMA22_retake_solution.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：首/中/末图像核对题型、年份、指定路径 D 算法。答案带手写注释，不自动视为无误。PDF 页：1,4,6。标 DMA22 的手写答案实际标题 Exam2023，与 F055 对应。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.12 F086 •19_retake_exam.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 19_retake_exam.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：整卷题目与分值逐小题核对；日期和是否含答案见历年卷地图。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.13 F087 •20_exam.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 20_exam.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：整卷题目与分值逐小题核对；日期和是否含答案见历年卷地图。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.14 F088 •21_exam.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 21_exam.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：整卷题目与分值逐小题核对；日期和是否含答案见历年卷地图。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.15 F089 •21_retake_exam.pdf

类型：用户提供的课程/习题/试卷材料；版本：文件标识 21_retake_exam.pdf；发布日期：按卷内封面识别；访问/核验日：2026-11-08。

阅读范围：整卷题目与分值逐小题核对；日期和是否含答案见历年卷地图。。

原 PDF 为用户本地文件，未公开镜像；扫描/手写内容采用明示抽样范围，文字提取可能丢失反相线等符号。

21.16 TUM-COURSE •TUM EDA —Design Methodology and Automation 官方课程页

类型：官方课程范围；版本：访问时公开页；发布日期：未标固定更新日；访问/核验日：2026-11-08。

阅读范围：Contents、Additional Material 及 Laboratory 入口；用于确认课程名称和综合/仿真/VHDL/测试范围。。

网页不是本次补考通知，不据此推断考试日期、允许资料或押题。

原始来源：原文链接

21.17 ABC-DOC •ABC: A System for Sequential Synthesis and Verification

类型：作者工具文档；版本：公开项目说明快照；发布日期：页面未标统一版本；访问/核验日：2026-11-08。

阅读范围：Introduction、Highlights 中的 AIG、综合与验证协同描述。。

只作机制与工程入口；不引用页面中依赖旧机器的运行时间作为本站预测。

原始来源：原文链接

21.18 ABC-CODE •berkeley-abc/abc 作者代码

类型：作者代码与实验入口；版本：master 入口；未固定提交做性能复现；发布日期：持续维护；访问/核验日：2026-11-08。

阅读范围：README、代码目录和 test 目录入口；可作为课程之后的深入实现阅读。。

未在本任务中编译执行 ABC，不宣称完成该工具实验或得到性能结论。

原始来源：原文链接

21.19 REFERENCE-SITE •memory-architecture-lab 参考站与源码

类型：用户指定的结构参考；版本：codex/source分支访问时快照；发布日期：README 资料截止 2026-10-30；访问/核验日：2026-11-08。

阅读范围：README、dist/index.html、app.js 导航与本地记录结构、知识树与下载流程。。

复用静态站、hash 章节路由、课程数据分离与离线下载的结构思路；DMA 内容与算法单独重建，原站保持原状。

原始来源：原文链接

21.20 BRYANT86 •Randal E. Bryant —Graph-Based Algorithms for Boolean Function Manipulation

类型：原始论文（作者注释电子版）；版本：IEEE Transactions on Computers C-35(8), pp.677–691；作者重绘图并附 Update 注释的 28 页电子版；发布日期：1986-08 (原始发表)；电子注释版未标统一修订日期；访问/核验日：2026-11-08。

阅读范围：§1–2：PDF pp.1–7，Shannon/cofactor、固定变量序、约简规则及规范性；导论中变量序敏感与表示规模最坏情形讨论。。

区分 1986 原刊页码与 28 页电子版 PDF 页码；不引用历史机器运行时间作为现代性能，不声称对全部函数都紧凑。网站的 MUX 节点数为独立手算与程序穷举核验。

原始来源：原文链接

21.21 MULTILEVEL90 •Robert K. Brayton / Gary D. Hachtel / Alberto L. Sangiovanni-Vincentelli —Multilevel Logic Synthesis

类型：原始综述论文（作者研究机构扫描档案）；版本：Proceedings of the IEEE 78(2), pp.264–300；37 页扫描 PDF；发布日期：1990-02；访问/核验日：2026-11-08。

阅读范围：视觉核对首页印刷 p.264/PDF p.1 的作者和发表版本；§IV.D Kernels and Kernel Intersections，印刷 p.272/PDF p.9 的 weak division、cube-free、kernel/co-kernel 定义及 Table 1。。

扫描 PDF 无可抽取文本，采用渲染页核验；仅上述章节精读，不声称全文复现实验。kernel 枚举为深入选学，课程主线仍是 L25/T25 的函数分解。

原始来源：原文链接

21.22 SIS92 •Ellen M. Sentovich 等—SIS: A System for Sequential Circuit Synthesis

类型：原始技术报告元数据；版本：UC Berkeley technical report UCB/ERL M92/41；发布日期：1992-05；访问/核验日：2026-11-08。

阅读范围：Berkeley 官方报告元数据页：题名、报告编号、作者和原始报告链接，用于定位 L25 提及的 SIS 背景。。

原始 PDF 通过 web 获取超时，未精读全文；不引用其内部算法细节、基准数字或声称运行 SIS。此来源只提供研究线索。

原始来源：原文链接

21.23 IEEE1076 •IEEE 1076-2019 —IEEE Standard for VHDL Language Reference Manual

类型：规范元数据；版本：IEEE 1076-2019；发布日期：2019-12-23；访问/核验日：2026-11-08。

阅读范围：官方公开页面的标准范围、2019-09-05 批准日期、2019-12-23 发布日期、前序 1076-2008 与获取方式。。

完整规范需要购买或订阅；未读取付费全文，不引用未经查阅的条款编号。

原始来源：原文链接

21.24 GHDL-SIM •GHDL Documentation —Simulation (runtime)

类型：官方工具文档；版本：页面标识 7.0.0-dev (访问时快照)；发布日期：未标固定发布日期；访问/核验日：2026-11-08。

阅读范围：Simulation options 中的–assert-level、–stop-time、–stop-delta、–disp-time；Export waveforms 中的–wave 与 VCD 说明。。

工具运行选项属于 GHDL 实现接口，不能当成 IEEE 标准要求；未把开发文档默认值当成本机安装版本验证结果。

原始来源：原文链接

21.25 GHDL-STD •GHDL Documentation —Implementation of VHDL

类型：官方工具文档；版本：7.0.0-dev；VHDL 标准模式 87/93/00/02/08/19；发布日期：未标固定发布日期；访问/核验日：2026-11-08。

阅读范围：VHDL standards 小节：–std 选择、版本兼容组以及 2008/2019 部分实现限制。。

页面会更新；课程示例明确区分旧写法与 2008 特性，不声称 GHDL 完整实现所有 2019 特性。

原始来源：原文链接

21.26 ROTH66 •J. Paul Roth —Diagnosis of Automata Failures: A Calculus and a Method

类型：原始论文（全文访问受限）；版本：IBM Journal of Research and Development 10(4), pp.278–291；发布日期：1966-07；访问/核验日：2026-11-08。

阅读范围：核对论文题名、作者、年代与可索引摘要中的 D-cube 和 D-algorithm 来源；课程机制以 L25/T25 为主要证据。。

DOI/IBM 页面抓取不可用，Bitsavers 原始扫描副本返回 403；未读取全文，不引用性能数据或声称全文复现。

原始来源：原文链接

21.27 GOEL80 •Prabhakar Goel —An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits

类型：原始会议论文（索引可见，全文受限）；版本：FTCS-10, 1980, pp.145–151；与 1981 期刊版区分；发布日期：1980；访问/核验日：2026-11-08。

阅读范围：检索可索引摘要与首页信息；PODEM 名称、面向组合逻辑的隐式枚举思路。。

PDF 抓取 403。1981 IEEE TC C-30(3) pp.215–222 DOI 10.1109/TC.1981.1675757 也未读取全文；两版本参数与性能不混用。PODEM 为选学。

原始来源：原文链接

21.28 AGRAWAL-FAULTSIM •Michael L. Bushnell / Vishwani D. Agrawal —VLSI Testing Course, Lecture 7: Fault Simulation

类型：作者课程资料；版本：作者课程目录列 Lecture 7，20 slides；历史课程版本；发布日期：页面列 2004–2006 课程，单个讲义日期未核定；访问/核验日：2026-11-08。

阅读范围：作者课程目录及 Lecture 7 故障仿真、Lecture 11 ATPG、Lecture 25 BIST 的主题对应。本站四类方案仅作概念扩展，算例自行构造。。

只核对公开课程目录，未把未读取的 PPT 细节当成证据；无性能数字引用。核心手算来源为 L25/T25。

原始来源：原文链接

21.29 OPENTITAN-LFSR •OpenTitan —Primitive Component: LFSR

类型：作者工程文档/实现；版本：master 页面于访问日所见，非固定 release；发布日期：未标固定发布日期；访问/核验日：2026-11-08。

阅读范围：Parameters、Theory of Operations：GAL_XOR/FIB_XNOR，DefaultSeed，锁死保护，引用 XAPP052 的抽头表与声明验证范围。。

本站未运行 OpenTitan RTL，也未复现文档所述全部位宽验证；其锁死保护属于该实现，不能自动外推普通课堂 LFSR。

原始来源：原文链接

21.30 XAPP052 •Peter Alfke —Efficient Shift Registers, LFSR Counters, and Long Pseudo-Random Sequence Generators

类型：厂商应用笔记；版本：XAPP052 version 1.1；发布日期：1996-07-07；访问/核验日：2026-11-08。

阅读范围：官方版本元数据、摘要及 Introduction：LFSR 抽头/位编号约定、XNOR 例与锁死处理说明。。

历史 XC4000 系列实现材料；不引用现代器件性能，不直接套用其 XNOR 抽头到本站 GF(2) 余式递推。

原始来源：原文链接

21.31 本地资料审计

共 101 份真实 PDF，1710 个 PDF 页（含重复与合订）。PDF 页码使用阅读器页序，不总等于印刷页。

- **F001 / F070 / F009**：同一 112 页讲义体系；F001 封面 Issue October 31, 2025；F070 有注释且文本/文件大小不同；F009 旧版须单独标年份。不能当三个独立证据来源。
- **F002 / F093 / F057**：同一 99 页 Tutorial 体系；F002 为 2025 干净文本版，F093 带注释，F057 旧版。
- **F083 / F089 / F050 / F051**：卷面 2022-05-06 同场补考与答案，不重复计数。F083 与 F089 非二进制重复但题目一致。
- **F084 / F052**：2021-12-10 答案的标注/非标注副本。
- **F090 / F091 / F092 / F074 / F075 / F076 / F077 / F078 / F079 / F080 / F094 / F095 / F096 / F097 / F098 / F099 / F100 / F101**：合订、扫描、分日文件之间存在内容重叠；仅当复习索引，不增加独立证据计数。

21.31.1 扫描与合订本阅读范围

- **F003**，PDF 页 1：四栏手写综合速查表：布尔空间、QM、ROBDD、启发式、共享逻辑、多级与分解。整页上、下区域放大抽查；未逐字符转写所有公式。
- **F004**，PDF 页 1：四栏手写速查表：分解编码、FSM、事件仿真、VHDL、故障表、布尔差分、D 算法。整页上、下区域放大抽查；未把手写结论当权威依据。
- **F090**，PDF 页 1、17、33：33 页四拼合订：讲义节选、2025 课堂笔记与习题解。抽查首/中/末三页，文本可提取但多栏重叠；未通读 33 页所有子页。
- **F091**，PDF 页 1、8、15：15 页四拼 2025 课堂手写笔记：布尔基础至全局蕴含、scan。抽查首/中/末三页，手写内容不能依赖文本提取。
- **F092**，PDF 页 1、6、11：11 页四拼习题解：MUX/布尔、FSM、SR latch 与事件表。抽查首/中/末三页；与分日习题资料存在内容重叠。
- **F074**，PDF 页 1、6、11：2025-11-03：布尔模型、K-map/cube graph、Quine。图像抽查
- **F075**，PDF 页 1、7、13：2025-11-04：implicant/cover、配置图、恒真判定。图像抽查

- **F076**，PDF 页 1、5、9：2025-11-05：多输出共享、多级因式分解、函数分解。图像抽查
- **F077**，PDF 页 1、3、5：2025-11-06：FSM、状态等价、编码与逻辑实现。图像抽查
- **F078**，PDF 页 1、5、8：2025-11-07：故障激活/观测、布尔差分运算。图像抽查
- **F079**，PDF 页 1、6、10：2025-11-10：差分规则、局部可观测性、故障树。图像抽查
- **F080**，PDF 页 1、3、4：2025-11-12：全局蕴含、时序测试、scan。图像抽查
- **F096**，PDF 页 1、4、6：Tutorial41125：MUX、布尔表示与 cube graph、QM 覆盖。图像抽查
- **F097**，PDF 页 1、4、7：Tutorial51125：resolution、启发式与 ESPRESSO 概念。图像抽查
- **F098**，PDF 页 1、4、7：Tutorial61125：共同 implicant、LUT 分解、shared ROBDD。图像抽查
- **F099**，PDF 页 1、3、5：Tutorial71125：FSM 最小化、SR latch 和三值 NAND。图像抽查
- **F094**，PDF 页 1、4、6：Tutorial101125：故障覆盖表与布尔差分。图像抽查
- **F095**，PDF 页 1、6、10：Tutorial111125：故障仿真、恒真/ROBDD 和综合练习。图像抽查
- **F100**，PDF 页 1、11、21：Week1 扫描笔记：MUX、ROBDD、三值逻辑。首/中/末图像抽查，正文无可提取文字。
- **F101**，PDF 页 1、9、17：Week2 扫描笔记：故障覆盖、D 算法、综合解题。首/中/末图像抽查，正文无可提取文字。
- **F085**，PDF 页 1、4、6：标 DMA22 的手写答案实际标题 Exam2023，与 F055 对应。首/中/末图像核对题型、年份、指定路径 D 算法。答案带手写注释，不自动视为无误。
- 未声称逐页通读全部 101 个文件；2025 正式讲义/习题和 5 场近年卷为主，手写/合订文件有明确抽样范围。
- 讲义公式中的反相横线、上下标与门图在文本提取时可能丢失，关键解法需以原页图形及真值表复核。
- 未取得本次补考正式通知，所以不能确认今年题型权重、允许资料、及格线或考试确切日期。
- 所有原始课程 PDF 保持本地，仅发布原创中文讲解、题型索引与来源说明；公开站点不镜像原卷、课程资料或个人批注。
- 外部技术来源提供机制和实现边界；不能用较新的语言标准参数覆盖课程使用的语义约定。